

```

    not_translatable_external_content,
    not_translated_external_content,
    translated_external_content,
    language_specific_content,
    http_file,
    http_class_directory,
    http_protocol);
  (*

```

NOTE Les schémas conceptuels ci-dessus référencés peuvent être consultés dans les documents suivants:

support_resource_schema	ISO 10303-41
person_organization_schema	ISO 10303-41
measure_schema	ISO 10303-41
ISO13584_IEC61360_language_resource_schema (qui est dupliqué pour commodité dans ce document)	CEI 61360-2
ISO13584_IEC61360_class_constraint_schema (qui est dupliqué pour commodité dans ce document)	CEI 61360-2
ISO13584_IEC61360_item_class_case_of_schema (qui est dupliqué pour commodité dans ce document)	CEI 61360-2
ISO13584_external_file_schema	ISO 13584-24:2003

5.4 Définitions de constantes

Le présent paragraphe contient des définitions de constantes utilisées dans les définitions de types (voir 5.11).

EXPRESS specification:

```

*)
CONSTANT
    dictionary_code_len: INTEGER := 131;
    property_code_len: INTEGER := 35;
    class_code_len: INTEGER := 35;
    data_type_code_len: INTEGER := 35;
    supplier_code_len: INTEGER := 149;
    version_len: INTEGER := 10;
    revision_len: INTEGER := 3;
    value_code_len: INTEGER := 35;
    pref_name_len: INTEGER := 255;
    short_name_len: INTEGER := 30;
    syn_name_len: INTEGER := pref_name_len;
    DET_classification_len: INTEGER := 3;
    source_doc_len: INTEGER := 255;
    value_format_len: INTEGER := 80;
    sep_cv: STRING := '#';
    sep_id: STRING := '#';
END_CONSTANT;
  (*

```

5.5 Identification d'un dictionnaire

Une entité **dictionary_identification** permet d'identifier sans ambiguïté une version particulière d'un dictionnaire particulier d'un fournisseur particulier d'informations, normalisée ou non. Elle contient un **code** défini par le fournisseur du dictionnaire qui identifie le

dictionnaire, un numéro de **version** et un numéro de **revision** qui caractérisent un état particulier de ce dictionnaire.

NOTE 1 La condition pour laquelle il convient d'incrémenter la version et la révision du dictionnaire est définie dans la CEI 61360-1.

EXPRESS specification:

```

*)
ENTITY dictionary_identification;
    code: dictionary_code_type;
    version: version_type;
    revision: revision_type;
    defined_by: supplier_bsu;
DERIVE
    absolute_id: identifier :=
        defined_by.absolute_id + sep_id + code + sep_cv +
version;
UNIQUE
    UR1: absolute_id;
END_ENTITY; -- dictionary_identification
(

```

Définitions des attributs:

code: le code qui caractérise le dictionnaire.

version: le numéro de version qui caractérise la version du dictionnaire.

revision: le numéro de révision qui caractérise la révision du dictionnaire.

defined_by: le fournisseur qui définit le dictionnaire.

absolute_id: l'identification unique du dictionnaire.

Propositions formelles:

UR1: l'identificateur de dictionnaire défini par l'attribut **absolute_id** est unique.

Propositions informelles:

IP1: lorsqu'un dictionnaire est défini par un document de norme qui contient un seul dictionnaire, le **code** du dictionnaire doit être le numéro normalisé du document qui décrit ce dictionnaire si ce document définit seulement un seul dictionnaire. Il doit être le nom défini pour le dictionnaire pertinent dans le document qui le décrit si ce document définit plusieurs dictionnaires. Sauf spécification contraire, la version doit être mise à 1 et les numéros de révision doivent être mis à 0 pour les dictionnaires définis par des documents de normes.

NOTE 2 La représentation des numéros de normes pour les documents de normes est spécifiée en 5.1 et 5.2 de l'ISO 13584-26:2000.

5.6 Basic Semantic Units (Unités sémantiques de base): définition et utilisation du dictionnaire

5.6.1 Exigences pour l'échange

Dans l'échange des données de dictionnaire et de bibliothèque de composants, il est de coutume de diviser les données. Par exemple, un dictionnaire pourrait être mis à jour avec certaines classes qui spécifient leur superclasse par référence à une classe préexistante, ou lorsque le contenu d'une bibliothèque est échangé, des éléments de dictionnaire sont seulement référencés et pas inclus chaque fois. Il doit être possible de se référer sans ambiguïté et de manière cohérente aux données du dictionnaire.

Ainsi, une exigence claire est, en premier lieu, d'être capable d'échanger des éléments de données et, en second lieu, d'avoir des relations entre ces éléments. Ceci est montré à la Figure 2.

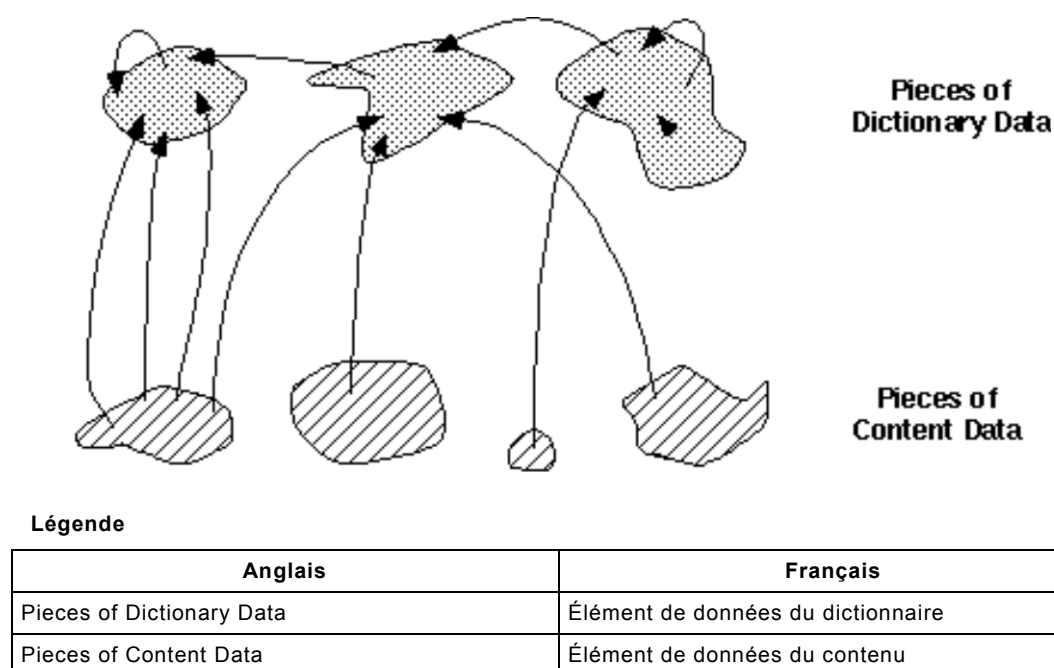


Figure 2 – Éléments de données avec relations

Chacun de ces éléments correspond à un fichier physique (conformément à l'ISO 10303-21). Les attributs EXPRESS (ISO 10303-11:2004) peuvent seulement contenir des références à des données au sein du même fichier physique. Il est donc impossible d'utiliser directement des attributs EXPRESS pour mettre en œuvre des références entre des éléments.

5.6.2 Architecture à trois niveaux des données du dictionnaire

5.6.2.1 Généralités

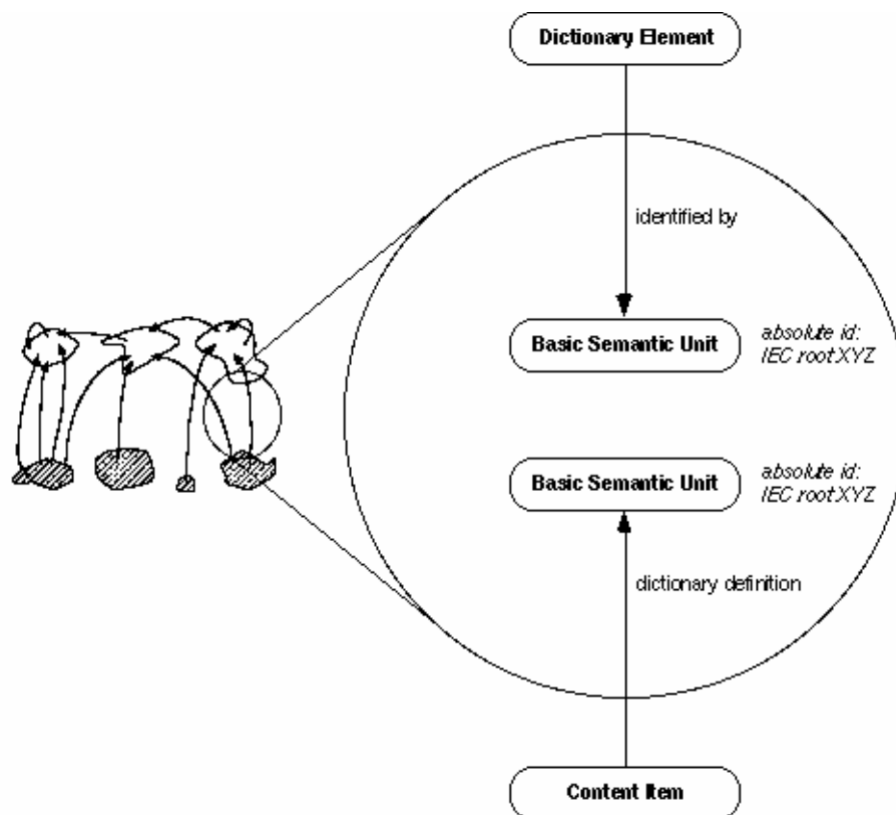
Dans le présent article, le concept de **basic_semantic_unit** (BSU) est introduit comme un moyen de mettre en œuvre ces références entre éléments. Une BSU fournit une identification universellement unique pour les descriptions du dictionnaire. Ceci est montré à la Figure 3.

Supposons qu'un certain élément du contenu souhaite se référer à une certaine description du dictionnaire.

EXEMPLE 1 Pour acheminer la valeur d'une propriété d'un composant.

Il le fait en se référant à une unité sémantique de base par le biais de l'attribut `dictionary_definition`.

Une description de dictionnaire (dictionary_element) se réfère à une unité sémantique de base par le biais de l'attribut identified_by. Une relation indirecte est établie à partir de la correspondance des identificateurs absolus des unités sémantiques de base.



Légende

Anglais	Français
Dictionary element	Élément du dictionnaire
identified by	identifié par
Basic Semantic Unit	Unité sémantique de base (BSU)
dictionary definition	définition du dictionnaire
Content Item	article du contenu
Absolute id	Identifiant absolu
IEC root XYZ	XYZ IEC racine

**Figure 3 – Mise en œuvre de relations "entre éléments"
à l'aide d'unités sémantiques de base**

Noter que :

- l'élément du dictionnaire et l'article du contenu peuvent être tous les deux présents dans le même fichier physique, sans que cela constitue une exigence;
- l'élément du dictionnaire peut ne pas être présent pour l'échange d'un certain article du contenu le référençant. Dans ce cas, il est supposé être déjà présent dans le dictionnaire du système cible. Réciproquement, des données du dictionnaire peuvent être échangées sans une quelconque donnée de contenu;
- l'unité sémantique de base peut être une seule instance dans le cas où les instances d'élément du dictionnaire et d'article du contenu sont situées dans le même fichier physique;
- le même mécanisme s'applique aussi aux références entre divers éléments du dictionnaire

EXEMPLE 2 Entre une classe de composants et les **property_DET** associés.

Une BSU fournit une référence à une description du dictionnaire partout où elle est nécessaire.

EXEMPLE 3 Livraison de dictionnaire, livraison de mise à jour, livraison de bibliothèque, échange des données composants.

Les données associées à une propriété pourraient être échangées sous la forme d'une paire (**property_BSU**, <value>).

La Figure 3 présente les grandes lignes de la mise en œuvre de ce mécanisme général.

5.6.2.2 Basic_semantic_unit

Une **basic_semantic_unit** est une identification unique d'un **dictionary_element**. BSU est l'abréviation de Basic Semantic Unit (Unité sémantique de base).

EXPRESS specification:

```

*)
ENTITY basic_semantic_unit
ABSTRACT SUPERTYPE OF (ONEOF(
    supplier_BSU,
    class_BSU,
    property_BSU,
    data_type_BSU,
    supplier_related_BSU,
    class_related_BSU));
code: code_type;
version: version_type;
DERIVE
    dic_identifieur: identifieur := code + sep_cv + version;
INVERSE
    definition: SET [0:1] OF dictionary_element
        FOR identified_by;
    referenced_by: SET [0:1] OF content_item
        FOR dictionary_definition;
END_ENTITY; -- basic_semantic_unit
(*)

```

Définitions des attributs:

code: le code assigné pour identifier un certain élément de dictionnaire.

version: le numéro de version d'un certain élément de dictionnaire.

dic_identifieur: l'identification complète, consistant en la concaténation de "code" et "version".

definition: une référence à l'élément de dictionnaire identifié par cette BSU. S'il est absent dans un certain contexte d'échange, il est supposé être déjà présent dans le dictionnaire du système-cible.

referenced_by: articles utilisant l'élément de dictionnaire associé à cette BSU.

5.6.2.3 Dictionary_element

Un **dictionary_element** est une définition complète des données qui doit être capturée dans le dictionnaire sémantique pour certains concepts. Pour chaque concept, un sous-type séparé doit être utilisé. Le **dictionary_element** est associé à une **basic_semantic_unit** (BSU) qui sert à identifier de façon univoque cette définition dans le dictionnaire.

En incluant l'attribut **version** dans l'entité **basic_semantic_unit**, elle est partie intégrante de l'identification d'un élément du dictionnaire (contrairement aux attributs **revision** et **time_stamps**).

EXPRESS specification:

```

*)
ENTITY dictionary_element
ABSTRACT SUPERTYPE OF (ONEOF(
    supplier_element,
    class_and_property_elements,
    data_type_element));
    identified_by: basic_semantic_unit;
    time_stamps: OPTIONAL dates;
    revision: revision_type;
    administration: OPTIONAL administrative_data;
    is_deprecated: OPTIONAL BOOLEAN;
    is_deprecated_interpretation: OPTIONAL note_type;
WHERE
    WR1: NOT EXISTS (SELF.is_deprecated)
        OR EXISTS (SELF.is_deprecated_interpretation);
END_ENTITY; -- dictionary_element
( *

```

Définitions des attributs:

identified_by: la BSU identifiant cet élément du dictionnaire.

time_stamps: les dates facultatives de création et de mise à jour de cet élément du dictionnaire.

revision: le numéro de révision de cet élément du dictionnaire.

NOTE 1 Le type de l'attribut **identified_by** sera redéfini ultérieurement en **property_BSU** et **class_BSU** et sera ensuite utilisé pour coder, conjointement à l'attribut "code" des BSU, l'attribut "Code" respectif pour les propriétés et les classes. Il sera aussi utilisé pour coder l'attribut "Version Number" (Numéro de version) respectif pour les propriétés et les classes.

NOTE 2 L'attribut **time_stamps** sera utilisé comme point de départ pour coder, dans l'entité **dates**, les attributs de propriétés et de classes "Date of Original Definition" (Date de définition d'origine), "Date of Current Version" (Date de version courante) et "Date of Current Revision" (Date de révision courante) (voir 5.11.3.2).

NOTE 3 L'attribut **revision** sera utilisé pour coder l'attribut de propriété et de classe "Revision Number" (Numéro de révision).

administration: information facultative relative au cycle de vie du **dictionary_element**.

NOTE 4 L'attribut **administration** sera utilisé pour représenter les informations relatives à la gestion de configuration et à l'historique des traductions.

is_deprecated: un booléen (Boolean) facultatif. Lorsqu'il est "true" (vrai), il spécifie que **dictionary_element** ne doit plus être utilisé.

is_deprecated_interpretation: spécifie la justification de la dépréciation et comment il convient d'interpréter les valeurs d'instances de l'élément déconseillé et celles de sa **BSU** correspondante.

Propositions formelles:

WR1: lorsque **is_deprecated** existe, **is_deprecated_interpretation** doit exister.

Propositions informelles:

IP1: les valeurs d'instance de l'élément **is_deprecated_interpretation** doivent être définies au moment où la décision de dépréciation a été prise.

La Figure 4 présente un modèle de planification de la relation entre l'unité sémantique de base et l'élément du dictionnaire.

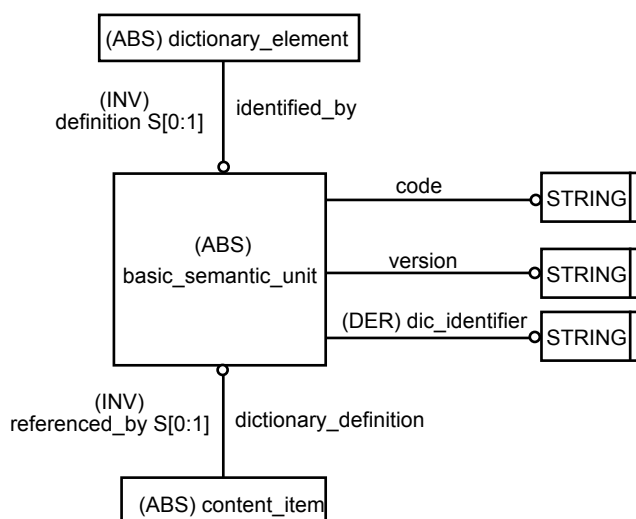


Figure 4 – Relation entre unité sémantique de base et élément de dictionnaire

5.6.2.4 Content_item

Un **content_item** est un élément de données renvoyant à sa description dans le dictionnaire. Il doit être sous-typé.

EXPRESS specification:

```

*)
ENTITY content_item
ABSTRACT SUPERTYPE;
    dictionary_definition: basic_semantic_unit;
END_ENTITY; -- content_item
( *

```

Définitions des attributs:

dictionary_definition: l'unité sémantique de base devant être utilisée pour se référer à la définition dans le dictionnaire.

5.6.3 Vue d'ensemble d'unités sémantiques de base et d'éléments de dictionnaire

Pour chaque sorte de données du dictionnaire, une paire de sous-types **basic_semantic_unit** et **dictionary_element** doit être définie. La Figure 5 présente, sous la forme d'un modèle de planification, les grandes lignes des unités sémantiques de base (BSU) et des éléments du dictionnaire définis ultérieurement. Noter que la relation entre BSU et éléments du dictionnaire est redéfinie pour chaque type de données et, de ce fait, seules des paires correspondantes peuvent être reliées. Cela n'est toutefois pas montré graphiquement ici.

Chaque catégorie de données du dictionnaire est traitée dans l'un des paragraphes suivants:

- pour les fournisseurs, voir 5.7;
- pour les classes, voir 5.8;
- pour les propriétés / types d'éléments de données, voir 5.9;
- pour les types de données, voir 5.10.

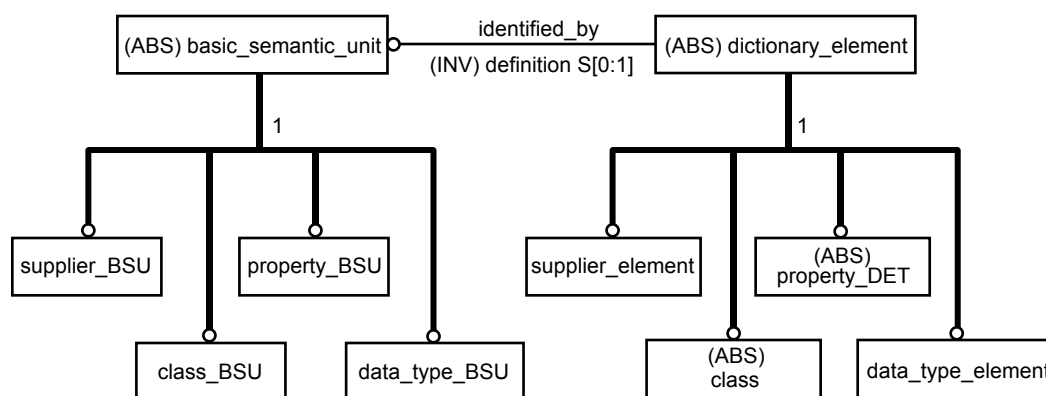


Figure 5 – BSU et éléments de dictionnaire courants

5.6.4 Identification d'éléments de dictionnaire: structure à trois niveaux

L'identification absolue des unités sémantiques de base repose sur la structure à trois niveaux ci-après:

- fournisseur (de données du dictionnaire);
- élément du dictionnaire défini par le fournisseur (tout élément du dictionnaire défini par le fournisseur qui est défini dans le modèle; dans le présent document, les éléments du dictionnaire définis par un fournisseur sont **property_DET** et **data_type_element**, mais il existe des dispositions pour étendre ce mécanisme à d'autres articles);
- version de l'élément du dictionnaire défini par le fournisseur.

Une identification absolue peut être obtenue par la concaténation du code applicable pour chaque niveau.

NOTE La structure sur cette identification absolue diffère de la structure définie dans l'édition 1 de la CEI 61360-2 (dupliquée pour la commodité dans l'ISO 13584-42:1998). Dans l'édition précédente, l'identification absolue de tout **dictionary_element** associé à un **name_scope** (y compris **property_DET** et **data_type_element**) consistait en: code fournisseur + code de classe (correspondant à la classe **name_scope**) + code d'élément du dictionnaire + version d'élément du dictionnaire. Dans la présente édition, le code de classe a été enlevé. Ainsi, le code d'élément du dictionnaire est unique, pour le même type d'élément du dictionnaire, sur toutes les classes définies par le même fournisseur. Pour les dictionnaires de référence existants, il convient que les organismes d'enregistrement, les autorités de maintenance ou les groupes de normalisation chargés des dictionnaires normalisés assurent cette unicité, en définissant éventuellement de nouveaux codes préfixés par les codes de classe **name_scope**.

Ce schéma d'identification est approprié dans le contexte multifournisseur. Si dans un certain domaine d'application, seules les données d'un seul fournisseur (de données) sont pertinentes, les parties correspondantes de cette identification, qui sont alors constantes, peuvent être éliminées. Pour les besoins d'échange, tous les niveaux doivent toutefois être présents, afin d'éviter des conflits d'identificateurs.

Ce schéma d'identification est formellement décrit dans l'attribut **absolute_id** des entités xxx_BSU définies de 5.7 à 5.12.

5.6.5 Possibilités d'extension pour d'autres types de données

5.6.5.1 Généralités

Le mécanisme BSU – élément de dictionnaire est très général et ne se limite pas aux quatre sortes de données utilisées ici (voir Figure 5). Le présent Article spécifie un certain nombre de moyens qui permettent des extensions d'autres sortes. Selon que le domaine d'application de l'identificateur est donné par une classe ou par un fournisseur, l'entité **xxx_related_BSU** correspondante doit être sous-typée. Il est nécessaire de redéfinir l'attribut **identified_by** de l'entité **dictionary_element** (comme cela est réalisé dans les paragraphes 5.7.3 à 5.10 ou dans les catégories de données courantes).

5.6.5.2 Supplier_related_BSU

Le **supplier_related_BSU** offre une prise en charge pour que les éléments de dictionnaire soient associés à des fournisseurs.

EXEMPLE Pour la série ISO 13584: bibliothèques de programmes.

EXPRESS specification:

```
*)
ENTITY supplier_related_BSU
ABSTRACT SUPERTYPE
SUBTYPE OF(basic_semantic_unit);
END_ENTITY; -- supplier_related_BSU
(*
```

5.6.5.3 Class_related_BSU

Le **class_related_BSU** offre une prise en charge pour que les éléments de dictionnaire soient associés à des classes.

EXEMPLE Pour les tableaux ISO 13584, documents, etc.

EXPRESS specification:

```
*)
ENTITY class_related_BSU
ABSTRACT SUPERTYPE
SUBTYPE OF(basic_semantic_unit);
END_ENTITY; -- class_related_BSU
(*
```

5.6.5.4 Supplier_BSU_relationship

Le **supplier_BSU_relationship** est une disposition permettant l'association des BSU à des fournisseurs.

EXPRESS specification:

```

*)
ENTITY supplier_BSU_relationship
ABSTRACT SUPERTYPE;
    relating_supplier: supplier_element;
    related_tokens: SET [1:?] OF supplier_related_BSU;
END_ENTITY; -- supplier_BSU_relationship
( *

```

Définitions des attributs:

relating_supplier: le **supplier_element** qui identifie le fournisseur de données.

related_tokens: l'ensemble d'éléments de dictionnaire associés au fournisseur identifié par l'attribut **relating_supplier**.

5.6.5.5 Class_BSU_relationship

L'entité **class_BSU_relationship** est une disposition permettant l'association des BSU à des classes.

EXPRESS specification:

```

*)
ENTITY class_BSU_relationship
ABSTRACT SUPERTYPE;
    relating_class: class;
    related_tokens: SET [1:?] OF class_related_BSU;
END_ENTITY; -- class_BSU_relationship
( *

```

Définitions des attributs:

relating_class: l'attribut **class** qui identifie l'élément de dictionnaire.

related_tokens: l'ensemble des éléments de dictionnaire associés à la classe identifiée par l'attribut **relating_class**.

5.7 Données fournisseur

5.7.1 Généralités

Le présent article contient des définitions pour la représentation des données relatives au fournisseur lui-même. Dans un environnement multifournisseur, il est nécessaire de pouvoir identifier la source d'un certain élément du dictionnaire. La Figure 6 présente un modèle de planification des données associées à des fournisseurs, suivi de la définition EXPRESS.

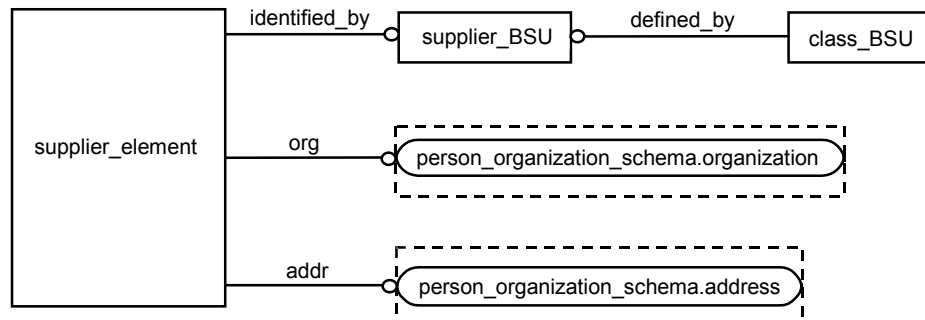


Figure 6 – Vue d'ensemble des données fournisseur et des relations

5.7.2 Supplier_BSU

L'entité **supplier_BSU** permet la prise en charge de l'identification unique des fournisseurs d'informations.

EXPRESS specification:

```

*)
ENTITY supplier_BSU
SUBTYPE OF(basic_semantic_unit);
    SELF\basic_semantic_unit.code: supplier_code_type;
DERIVE
    SELF\basic_semantic_unit.version: version_type := '1';
    absolute_id: identifier := SELF\basic_semantic_unit.code;
UNIQUE
    UR1: absolute_id;
END_ENTITY; -- supplier_BSU
(*
  
```

Définitions des attributs:

code: le code fournisseur attribué conformément à l'ISO 13584-26.

version: le numéro de version d'un code fournisseur doit être égal à 1.

absolute_id: l'identification absolue du fournisseur.

Propositions formelles

UR1: l'identificateur du fournisseur défini par l'attribut **absolute_id** est unique.

5.7.3 Supplier_element

L'entité **supplier_element** donne la description des fournisseurs du dictionnaire.

EXPRESS specification:

```

*)
ENTITY supplier_element
SUBTYPE OF(dictionary_element);
    SELF\dictionary_element.identified_by: supplier_BSU;
    org: organization;
  
```

```

        addr: address;
    INVERSE
        associated_items: SET [0:?] OF supplier_BSU_relationship
            FOR relating_supplier;
    END_ENTITY; -- supplier_element
    (*

```

Définitions des attributs:

identified_by: l'entité **supplier_BSU** utilisée pour identifier ce **supplier_element**.

org: les données organisationnelles de ce fournisseur.

addr: l'adresse de ce fournisseur.

associated_items: permet l'accès à d'autres sortes de données par le biais du mécanisme de BSU.

EXEMPLE Bibliothèque de programmes dans l'ISO 13584-24:2003.

5.8 Données de classe

5.8.1 Généralités

Le présent article contient des définitions pour la représentation des données de classe du dictionnaire.

Le Figure 7 présente, sous la forme d'un modèle de planification, les grandes lignes des données associées aux classes et leur relation à d'autres éléments du dictionnaire.

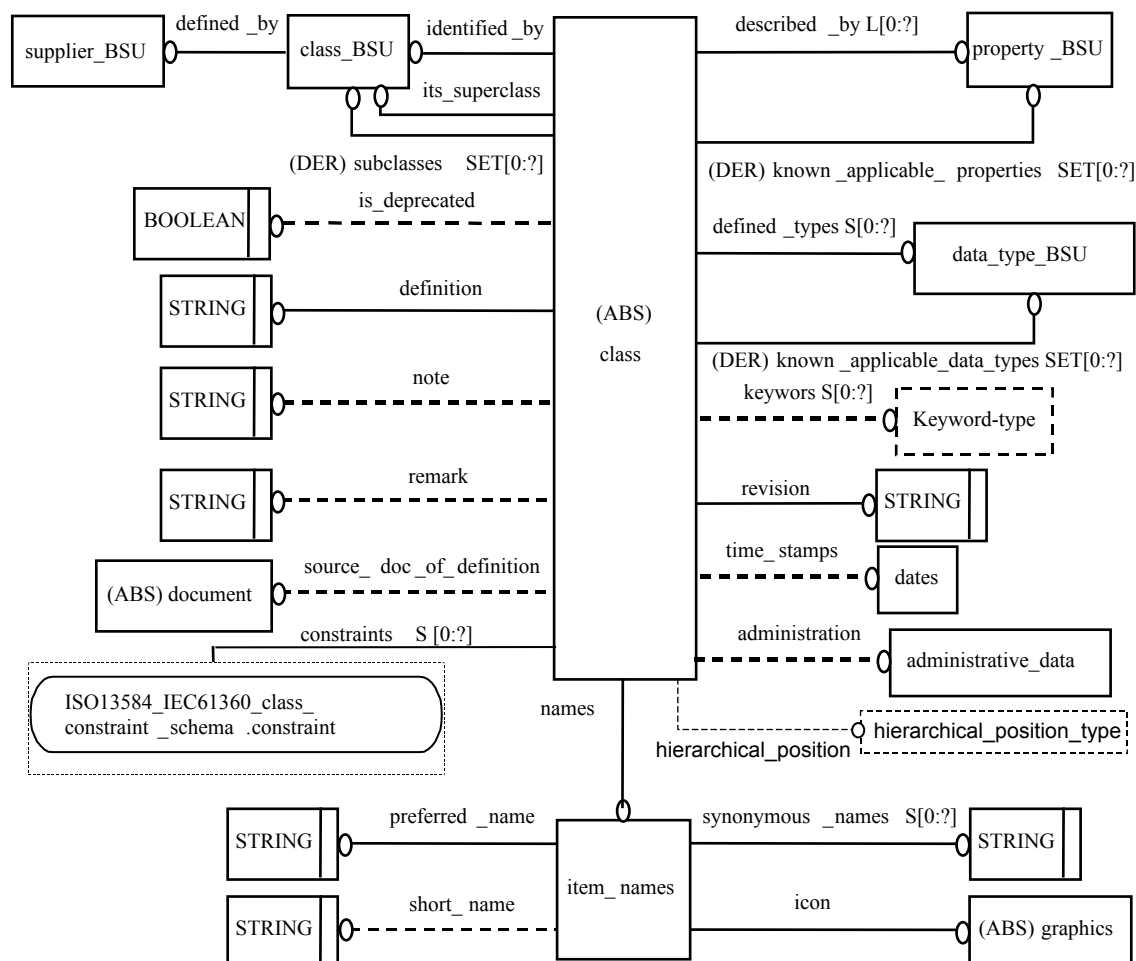


Figure 7 – Vue d'ensemble des données de classe et de leurs relations

Comme indiqué à la Figure 7 avec l'attribut **its_superclass**, les classes forment un arbre d'héritage. Il est important de noter que tout au long du présent document, les termes "héritage" et "hériter" représentent cette relation entre classes (définies dans le dictionnaire), bien que le langage EXPRESS ait aussi un concept d'héritage. Ils doivent être distingués de façon claire et nette pour éviter les incompréhensions.

Les données du dictionnaire pour les classes (telles que montrées à la Figure 7) sont étalées sur trois niveaux d'héritage:

- **class_and_property_elements** définit les données communes aux classes et aux **property_DET**;
- **class** permet que d'autres sortes de classes soient spécifiées ultérieurement;

EXEMPLE D'autres sous-types de **class**, en particulier **functional_view_class**, **functional_model_class** et **fm_class_view_of**, sont spécifiés dans l'ISO 13584-24:2003. Ils ne caractérisent pas les produits, mais ils permettent la prise en charge de l'échange des représentations produit particulières (par exemple: des représentations géométriques).

- **item_class** et **categorization_class** sont les entités qui contiennent les données de différentes classes d'objets du domaine d'application.

NOTE 1 Deux sous-types de **item_class**, appelés **component_class** et **material_class**, avaient été définis dans le modèle de dictionnaire de la première édition de la présente partie de CEI 61360. Ces sous-types sont déconseillés et ils ont été retirés de la présente partie de la CEI 61360.

NOTE 2 Les changements suivants assurent que les définitions de classe d'un dictionnaire conformes à la première édition de la 61360-2 se conforment à la présente édition: (1) remplacer **component_class** et **material_class** par **item_class** dans tout le dictionnaire de référence; (2) à chaque nouvelle **class item_class**, ajouter l'attribut **instance_sharable**, dont la valeur est **true**; (3) pour chaque nouvelle classe **item_class**, ajouter

l'attribut facultatif **hierarchical_position** sans attribuer de valeur; (4) pour chaque nouvelle classe **item_class**, ajouter l'attribut **keywords**, dont la valeur est une collection vide.

NOTE 3 Un autre sous-type of **item_class**, appelé **feature_class**, avait été fourni par l'ISO 13584-24:2003. Ce sous-type est également déconseillé et son utilisation est interdite dans les nouvelles mises en œuvre de la présente partie de la CEI 61360.

NOTE 4 Les changements suivants assurent que les définitions de classe d'un dictionnaire conformes à l'ISO 13584-25 se conforment à la présente partie de la CEI 61360: (1) remplacer **feature_class** par **item_class** dans tout le dictionnaire de référence; (2) à chaque nouvelle classe **item_class**, ajouter l'attribut **instance_sharable**, dont la valeur est false; (3) pour chaque nouvelle classe **item_class**, ajouter l'attribut facultatif **hierarchical_position** sans attribut de valeur; (4) pour chaque nouvelle classe **item_class**, ajouter l'attribut **keywords**, dont la valeur est une collection vide.

5.8.2 Détail de structure

5.8.2.1 Class_BSU

L'entité **class_BSU** permet la prise en charge de l'identification des classes.

EXPRESS specification:

```
*)
ENTITY class_BSU
SUBTYPE OF(basic_semantic_unit);
    SELF\basic_semantic_unit.code: class_code_type;
    defined_by: supplier_BSU;
DERIVE
    absolute_id: identifier
        := defined_by.absolute_id + sep_id + dic_identifier;
    known_visible_properties: SET [0:?] OF property_BSU
        := compute_known_visible_properties(SELF);
    known_visible_data_types: SET [0:?] OF data_type_BSU
        := compute_known_visible_data_types(SELF);
INVERSE
    subclasses: SET [0:?] OF class FOR its_superclass;
    added_visible_properties: SET [0:?] OF property_BSU
        FOR name_scope;
    added_visible_data_types: SET [0:?] OF data_type_BSU
        FOR name_scope;
UNIQUE
    UR1: absolute_id;
END_ENTITY; -- class_BSU
(*
```

Définitions des attributs:

code: le code affecté à cette classe par son fournisseur.

defined_by: le fournisseur définissant cette classe et son élément de dictionnaire.

absolute_id: l'identification unique de cette classe.

known_visible_properties: l'ensemble des **property_BSU** qui se réfèrent à la classe comme leur attribut **name_scope** (portée de nom) ou à toute superclasse connue de cette classe et qui sont donc visibles pour la classe (et n'importe laquelle de ses sous-classes)

NOTE 1 Lorsque l'attribut **dictionary_definition** d'une certaine classe est absent dans le même contexte d'échange (un contexte d'échange PLIB n'est jamais supposé être complet), la superclasse d'une classe peut ne

pas être connue. Par conséquent, les propriétés définies comme étant visibles par cette superclasse n'appartiennent pas à l'attribut **known_visible_properties**. C'est seulement pour le système de réception que tous les attributs **dictionary_definition** des BSU doivent être disponibles. Par conséquent, sur le système de réception, l'attribut **known_visible_properties** contient toutes les propriétés visibles pour la classe.

known_visible_data_types: l'ensemble des **data_type_BSU** qui se réfèrent à la classe comme leur attribut **name_scope** ou à toute superclasse connue de cette classe et qui sont donc visibles pour la classe (et n'importe laquelle de ses sous-classes)

NOTE 2 Lorsque l'attribut **dictionary_definition** d'une certaine classe est absent dans le même contexte d'échange (un contexte d'échange PLIB n'est jamais supposé être complet), la superclasse d'une classe peut ne pas être connue. Par conséquent, les entités **data_type** définies comme étant visibles par cette superclasse n'appartiennent pas à l'attribut **known_visible_data_types**. C'est seulement pour le système de réception que tous les attributs **dictionary_definition** des BSU doivent être disponibles. Par conséquent, sur le système de réception, l'attribut **known_visible_data_types** contient toutes les entités **data_type** visibles pour la classe.

subclasses: l'ensemble des classes spécifiant cette classe comme étant leur superclasse.

added_visible_properties: l'ensemble des entités **property_BSU** qui se réfèrent à la classe comme étant leur **name_scope** et qui sont donc visibles pour cette classe (et n'importe laquelle de ses sous-classes).

NOTE 3 Seuls les entités **property_BSU** qui appartiennent au même contexte d'échange sont référencées par cet attribut inverse. Sur le côté de réception, il peut déjà exister d'autres **property_BSU** qui se réfèrent à cette classe (un contexte d'échange PLIB n'est jamais supposé être complet).

NOTE 4 L'attribut **added_visible_properties** sera utilisé pour coder l'attribut de classe "Visible properties" (Propriétés visibles).

added_visible_data_types: l'ensemble des entités **data_type_BSU** qui se réfèrent à la classe comme étant leur **name_scope** et qui sont donc visibles pour cette classe (et n'importe laquelle de ses sous-classes).

NOTE 5 Seuls les entités **data_type_BSU** qui appartiennent au même contexte d'échange sont référencées par cet attribut inverse. Sur le côté de réception, il peut déjà exister d'autres **data_type_BSU** qui se réfèrent à cette classe (un contexte d'échange PLIB n'est jamais supposé être complet).

NOTE 6 L'attribut **added_visible_data_types** sera utilisé pour coder l'attribut de classe "Visible types" (Types visibles).

Propositions formelles

UR1: la concaténation du code fournisseur et du code de classe est unique.

5.8.2.2 Class_and_property_elements

L'entité **class_and_property_elements** capture les attributs qui sont communs aux **class** et aux **property_DET**.

EXPRESS specification:

```
*)
ENTITY class_and_property_elements
ABSTRACT SUPERTYPE OF(ONEOF(
    property_DET,
    class))
SUBTYPE OF(dictionary_element);
    names: item_names;
    definition: definition_type;
    source_doc_of_definition: OPTIONAL document;
    note: OPTIONAL note_type;
    remark: OPTIONAL remark_type;
```

```
END_ENTITY; -- class_and_property_elements
(*
```

Définitions des attributs:

names: les noms décrivant cet élément du dictionnaire.

definition: le texte décrivant cet élément du dictionnaire.

source_doc_of_definition: le document source de cette description textuelle.

note: informations supplémentaires sur n'importe quelle partie d'élément du dictionnaire, qui sont essentielles à la compréhension.

remark: texte explicatif clarifiant davantage la signification de cet élément du dictionnaire.

NOTE 1 L'attribut **names** sera utilisé comme un point de départ pour coder dans l'entité **item_names** les attributs de propriété et de classe "Preferred Name" (Nom préférentiel), "Short Name" (Nom court) et "Synonymous Name" (Synonyme).

NOTE 2 L'attribut **definition** sera utilisé pour coder l'attribut propriété "Definition" (Définition) et l'attribut classe "Definition" (Définition).

NOTE 3 L'attribut **source_of_doc_definition** sera utilisé pour coder l'attribut propriété "Source document of definition" (Document de définition source) et l'attribut classe "Source document of definition".

NOTE 4 L'attribut **note** sera utilisé pour coder l'attribut propriété et classe "Note".

NOTE 5 L'attribut **remark** sera utilisé pour coder l'attribut propriété et classe "Remark" (Remarque).

5.8.2.3 Class

L'entité **class** est une ressource abstraite pour toutes les sortes de classes.

EXPRESS specification:

```
*)
ENTITY class
ABSTRACT SUPERTYPE OF( ONEOF (item_class, categorization_class))
SUBTYPE OF(class_and_property_elements);
    SELF\dictionary_element.identified_by: class_BSU;
    its_superclass: OPTIONAL class_BSU;
    described_by: LIST [0:?] OF UNIQUE property_BSU;
    defined_types: SET [0:?] OF data_type_BSU;
    constraints: SET [0:?] OF constraint_or_constraint_id;
    hierarchical_position: OPTIONAL hierarchical_position_type;
    keywords: SET [0:?] OF keyword_type;
    sub_class_properties: SET [0:?] OF property_BSU;
    class_constant_values: SET [0:?] OF class_value_assignment;
DERIVE
    subclasses: SET [0:?] OF class := identified_by.subclasses;
    known_applicable_properties: SET [0:?] OF property_BSU
        := compute_known_applicable_properties(
            SELF\dictionary_element.identified_by);
    known_applicable_data_types: SET [0:?] OF data_type_BSU
        := compute_known_applicable_data_types(
            SELF\dictionary_element.identified_by);
    known_property_constraints: SET [0:?] OF property_constraint
```



```

:= compute_known_property_constraints(
    [SELF\dictionary_element.identified_by]);
INVERSE
    associated_items: SET [0:?] of class_BSU_relationship
        FOR relating_class;
WHERE
    WR1: acyclic_superclass_relationship(SELF.identified_by, []);
    WR2: NOT all_class_descriptions_reachable(
        SELF\dictionary_element.identified_by)
        OR (list_to_set(SELF.described_by) <=
            SELF\dictionary_element.identified_by
            \class_BSU.known_visible_properties);
    WR3: NOT all_class_descriptions_reachable(
        SELF\dictionary_element.identified_by)
        OR (SELF.defined_types <=
            SELF\dictionary_element.identified_by
            \class_BSU.known_visible_data_types);
    WR5: NOT all_class_descriptions_reachable(
        SELF\dictionary_element.identified_by)
        OR (QUERY (cdp <* described_by
            | (SIZEOF (cdp\basic_semantic_unit.definition)=1)
            AND (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
                +'.DEPENDENT_P_DET') IN TYPEOF
                (cdp\basic_semantic_unit.definition[1]))
            AND NOT
                (cdp\basic_semantic_unit.definition[1].depends_on
                <= known_applicable_properties))=[]);
    WR6: check_datatypes_applicability(SELF);
    WR7: QUERY (cons <* constraints
        | ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
            +'.INTEGRITY_CONSTRAINT' IN TYPEOF (cons))
        AND
        (cons\property_constraint.constrained_property
            .definition) =1)
        AND NOT correct_constraint_type(
            cons\integrity_constraint.redefined_domain,
            cons\property_constraint.constrained_property
            .definition[1].domain)) = [];
    WR8: QUERY (cons <* constraints
        | (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
            +'.CONFIGURATION_CONTROL_CONSTRAINT') IN TYPEOF (cons))
        AND NOT correct_precondition (cons, SELF)) = [];
    WR9: NOT all_class_descriptions_reachable(
        SELF\dictionary_element.identified_by)
        OR (QUERY (cons <* constraints
            | (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
                +'.PROPERTY_CONSTRAINT') IN TYPEOF (cons))
            AND NOT
                ((cons\property_constraint.constrained_property
                IN SELF\dictionary_element.identified_by
                \class_BSU.known_visible_properties)
                OR (cons\property_constraint.constrained_property
                IN known_applicable_properties)))=[]);
    WR10: (SIZEOF( QUERY (lab <* keywords

```

```

        | ('ISO13584_IEC61360_DICTIONARY_SCHEMA'
        + '.LABEL_WITH_LANGUAGE') IN TYPEOF (lab)))
    = SIZEOF( keywords))
    OR (SIZEOF (QUERY (lab <* keywords
    | ('ISO13584_IEC61360_DICTIONARY_SCHEMA'
    + '.LABEL_WITH_LANGUAGE') IN TYPEOF (lab)))
    = SIZEOF( keywords));
WR11: (('ISO13584_IEC61360_ITEM_CLASS_CASE_OF_SCHEMA'
+ '.A_PRIORI_SEMANTIC_RELATIONSHIP')
IN TYPEOF (SELF)) OR
( QUERY(p <* sub_class_properties
| NOT(p IN SELF.described_by)) = []);
WR12: NOT all_class_descriptions_reachable(SELF.identified_by)
OR
(QUERY(va <* class_constant_values |
NOT is_class_valued_property(
va.super_class_defined_property, SELF.identified_by)) =
[]);
WR13: QUERY(val <* SELF.class_constant_values
| QUERY (v <* class_value_assigned (
val.super_class_defined_property, SELF.identified_by)
| val.assigned_value <> v) <>[]) = [];
END_ENTITY; -- class
(*

```

Définitions des attributs:

identified_by: l'entité **class_BSU** identifiant cette classe.

its_superclass: référence à la classe dont la classe courante est une sous-classe.

described_by: la liste des références à des propriétés complémentaires disponibles à l'utilisation dans la description des produits au sein d'une classe, et de n'importe laquelle de ses sous-classes.

NOTE 1 Une propriété peut également être applicable à une classe lorsque cette propriété est importée d'une autre classe par le biais d'une entité **a_priori_semantic_relationship** telle que définie en 8.5 de la présente partie de la CEI 61360. Par conséquent, les propriétés référencées par l'attribut **described_by** ne définissent pas toutes les propriétés applicables pour cette classe.

NOTE 2 L'ordre de la liste est l'ordre de présentation conseillé par le fournisseur.

NOTE 3 Une propriété qui est une propriété dépendante d'un contexte (**context_dependent_P_DET**) peut devenir applicable à une classe seulement si tous les paramètres de contexte (**condition_DET**) dont dépend sa valeur sont également applicables à cette classe. Cela est énoncé dans la règle "WHERE" 5 (WR5).

defined_types: l'ensemble des références aux types qui peuvent être utilisés pour les divers **property_DET** dans tout l'arbre d'héritage descendant de cette classe.

NOTE 4 Une entité **data_type** peut également être applicable à une classe lorsque cette entité **data_type** est importée d'une autre classe par le biais d'une entité **a_priori_semantic_relationship** telle que définie en 8.5 de la présente partie de la CEI 61360. Par conséquent, les types de données référencés par l'attribut **described_types** ne définissent pas tous les types de données applicables pour cette classe.

constraints: l'ensemble des contraintes qui limitent les domaines cibles des valeurs de certaines propriétés de la classe à certains sous-ensembles de leurs domaines hérités de valeurs.

NOTE 5 Il convient que chaque contrainte dans l'attribut **constraints** soit remplie par les instances de classe. Ainsi, l'attribut **constraints** est donc une conjonction de contraintes.

hierarchical_position: la représentation codée de la position d'une classe dans la hiérarchie d'inclusion des classes à laquelle elle appartient; une **hierarchical_position** d'une classe change lorsque la structure des classes d'une ontologie change. Par conséquent, elle ne peut pas servir d'identificateur stable pour des classes.

NOTE 6 Cette sorte de nom codé est utilisée en particulier dans des hiérarchies de catégorisation de produits pour représenter la structure d'inclusion des classes à travers un certain nombre de conventions de codage.

EXEMPLE 1 Dans l'UNSPSC, *Manufacturing Components and Supplies* (Composants et fournitures de fabrication) a la position hiérarchique 31000000, *Hardware* (Équipement matériel) a la position hiérarchique 31160000 et *Bolt* (Boulon) a la position hiérarchique 31161600. Par convention, cette représentation de la position hiérarchique permet de déclarer que *Manufacturing Components and Supplies* est au premier niveau de la hiérarchie, que *Hardware* est au deuxième niveau de la hiérarchie et est inclus dans *Manufacturing Components and Supplies* et que *Bolt* est au troisième niveau de la hiérarchie et est inclus dans *Hardware*.

keywords: un ensemble de mots clés, éventuellement en plusieurs langues, permettant d'effectuer des recherches dans la classe.

sub_class_properties: déclare les propriétés de valeur d'une classe, c'est-à-dire que dans les sous-classes, une seule valeur sera assignée par classe. Voir 5.9.6.

class_constant_values: affectations dans la classe courante pour les propriétés de valeur déclarées d'une classe en des superclasses. Voir 5.9.6.

subclasses: l'ensemble de classes spécifiant cette classe comme étant leur superclasse

known_applicable_properties: Les **property_BSU** qui sont référencées par la classe, ou n'importe laquelle de sa/ses superclasse(s) connue(s) par leur attribut **described_by** et qui sont donc applicables à cette classe (et n'importe laquelle de ses sous-classes).

NOTE 7 Lorsque l'attribut **dictionary_definition** d'une certaine classe est absent dans le même contexte d'échange (un contexte d'échange PLIB n'est jamais supposé être complet), la superclasse d'une classe peut ne pas être connue. Par conséquent, les propriétés définies comme étant applicables par cette superclasse n'appartiennent pas à l'attribut **known_applicable_properties**. C'est seulement pour le système de réception que tous les attributs **dictionary_definition** des **BSU** doivent être disponibles. Par conséquent, sur le système de réception, l'attribut **known_applicable_properties** contient toutes les propriétés qui sont applicables à une classe en vertu du fait d'être référencées par un attribut **described_by**.

known_applicable_data_types: Les **data_type_BSU** qui sont référencées par la classe, ou n'importe laquelle de sa/ses superclasse(s) connue(s) par leur attribut **defined_types** et qui sont donc applicables à cette classe (et n'importe laquelle de ses sous-classes).

NOTE 8 Lorsque l'attribut **dictionary_definition** d'une certaine classe est absent dans le même contexte d'échange (un contexte d'échange PLIB n'est jamais supposé être complet), la superclasse d'une classe peut ne pas être connue. Par conséquent, les entités **data_type** définies comme étant applicables par cette superclasse n'appartiennent pas à l'attribut **known_applicable_data_types**. C'est seulement pour le système de réception que tous les attributs **dictionary_definition** des **BSU** doivent être disponibles. Par conséquent, sur le système de réception, l'attribut **known_applicable_data_types** contient toutes les **data_type** qui sont applicables à une classe en vertu du fait d'être référencées par un attribut **defined_types**.

known_property_constraints: les **constraints** sur une propriété qui sont référencées par la classe, ou par n'importe laquelle de ses superclasses "is-a" connues par leur attribut **constraints** ou, dans le cas d'une classe qui est un sous-type de **a_priori_semantic_relationship**, par son attribut **referenced_constraints**.

associated_items: permet d'accéder à d'autres sortes de données en utilisant le mécanisme de BSU.

Propositions formelles

WR1: la structure d'héritage définie par la hiérarchie des classes ne contient pas de boucles.

WR2: seules les propriétés qui sont visibles pour une classe peuvent devenir applicables à cette classe par le fait d'être référencées par l'attribut **described_by**.

WR3: seules les types de données qui sont visibles pour une classe peuvent devenir applicables à cette classe par le fait d'être référencés par l'attribut **defined_types**.

WR4: seules les propriétés qui ne sont pas applicables pour une classe par héritage peuvent devenir applicables à cette classe par le fait d'être référencées par l'attribut **described_by**.

WR5: seules les propriétés dépendantes d'un contexte (**dependent_P_DET**) dont tous les paramètres de contexte (**condition_DET**) sont applicables dans la classe peuvent devenir applicables pour cette classe par le fait d'être référencées par son attribut **described_by**.

WR6: seuls les types de données qui ne sont pas applicables pour une classe par héritage peuvent devenir applicables à cette classe par le fait d'être référencés par l'attribut **defined_types**.

NOTE 9 L'attribut **its_superclass** sera utilisé pour coder l'attribut "Superclass" (Superclasse) d'une classe.

NOTE 10 L'attribut **described_by** fournit le codage pour l'attribut "Applicable Properties" d'une classe.

NOTE 11 L'attribut **defined_types** est utilisé pour coder l'attribut "Applicable Types" (Types applicables) d'une classe.

WR7: l'ensemble des contraintes qui sont des contraintes de propriété doit définir des restrictions qui soient compatibles avec le domaine des valeurs des propriétés auxquelles elles s'appliquent.

WR8: toutes les propriétés référencées dans la précondition d'une **configuration_control_constraint** doivent être applicables à la classe.

WR9: toutes les propriétés référencées dans l'attribut **constraint** doivent être soit visibles, soit applicables à la classe.

WR10: soit tous les attributs **keywords** sont représentés comme étant des **label_with_languages**, soit ils sont tous représentés comme étant des **labels**.

WR11: Si **class** n'est pas une **a_priori_semantic_relationship**, **sub_class_properties** doit appartenir à la liste d'attributs **described_by**.

NOTE 12 Par le biais d'une **a_priori_semantic_relationship**, l'attribut **sub_class_properties** peut également être importé.

WR12: les propriétés référencées dans **class_constant_values** sont déclarées comme étant de valeur pour une classe dans une certaine superclasse de la classe courante ou dans la classe courante elle-même.

NOTE 13 L'attribut **sub_class_properties** de l'entité **class** est utilisé pour coder l'attribut "Class valued properties" (Propriétés valuées d'une classe) des classes.

NOTE 14 L'attribut **class_constant_values** de l'entité **class** est utilisé pour coder l'attribut "Class constant values" (Valeurs de constantes de classe) des classes.

WR13: Si une propriété référencée dans **class_constant_values** a déjà reçu une valeur qui lui est assignée dans une superclasse, il convient que la valeur assignée dans la classe courante soit la même.

Propositions informelles:

IP1: Si toutes les superclasses "is-a" de la classe sont disponibles, les **known_property_constraints** sont toutes les contraintes qui s'appliquent aux propriétés qui sont reliées à cette classe comme propriétés soit visibles, soit applicables.

5.8.3 Item_class

L'entité **item_class** permet la modélisation de tout type d'entité du domaine d'application qui peut être capturé par une classe de caractérisation définie par une structure de classe et un ensemble de propriétés. En particulier, les instances produits et aussi les instances aspects particuliers des produits représentées comme étant des caractéristiques intrinsèques sont mises en correspondances avec l'entité **item_class**.

L'entité **item_class** inclut un attribut **instance_sharable** qui spécifie le statut conceptuel de l'article. Si cet attribut est "true", chaque instance représente un article indépendant, autrement elle est une caractéristique intrinsèque, c'est-à-dire un article dépendant qui doit être le composant d'un autre article. Ceci ne prescrit aucune mise en œuvre spécifique au niveau de la représentation des données.

EXEMPLE La *tête d'une vis* est une caractéristique intrinsèque décrite par un nombre de propriétés, mais elle peut exister seulement lorsqu'elle est référencée par une vis. Elle est représentée comme étant une **item_class** avec l'attribut **instance_sharable** égal à *false*.

NOTE 1 Deux sous-types de **item_class**, appelés **component_class** et **material_class**, avaient été définis dans le modèle de dictionnaire de la première édition de l'ISO 13584-42 et celle de la CEI 61360-2. Ces sous-types sont déconseillés et ils ont été retirés de la présente édition de la CEI 61360.

NOTE 2 Un autre sous-type of **item_class**, appelé **feature_class**, avait été fourni par l'ISO 13584-24:2003. Ce sous-type est également déconseillé et son utilisation n'est pas recommandée dans les nouvelles mises en œuvre de la présente partie de la CEI 61360.

EXPRESS specification:

```
*)
ENTITY item_class
  SUBTYPE OF(class);
    simplified_drawing: OPTIONAL graphics;
    coded_name: OPTIONAL value_code_type;
    instance_sharable: OPTIONAL BOOLEAN;
END_ENTITY; -- item_class
(*
```

Définitions des attributs:

simplified_drawing: dessins (**graphics**) facultatifs qui peuvent être associés à la classe décrite.

NOTE 3 L'attribut **simplified_drawing** de l'entité **item_class** est utilisé pour coder l'attribut "Simplified Drawing" (Dessin simplifié) pour des classes.

coded_name: peut être utilisé comme une valeur de constante de classe pour caractériser la classe dans un domaine de valeurs d'un attribut **sub_class_properties** de sa superclasse.

NOTE 4 Cet attribut n'est pas utilisé dans la série ISO 13584. Il est seulement utilisé dans la série CEI 61360.

instance_sharable: lorsqu'il est "false", il spécifie que les instances de **item_class** sont des caractéristiques intrinsèques; lorsqu'il n'est pas donné ou est "true", il spécifie que les instances de **item_class** sont des articles autonomes.

NOTE 5 Dans le modèle de dictionnaire commun ISO13584/CEI61360, c'est la mise en œuvre qui permet de décider si plusieurs instances en monde réel de caractéristiques intrinsèques modélisées par le même ensemble de paires "propriété-valeur" sont représentées par plusieurs éléments de données EXPRESS ou par le même élément de données dans le fichier d'échange de données. Ainsi, une instance de **item_class** dont l'attribut **instance_sharable** est égal à *false* et qui est référencée par plusieurs instances de **item_class** au niveau de modèle de données est interprétée comme plusieurs instances en monde réel de la même caractéristique intrinsèque.

5.8.4 Categorization_class

L'entité **categorization_class** permet la modélisation d'un regroupement d'un ensemble d'objets qui constitue un élément d'une catégorisation.

EXEMPLE 1 "Composants de fabrication et fournitures", et "Optique industrielle" sont des exemples de classes catégorisation de produits définies dans l'UNSPSC.

Ni les propriétés ni les datatypes ni les contraintes ne sont associés, comme étant visibles ou applicables, à une telle classe. En outre, les **categorization_class** ne peuvent pas être reliées les unes aux autres par la relation d'héritage "is-a", mais elles peuvent seulement être reliées les unes aux autres par la relation de classes "is-case-of". Un attribut spécifique, appelé **categorization_class_superclasses**, permet d'enregistrer les **categorization_class** qui sont des superclasses de **categorization_class** dans une hiérarchie "case-of".

NOTE En utilisant les constructions de ressources "case-of", des **item_class** peuvent également être reliées à des **categorization_class**.

EXEMPLE 2 L'exemple ci-après montre comment classes de caractérisation et classes de catégorisation peuvent être reliées pour atteindre un certain nombre d'objectifs particuliers. Un fournisseur de roulements à billes souhaite concevoir sa propre ontologie et la rendre facile à récupérer et à utiliser. Pour atteindre ces objectifs, il souhaite utiliser des propriétés normalisées et être relié à des classifications normalisées. Le fournisseur fournit seulement des roulements à billes, mais certains roulements sont scellés, d'autres non. Des propriétés particulières peuvent être associées aux roulements scellés et aux roulements non scellés, mais ces catégories n'existent pas en tant que classes dans les ontologies des roulements normalisées. Donc, le fournisseur de roulements procède comme suit. (1) Il conçoit une ontologie propriétaire constituée de trois classes de caractérisation: *my_bearings* (mes roulements), *my_sealed_bearings* (mes roulements scellés), *my_non_sealed_bearing* (mes roulements non scellés). Les deux dernières classes sont reliées à la première par la relation d'héritage "is-a" et toutes les propriétés assignées à la première classe sont héritées par la dernière. (2) Pour utiliser certaines des propriétés définies dans l'ISO/TS 23768-1 (à publier), le fournisseur de roulements spécifie que sa classe *my_bearings* est "case-of" la classe normalisée de roulements *ball bearing* définies dans l'ISO/TS 23768-1 (à publier). Par cette relation "case-of", il peut importer dans sa classe *my_bearings* les propriétés définies par la norme: *bore diameter* (diamètre d'alésage), *outside diameter* (diamètre extérieur), *ISO tolerance class* (classe de tolérance ISO). De plus, il créait les propriétés recherchées qui ne sont pas définies dans la norme. (3) Pour faciliter la récupération du serveur qui affiche le catalogue fournisseur, il expose un petit fragment de la classification UNSPSC et une relation "case-of" entre la classe *ball_bearings* de l'UNSPSC et sa propre classe *my_bearing*. Le résultat est présenté à la Figure 8 ci-dessous:

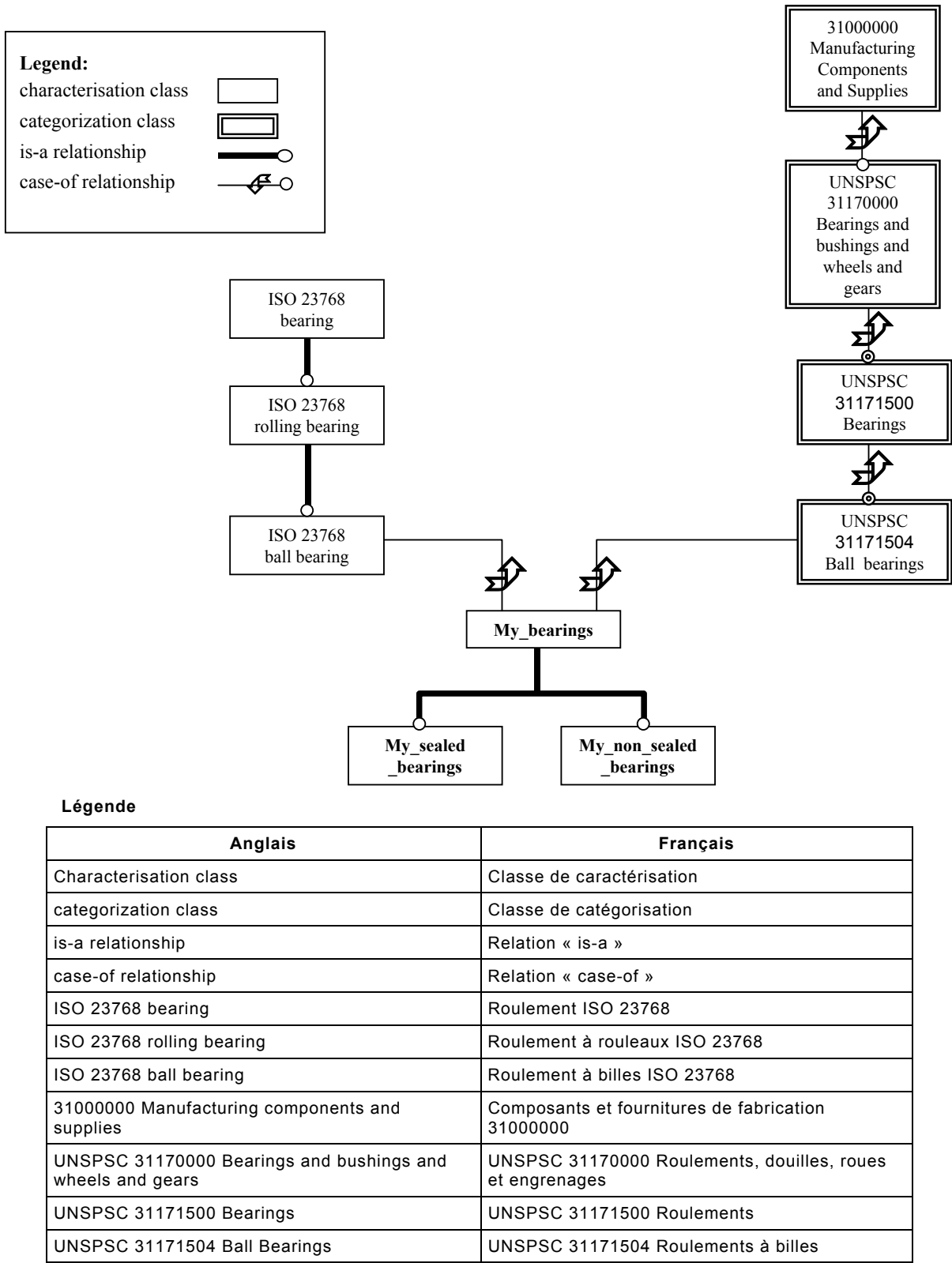


Figure 8 – Exemple d'ontologie fournisseur

EXPRESS specification:

```
*)
ENTITY categorization_class
SUBTYPE OF(class);
    categorization_class_superclasses: SET [0:?] of class_BSU;
```



```

WHERE
  WR1: QUERY (cl <* SELF. categorization_class_superclasses
    | NOT (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
    +'.CATEGORIZATION_CLASS') IN TYPEOF(cl.definition[1])))
    = [];
  WR2: NOT EXISTS(SELF\class.its_superclass);
  WR3: SIZEOF(SELF\class.described_by) = 0;
  WR4: SIZEOF(SELF\class.defined_types) = 0;
  WR5: SIZEOF(SELF\class.constraints) = 0;
  WR6: SIZEOF(compute_known_visible_properties
    (SELF\dictionary_element.identified_by)) = 0;
  WR7: SIZEOF(SELF\class.sub_class_properties) = 0;
  WR8: SIZEOF(SELF\class.class_constant_values) = 0;
  WR9: SIZEOF(SELF\class.identified_by.known_visible_properties)
    = 0;
  WR10:
    SIZEOF(SELF\class.identified_by.known_visible_data_types)
      = 0;

END_ENTITY; -- categorization_class
(*)

```

Définitions des attributs:

categorization_class_superclasses: les **categorization_class** qui sont supérieures d'un niveau à la classe de catégorisation dans une hiérarchie de classe "case-of".

Propositions formelles

WR1: seules des **categorization_class** peuvent apparaître comme étant des superclasses d'une entité **categorization_class**.

WR2: une **categorization_class** ne doit pas avoir de superclasse "is-a".

WR3: aucune propriété ne doit être associée à **categorization_class**.

WR4: aucun datatype ne doit être associé à **categorization_class**.

WR5: aucune contrainte ne doit être associée à **categorization_class**.

WR6: une **categorization_class** ne doit pas être la classe définition de propriétés pour une quelconque propriété.

WR7: aucune propriété de sous-classe ne doit être associée à **categorization_class**.

WR8: aucune valeur de constante de classe ne doit être associée à **categorization_class**.

WR9: aucune propriété visible ne doit être associée à **categorization_class**.

WR10: aucun datatype visible ne doit être associé à **categorization_class**.

5.9 Type de données d'un élément / propriétés des données

5.9.1 Généralités

Le présent article contient des définitions des propriétés pour les données du dictionnaire.

5.9.2 Property_BSU

L'entité **property_BSU** permet la prise en charge de l'identification d'une propriété.

EXPRESS specification:

```

*)
ENTITY property_BSU
SUBTYPE OF(basic_semantic_unit);
    SELF\basic_semantic_unit.code: property_code_type;
    name_scope: class_BSU;
DERIVE
    absolute_id: identifier :=
        name_scope.defined_by.absolute_id
        + sep_id + dic_identifier;
INVERSE
    describes_classes: SET OF class FOR described_by;
UNIQUE
    UR1: absolute_id;
WHERE
    WR1: QUERY(c <* describes_classes |
        NOT(is_subclass(c, name_scope.definition[1]))= []);
END_ENTITY; -- property_BSU
(*

```

Définitions des attributs:

code: pour permettre l'identification unique de la propriété sur toutes les ontologies définies par le même fournisseur **name_scope.defined_by**.

name_scope: la référence à la classe à laquelle ou en dessous de laquelle l'élément de propriété est disponible comme référence par l'attribut **described_by**.

absolute_id: l'identification unique de cette propriété.

describes_classes: les classes déclarant cette propriété disponible à l'emploi pour la description d'un produit.

Propositions formelles:

WR1: toute classe référencée par l'attribut **describes_classes** de **property_BSU** est soit la classe référencée par son attribut **name_scope**, soit une sous-classe de cette classe.

UR1: l'identificateur de propriété **absolute_id** est unique.

NOTE L'attribut **name_scope** de **property_BSU** sera utilisé pour coder l'attribut "Definition class" (Classe de définition) pour les propriétés.

5.9.3 Property_DET

L'entité **property_DET** capture la description des propriétés d'un dictionnaire.

EXPRESS specification:

```

*)
ENTITY property_DET
ABSTRACT SUPERTYPE OF (ONEOF(
    condition_DET, dependent_P_DET, non_dependent_P_DET))
SUBTYPE OF (class_and_property_elements);
    SELF\dictionary_element.identified_by: property_BSU;
    preferred_symbol: OPTIONAL mathematical_string;
    synonymous_symbols: SET [0:?] OF mathematical_string;
    figure: OPTIONAL graphics;
    det_classification: OPTIONAL DET_classification_type;
    domain: data_type;
    formula: OPTIONAL mathematical_string;
DERIVE
    describes_classes: SET [0:?] OF class
        := identified_by.describes_classes;
END_ENTITY; -- property_DET
( *

```

Définitions des attributs:

identified_by: l'entité **property_BSU** identifiant cette propriété.

preferred_symbol: une description plus courte de cette propriété.

synonymous_symbols: synonyme de la description plus courte de cette propriété.

figure: une entité **graphics** facultative qui décrit la propriété.

det_classification: la classe ISO 80000/CEI 80000 (anciennement ISO 31) pour cette propriété.

domain: la référence **data_type** associée à la propriété.

formula: expression mathématique pour expliquer la propriété.

describes_classes: les classes déclarant cette propriété comme étant disponible à l'emploi dans la description d'un produit.

NOTE 1 L'attribut **preferred_symbol** est utilisé pour coder l'attribut "Preferred Letter Symbol" (Symbole littéral préférentiel) des propriétés.

NOTE 2 L'attribut **synonymous_symbols** est utilisé pour coder l'attribut "Synonymous Letter Symbol" (Symbole littéral synonyme) des propriétés.

NOTE 3 L'attribut **det_classification** est utilisé pour coder l'attribut "Property Type Classification" (Classification de type de propriété) d'une propriété.

NOTE 4 L'attribut **domain** est utilisé comme point de départ pour le codage de l'attribut de propriété "Data Type". L'entité **data_type** sera sous-typée pour divers types de données possibles.

NOTE 5 L'attribut **formula** est utilisé pour coder l'attribut "Formula" des propriétés.

La Figure 9 présente un modèle de planification des données associées à des **property_DET**.

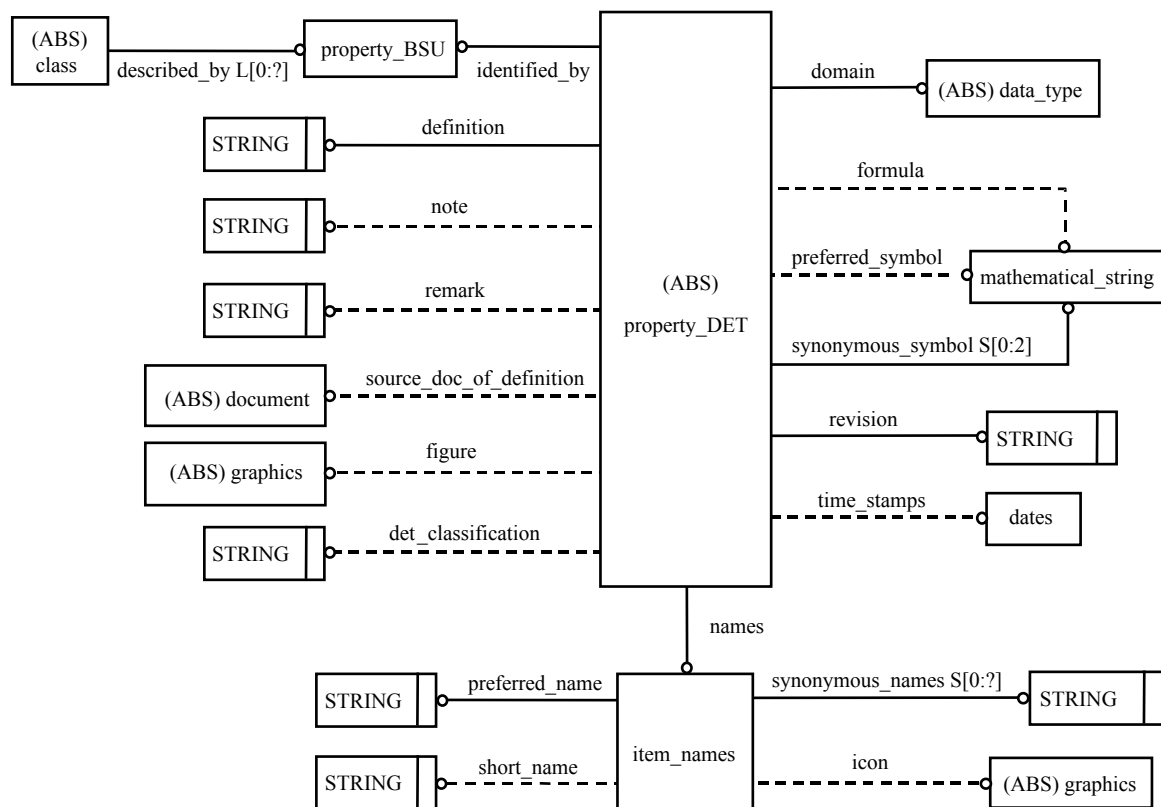


Figure 9 – Vue d'ensemble des attributs des données d'un élément et de leurs relations

5.9.4 Types de données d'un élément, conditionnels, dépendants et indépendants

La Figure 10 décrit les diverses sortes de Data Element Types (types de données d'un élément) dans le format d'un modèle de planification.

Noter que la Figure 10 est simplifiée: la relation "**depends_on**" est essentiellement mise en œuvre avec une référence BSU, mais une contrainte est spécifiée stipulant que **property_DET** référencée doit être une **condition_DET** (voir l'entité **dependent_P_DET**).

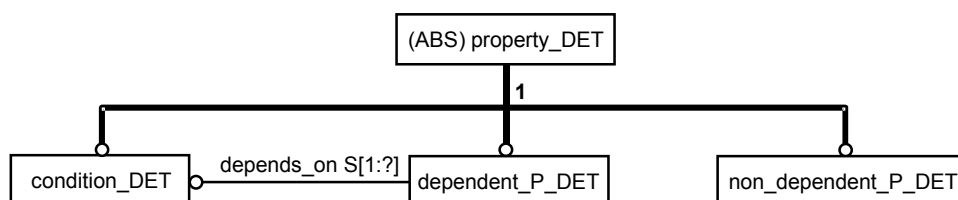


Figure 10 – Genres des types de données d'un élément

5.9.5 Détail de structure

5.9.5.1 Condition_DET

Une **condition_DET** est une propriété dont peuvent dépendre d'autres propriétés.

EXPRESS specification:

```

*)
ENTITY condition_DET
SUBTYPE OF(property_DET);
END_ENTITY; -- condition_DET
( *

```

5.9.5.2 Dependent_P_DET

Une **dependent_P_DET** est une propriété dont la valeur dépend explicitement de la/des valeur(s) d'une/de certaine(s) condition(s).

EXEMPLE La résistance d'une thermistance dépend de la température ambiante. Il convient de présenter la résistance d'une thermistance comme étant une **dependent_P_DET** et la température ambiante d'une thermistance comme étant une **condition_DET**.

EXPRESS specification:

```

*)
ENTITY dependent_P_DET
SUBTYPE OF(property_DET);
    depends_on: SET [1:?] OF property_BSU;
WHERE
    WR1: QUERY(p <* depends_on | NOT(definition_available_implies(
        p, ('ISO13584_IEC61360_DICTIONARY_SCHEMA.CONDITION_DET'
        IN TYPEOF(p.definition[1])))) = []);
END_ENTITY; -- dependent_P_DET
( *

```

Définitions des attributs:

depends_on: l'ensemble des unités sémantiques de base identifiant les propriétés dont dépend cette propriété.

Propositions formelles:

WR1: seules des **condition_DET** doivent être utilisées dans l'ensemble **depends_on**.

5.9.5.3 Non_dependent_P_DET

Une **non_dependent_P_DET** est une propriété qui ne dépend pas explicitement de certaines conditions.

EXPRESS specification:

```

*)
ENTITY non_dependent_P_DET
SUBTYPE OF(property_DET);
END_ENTITY; -- non_dependent_P_DET
( *

```

NOTE 1 Les trois sous-types **condition_DET**, **dependent_P_DET** et **non_dependent_P_DET** de l'entité **property_DET** sont utilisés pour coder les différentes sortes de propriétés (voir Article 7). L'entité **condition_DET** est utilisée pour les paramètres de contexte, **dependent_P_DET** est utilisée pour les caractéristiques dépendant d'un contexte et **non_dependent_P_DET** est utilisée pour les caractéristiques produit.

NOTE 2 L'attribut **depends_on** de l'entité **dependent_P_DET** est utilisé pour coder l'attribut "Condition" des propriétés.

5.9.6 Class_value_assignment

Les propriétés de valeur d'une classe sont les propriétés qui ne peuvent pas être assignées individuellement à une instance d'une classe, mais peuvent seulement être assignées pour toutes les instances appartenant à une classe. Ces propriétés sont déclarées en étant incluses dans la liste de **sub_class_properties** d'une entité **item_class**. Ensuite, une telle propriété peut avoir une valeur qui lui est assignée pour toutes les instances de toute **item_class** qui est une sous-classe où la propriété de valeur d'une classe est déclarée ou dans cette classe elle-même. Une valeur d'une propriété de valeur d'une classe est assignée à une **item_class** par une **class_value_assignment** référencée par l'attribut **class_constant_values** de cette classe.

NOTE Des propriétés de valeur d'une classe peuvent être de n'importe quel type de données.

EXPRESS specification:

```

*)
ENTITY class_value_assignment;
    super_class_defined_property: property_BSU;
    assigned_value: primitive_value;
WHERE
    WR1:
        definition_available_implies(super_class_defined_property,
            compatible_data_type_and_value(super_class_defined_property.
                definition[1]\property_DET.domain, assigned_value));
END_ENTITY; -- class_value_assignment
( *

```

Définitions des attributs:

super_class_defined_property: la référence à la propriété (définie dans la classe ou dans n'importe laquelle de ses superclasses comme appartenant à l'ensemble des **sub_class_properties**) à laquelle la valeur **assigned_value** est assignée.

assigned_value: la valeur assignée à la propriété, valide pour toute la classe référençant cette instance de **class_value_assignment** dans son ensemble de **class_constant_values**, et toutes ses sous-classes.

Proposition formelle:

WR1: la valeur assignée à la **super_class_defined_property** doit avoir un type compatible avec celui du domaine des valeurs de la **super_class_defined_property**.

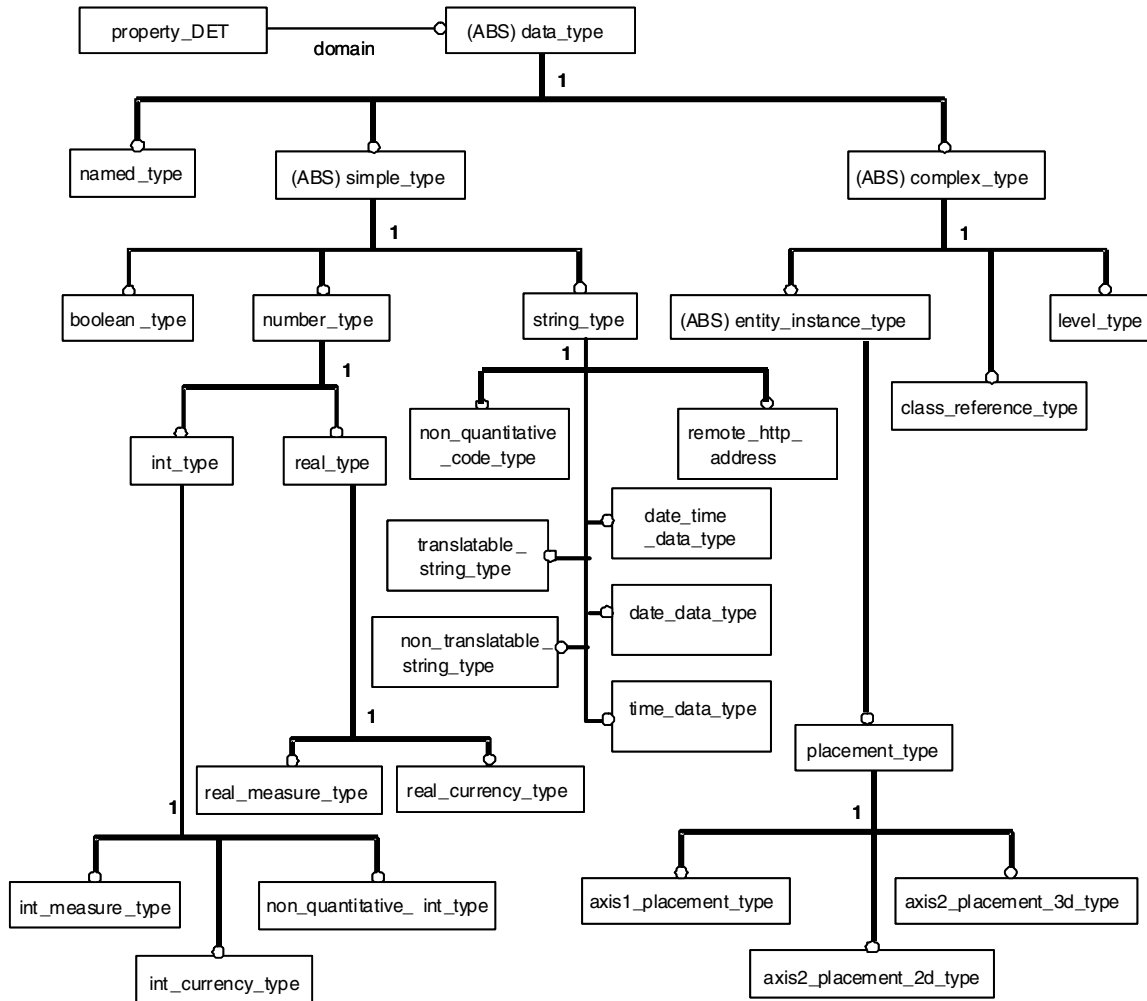


Figure 11 – Hiérarchie d'entités pour le système de types

5.10 Données de domaine: le système de types

5.10.1 Généralités

Le présent article contient des définitions pour la représentation des types de données de **property_DET**. La Figure 11 présente, sous la forme d'un modèle de planification, les grandes lignes d'une hiérarchie d'entités pour les types de données.

Contrairement aux autres éléments de dictionnaire (Fournisseurs, Classes, Propriétés), une identification avec le concept d'unité sémantique de base n'est pas obligatoire pour **data_type**, car elle sera directement attachée à **property_DET** dans un grand nombre de cas et, donc, n'a pas besoin d'une identification. Cependant, les entités **data_type_BSU** et **data_type_element** permettent une identification unique lorsque celle-ci est adaptée. Elle permet la prise en charge de la réutilisation de la même définition type dans une autre définition de **property_DET**, même à l'extérieur du fichier physique courant.

5.10.2 Détail de structure

5.10.2.1 Data_type_BSU

L'entité **data_type_BSU** permet la prise en charge de l'identification des **data_type_elements**.

EXPRESS specification:

```

*)
ENTITY data_type_BSU
  SUBTYPE OF(basic_semantic_unit);
    SELF\basic_semantic_unit.code: data_type_code_type;
    name_scope: class_BSU;
  DERIVE
    absolute_id: identifier :=
      name_scope.defined_by.absolute_id    (* Supplier*)
      + sep_id + dic_identfier;           (* Data_type *)
  INVERSE
    defining_class: SET OF class FOR defined_types;
  UNIQUE
    absolute_id;
  WHERE
    WR1: is_subclass(defining_class[1], name_scope.definition[1]);
END_ENTITY; -- data_type_BSU
(*

```

Définitions des attributs:

code: pour permettre l'identification unique du type de données sur toutes les ontologies définies par le même fournisseur **name_scope.defined_by**.

name_scope: la référence à la classe à laquelle ou en dessous de laquelle l'élément type de données est disponible comme référence par l'attribut **defined_types**.

absolute_id: l'identification unique de cette propriété.

defining_class: les classes déclarant **data_type** comme disponible à l'emploi pour la description d'un produit.

Propositions formelles:

WR1: la classe utilisée dans l'attribut **name_scope** est une superclasse de celle où ce **data_type** est définie.

NOTE L'attribut **name_scope** est utilisé pour coder la référence à une classe à laquelle le type de données connexe appartient. Celui-la même, à côté de l'entité **data_type_element** (voir ci-dessous), fait partie du codage de l'attribut de classe "Visible Types".

5.10.2.2 Data_type_element

L'entité **data_type_element** décrit les types d'élément d'un dictionnaire. Noter qu'il n'est pas nécessaire dans chaque cas d'avoir BSU et **dictionary_element** pour un certain **data_type**, car une **property_DET** peut se référer directement à **data_type**. L'utilisation de la relation BSU est seulement nécessaire lorsqu'un fournisseur souhaite se référer au même type dans un fichier physique différent.

EXPRESS specification:

```

*)
ENTITY data_type_element
  SUBTYPE OF(dictionary_element);
    SELF\dictionary_element.identified_by: data_type_BSU;

```

```

        names: item_names;
        type_definition: data_type;
    END_ENTITY; -- data_type_element
    (*

```

Définitions des attributs:

identified_by: la BSU qui identifie **data_type_element** décrite.

names: les noms qui permettent la description de **data_type_element** définie.

type_definition: la description du type transporté par **data_type_element**.

NOTE L'attribut **identified_by** redéclaré est utilisé pour coder la référence à la BSU, à laquelle ce **data_type_element** est reliée. Celui-la-même qui, à côté de l'entité **data_type_BSU** (voir ci-dessus), est utilisé pour coder l'attribut de classe "Visible types".

5.10.3 Le système de types

5.10.3.1 Data_type

L'entité **data_type** sert de supertype commun pour les entités utilisées pour indiquer le type du DET associé.

EXPRESS specification:

```

    *)
    ENTITY data_type
    ABSTRACT SUPERTYPE OF (ONEOF(
        simple_type,
        complex_type,
        named_type));
    constraints: SET [0:?] OF domain_constraint;
    WHERE
        WR1: QUERY (cons <* constraints
                    |NOT correct_constraint_type(cons, SELF)) = [];
    END_ENTITY; -- data_type
    (*

```

Définitions des attributs:

constraints: l'ensemble des contraintes du domaine qui limitent le domaine des valeurs pour le type de données.

NOTE Il convient que chaque contrainte de domaine dans l'attribut **constraints** soit satisfaite. Ainsi, l'attribut **constraints** est donc une conjonction de contraintes

Proposition formelle:

WR1: l'ensemble des contraintes du domaine doit définir des restrictions qui soient compatibles avec le domaine des valeurs du type de données.

5.10.3.2 Simple_type

L'entité **simple_type** sert de supertype commun pour les entités utilisées pour indiquer un type simple du DET associé.

EXPRESS specification:

```

*)
ENTITY simple_type
ABSTRACT SUPERTYPE OF (ONEOF(
    number_type,
    boolean_type,
    string_type))
SUBTYPE OF (data_type);
    value_format: OPTIONAL value_format_type;
END_ENTITY; -- simple_type
(*

```

Définitions des attributs:

value_format: le codage facultatif du format des valeurs pour des propriétés.

NOTE 1 L'attribut **value_format** de l'entité **simple_type** est utilisé pour coder l'attribut "Value format" des propriétés.

NOTE 2 Si une **string_pattern_constraint** quelconque s'applique à la valeur d'un type simple, elle prévaut sur le **value_format**.

5.10.3.3 Number_type

L'entité **number_type** permet la prise en charge des valeurs des DET qui sont du type NUMBER.

EXPRESS specification:

```

*)
ENTITY number_type
ABSTRACT SUPERTYPE OF (ONEOF(
    int_type,
    real_type,
    rational_type))
SUBTYPE OF (simple_type);
END_ENTITY; -- number_type
(*

```

5.10.3.4 Int_type

L'entité **int_type** permet la prise en charge des valeurs des DET qui sont du type INTEGER.

EXPRESS specification:

```

*)
ENTITY int_type
SUPERTYPE OF (ONEOF(
    int_measure_type,
    int_currency_type,
    non_quantitative_int_type))
SUBTYPE OF (number_type);
END_ENTITY; -- int_type
(*

```

5.10.3.5 Int_measure_type

L'entité **int_measure_type** permet la prise en charge des valeurs des DET qui sont des mesures de type INTEGER. Elle spécifie une **unit**, ou un identificateur d'unité (**unit_id**), dans laquelle sont exprimées les valeurs échangées comme "single integer" (entier simple). Elle peut également spécifier des unités de substitution, ou des identificateurs d'unités de substitution, qu'il est autorisé d'utiliser lorsque chaque valeur est explicitement associée à son unité.

NOTE 1 Soit un attribut **unit**, soit un attribut **unit_id** est obligatoire. S'ils sont fournis tous les deux, l'attribut **unit** prévaut.

NOTE 2 Lorsque les attributs **alternative_units** et **alternative_unit_ids** sont donnés tous les deux, ils ont tous les deux la même taille et l'attribut **alternative_units** prévaut.

NOTE 3 Les **dic_unit_identifieur** utilisés dans les attributs **unit_id** et **alternative_unit_ids** sont des identificateurs d'unités qui peuvent être réduits en une **dic_unit** à partir d'un serveur ISO/TS 29002-20.

NOTE 4 Chaque **dic_unit** définie dans l'attribut **alternative_units** et chaque entité **dic_unit** identifiée dans l'attribut **alternative_unit_ids** doivent être associées à un attribut **string_representation**, dont l'attribut **text_representation** peut être utilisé pour caractériser l'unité de remplacement utilisée au niveau d'une instance.

EXPRESS specification:

```

*)
ENTITY int_measure_type
SUBTYPE OF(int_type);
    unit: OPTIONAL dic_unit;
    alternative_units: OPTIONAL LIST [1:?] OF dic_unit;
    unit_id: OPTIONAL dic_unit_identifieur;
    alternative_unit_ids: OPTIONAL LIST [1:?] OF
dic_unit_identifieur;
WHERE
    WR1: EXISTS(unit) OR EXISTS(unit_id);
    WR2: NOT EXISTS(alternative_units) OR
        NOT EXISTS(alternative_unit_ids) OR
        (SIZEOF(alternative_units) =
SIZEOF(alternative_unit_ids));
    WR3: NOT EXISTS(alternative_units)
        OR (QUERY (un <* SELF.alternative_units
|NOT EXISTS (un.string_representation))
        = []);
END_ENTITY; -- int_measure_type
( *
```

Définitions des attributs:

unit: l'unité de référence par défaut associée à la valeur de **int_measure_type**.

alternative_units: la liste d'autres unités qui peuvent être utilisées pour exprimer la valeur de **int_measure_type**.

NOTE 5 L'ordre de la liste est utilisé pour assurer que les **alternative_units** et **alternative_unit_ids**, s'ils existent tous les deux, définissent la même unité dans le même ordre.

unit_id: l'identificateur d'unité de référence par défaut associée à la mesure décrite.

NOTE 6 L'attribut **unit** et l'attribut **unit_id** sont utilisés tous les deux pour coder l'attribut "Unit" des propriétés. Lorsqu'ils sont fournis tous les deux, **unit** prévaut.

NOTE 7 Si la valeur d'une propriété dont le domaine est **int_measure_type** est échangée comme étant un nombre entier simple, cela signifie que cette valeur est exprimée dans l'unité de mesure **unit** ou **unit_id**.

alternative_unit_ids: la liste des identificateurs d'autres unités qui peuvent être utilisées pour exprimer la valeur de **int_measure_type**.

NOTE 8 Lorsque la valeur d'une propriété dont le domaine est une **int_measure_type** est évaluée dans une unité soit définie au moyen de l'attribut **alternative_units**, soit identifiée au moyen de l'attribut **alternative_unit_ids**, sa valeur ne peut pas être représentée comme étant un entier simple. Il est nécessaire de la représenter sous forme d'une paire (valeur, unité).

Propositions formelles:

WR1: l'un des deux attributs **unit** et **unit_id** doit exister.

WR2: Si les attributs **alternative_units** et **alternative_unit_ids** existent tous les deux, ils doivent avoir la même longueur.

WR3: chaque **dic_unit** dans **alternative_units** doit avoir un attribut **string_representation**.

Propositions informelles:

IP1: Les **dic_unit_identifiers** utilisés dans les attributs **unit_id** et **alternative_unit_ids** doivent être résolus en une entité **dic_unit** à partir d'un serveur ISO/TS 29002-20 existant.

IP2: lorsque les attributs **unit** et **unit_id** sont fournis tous les deux, ils doivent définir la même unité.

IP3: lorsque les attributs **alternative_units** et **alternative_unit_ids** sont fournis tous les deux, ils doivent définir la même liste d'unités, et ce, dans le même ordre.

IP4: lorsque l'attribut **alternative_unit_ids** est fourni, toutes les unités que l'attribut identifie doivent se résoudre en une entité **dic_unit** qui a un attribut **string_representation**.

5.10.3.6 Int_currency_type

L'entité **int_currency_type** permet la prise en charge des valeurs des DET qui sont des monnaies entières.

EXPRESS specification:

```
*)
ENTITY int_currency_type
  SUBTYPE OF(int_type);
    currency: OPTIONAL currency_code;
END_ENTITY; -- int_currency_type
(*
```

Définitions des attributs:

currency: le code associé de la monnaie décrite conforme à l'ISO 4217. S'il est absent, le code de monnaie doit être échangé ensemble avec les données (valeurs).

5.10.3.7 Non_quantitative_int_type

L'entité **non_quantitative_int_type** est un type énumération dans lequel des éléments de l'énumération sont représentés avec une valeur INTEGER (voir aussi l'entité **non_quantitative_code_type** et la Figure 12).

EXPRESS specification:

```

*)
ENTITY non_quantitative_int_type
SUBTYPE OF(int_type);
    domain: value_domain;
WHERE
    WR1: QUERY(v <* domain.its_values |
        'ISO13584_IEC61360_DICTIONARY_SCHEMA.VALUE_CODE_TYPE' IN
        TYPEOF(v.value_code)) = [];
END_ENTITY; -- non_quantitative_int_type
( *

```

Définitions des attributs:

domain: l'ensemble des valeurs énumérées décrites dans l'entité **value_domain**.

Propositions formelles:

WR1: les valeurs associées à la liste de **domain.its_values** ne doivent pas contenir un **value_code_type**.

5.10.3.8 Real_type

L'entité **real_type** permet la prise en charge des valeurs des DET qui sont du type REAL.

EXPRESS specification:

```

*)
ENTITY real_type
SUPERTYPE OF(ONEOF(
    real_measure_type,
    real_currency_type))
SUBTYPE OF(number_type);
END_ENTITY; -- real_type
( *

```

5.10.3.9 Real_measure_type

L'entité **real_measure_type** permet la prise en charge des valeurs des DET qui sont des mesures de type REAL. Elle spécifie une **unit**, ou un identificateur d'unité, dans laquelle sont exprimées les valeurs échangées comme étant "single real" (réelles simples). Elle peut également spécifier des unités de substitution, ou des identificateurs d'unités de substitution, qu'il est autorisé d'utiliser lorsque chaque valeur est explicitement associée à son unité.

NOTE 1 Soit un attribut **unit**, soit un attribut **unit_id** est obligatoire. S'ils sont fournis tous les deux, l'attribut **unit** prévaut.

NOTE 2 Lorsque les attributs **alternative_units** et **alternative_unit_ids** sont donnés tous les deux, ils ont tous les deux la même taille et l'attribut **alternative_units** prévaut.

NOTE 3 Les **dic_unit_identifieur** utilisés dans les attributs **unit_id** et **alternative_unit_ids** sont des identificateurs d'unités qui peuvent se résoudre en une entité **dic_unit** à partir d'un serveur ISO/TS 29002-20.

NOTE 4 Chaque **dic_unit** définie dans l'attribut **alternative_units** et chaque **dic_unit** identifiée dans l'attribut **alternative_unit_ids** doivent être associées à un attribut **string_representation**, dont l'attribut **text_representation** peut être utilisé pour caractériser l'unité de remplacement utilisée au niveau d'instance.

EXPRESS specification:

```

*)
ENTITY real_measure_type
SUBTYPE OF(real_type);
    unit: OPTIONAL dic_unit;
    alternative_units: OPTIONAL LIST [1:?] OF dic_unit;
    unit_id : OPTIONAL dic_unit_identifier;
    alternative_unit_ids: OPTIONAL LIST [1:?] OF
dic_unit_identifier;
WHERE
    WR1: EXISTS(unit) OR EXISTS(unit_id);
    WR2: NOT EXISTS(alternative_units)
        OR NOT EXISTS(alternative_unit_ids)
        OR (SIZEOF(alternative_units) =
            SIZEOF(alternative_unit_ids));
    WR3: NOT EXISTS(alternative_units)
        OR (QUERY (un <* SELF.alternative_units
|NOT EXISTS (un.string_representation))
        = []);
END_ENTITY; -- real_measure_type
(*

```

Définitions des attributs:

unit: l'unité de référence par défaut associée à la valeur de **real_measure_type**.

alternative_units: la liste d'autres unités qui peuvent être utilisées pour exprimer la valeur de **real_measure_type**.

NOTE 5 L'ordre de la liste est utilisé pour assurer que les **alternative_units** et **alternative_unit_ids**, s'ils existent tous les deux, définissent la même unité dans le même ordre.

unit_id: l'identificateur d'unité de référence par défaut associée à la mesure décrite.

NOTE 6 L'attribut **unit** et l'attribut **unit_id** sont utilisés tous les deux pour coder l'attribut "Unit" pour des propriétés. Lorsqu'ils sont fournis tous les deux, **unit** prévaut.

NOTE 7 Si la valeur d'une propriété dont le domaine est **real_measure_type** est échangée comme étant un nombre réel simple, cela signifie que cette valeur est exprimée dans l'unité de mesure **unit** ou **unit_id**.

alternative_unit_ids: la liste des identificateurs d'autres unités qui peuvent être utilisées pour exprimer la valeur de **real_measure_type**.

NOTE 8 Lorsque la valeur d'une propriété dont le domaine est un **real_measure_type** est évaluée dans une unité soit définie au moyen de l'attribut **alternative_units**, soit identifiée au moyen de l'attribut **alternative_unit_ids**, sa valeur ne peut pas être représentée comme étant un réel simple. Elle sera représentée sous forme d'une paire (valeur, unité).

Propositions formelles:

WR1: l'un des deux attributs **unit** et **unit_id** doit exister.

WR2: Si les attributs **alternative_units** et **alternative_unit_ids** existent tous les deux, ils doivent avoir la même longueur.

WR3: chaque **dic_unit** dans l'attribut **alternative_units** doit avoir un attribut **string_representation**.

Propositions informelles:

IP1: Les **dic_unit_identifiers** utilisés dans les attributs **unit_id** et **alternative_unit_ids** doivent se résoudre en une **dic_unit** à partir d'un serveur ISO/TS 29002-20 existant.

IP2: lorsque les attributs **unit** et **unit_id** sont fournis tous les deux, ils doivent définir la même unité.

IP3: lorsque les attributs **alternative_units** et **alternative_unit_ids** sont fournis tous les deux, ils doivent définir la même liste d'unités, et ce, dans le même ordre.

IP4: lorsque l'attribut **alternative_unit_ids** est fourni, toutes les unités que l'attribut identifie doivent se résoudre en une entité **dic_unit** qui a un attribut **string_representation**.

5.10.3.10 Real_currency_type

L'entité **real_currency_type** définit les monnaies réelles.

EXPRESS specification:

```
*)
ENTITY real_currency_type
  SUBTYPE OF (real_type);
    currency: OPTIONAL currency_code;
END_ENTITY; -- real_currency_type
(*
```

Définitions des attributs:

currency: le code associé de la monnaie décrite conforme à l'ISO 4217. S'il est absent, le code de monnaie doit être échangé en même temps que les données (valeurs).

5.10.3.11 Rational_type

L'entité **rational_type** permet la prise en charge de valeurs des DET qui sont du type rationnel.

NOTE Dans l'ISO 13584-32, les valeurs rationnelles sont représentées par trois éléments XML entiers: la partie entière, le numérateur et le dénominateur. Dans l'ISO 13584-24:2003, les valeurs rationnelles sont représentées par une matrice de trois nombres entiers: la partie entière, le numérateur et le dénominateur.

EXPRESS specification:

```
*)
ENTITY rational_type
  SUPERTYPE OF (
    rational_measure_type)
  SUBTYPE OF (number_type);
END_ENTITY; -- rational_type
(*
```

5.10.3.12 Rational_measure_type

L'entité **rational_measure_type** permet la prise en charge des valeurs des DET qui sont des mesures de type RATIONAL.

EXEMPLE Diamètre de vis: 4 1/8 pouces.

L'entité **rational_measure_type** spécifie une **unit**, ou un identificateur d'unité, dans laquelle sont exprimées les valeurs échangées comme "rational" (rationnelles). Elle peut également spécifier des unités de substitution, ou des identificateurs d'unités de substitution, qu'il est autorisé d'utiliser lorsque chaque valeur est explicitement associée à son unité.

NOTE 1 Soit un attribut **unit**, soit un attribut **unit_id** est obligatoire. S'ils sont fournis tous les deux, l'attribut **unit** prévaut.

NOTE 2 Lorsque les attributs **alternative_units** et **alternative_unit_ids** sont donnés tous les deux, ils ont tous les deux la même taille et l'attribut **alternative_units** prévaut.

NOTE 3 Les **dic_unit_identifier** utilisés dans les attributs **unit_id** et **alternative_unit_ids** sont des identificateurs d'unités qui peuvent se résoudre en une entité **dic_unit** à partir d'un serveur ISO/TS 29002-20.

NOTE 4 Chaque **ic_unit** définie dans l'attribut **alternative_units** et chaque **dic_unit** identifiée dans l'attribut **alternative_unit_ids** sont associées à un attribut **string_representation**, dont l'attribut **text_representation** peut être utilisé pour caractériser l'unité de remplacement utilisée au niveau d'instance.

EXPRESS specification:

```

*)
ENTITY rational_measure_type
SUBTYPE OF(rational_type);
    unit: OPTIONAL dic_unit;
    alternative_units: OPTIONAL LIST [1:?] OF dic_unit;
    unit_id: OPTIONAL dic_unit_identifier;
    alternative_unit_ids: OPTIONAL LIST [1:?] OF
dic_unit_identifier;
WHERE
    WR1: EXISTS(unit) OR EXISTS(unit_id);
    WR2: NOT EXISTS(alternative_units)
        OR NOT EXISTS(alternative_unit_ids)
        OR (SIZEOF(alternative_units) =
        SIZEOF(alternative_unit_ids));
    WR3: NOT EXISTS(alternative_units) OR (QUERY (un <*
        SELF.alternative_units |
        NOT EXISTS (un.string_representation)) = []);
END_ENTITY; -- rational_measure_type
(*

```

Définitions des attributs:

unit: l'unité de référence par défaut associée à la valeur de **rational_measure_type**.

alternative_units: la liste d'autres unités qui peuvent être utilisées pour exprimer la valeur de **rational_measure_type**.

NOTE 5 L'ordre de la liste est utilisé pour assurer que les **alternative_units** et **alternative_unit_ids**, s'ils existent tous les deux, définissent la même unité dans le même ordre.

unit_id: l'identificateur d'unité de référence par défaut associée à la mesure décrite.

NOTE 6 L'attribut **unit** et l'attribut **unit_id** sont utilisés tous les deux pour coder l'attribut "Unit" pour des propriétés. Lorsqu'ils sont fournis tous les deux, **unit** prévaut.

NOTE 7 Si la valeur d'une propriété dont le domaine est une **rational_measure_type** est échangée comme étant un nombre rationnel simple, cela signifie que cette valeur est exprimée dans l'unité de mesure **unit** ou **unit_id**.

alternative_unit_ids: la liste des identificateurs d'autres unités qui peuvent être utilisées pour exprimer la valeur de **rational_measure_type**.

NOTE 8 Lorsque la valeur d'une propriété dont le domaine est une **rational_measure_type** est évaluée dans une unité soit définie au moyen de l'attribut **alternative_units**, soit identifiée au moyen de l'attribut **alternative_unit_ids**, sa valeur ne peut pas être représentée comme étant un rationnel simple. Il est nécessaire de la représenter sous forme d'une paire (valeur, unité).

Propositions formelles:

WR1: l'un des deux attributs **unit** et **unit_id** doit exister.

WR2: si les attributs **alternative_units** et **alternative_unit_ids** existent tous les deux, ils doivent avoir la même longueur.

WR3: chaque **dic_unit** dans **alternative_units** doit avoir un attribut **string_representation**.

Propositions informelles:

IP1: Les **dic_unit_identifiers** utilisés dans les attributs **unit_id** et **alternative_unit_ids** doivent se résoudre en une entité **dic_unit** à partir d'un serveur ISO/TS 29002-20 existant.

IP2: lorsque les attributs **unit** et **unit_id** sont fournis tous les deux, ils doivent définir la même unité.

IP3: lorsque les attributs **alternative_units** et **alternative_unit_ids** sont fournis tous les deux, ils doivent définir la même liste d'unités, et ce, dans le même ordre.

IP4: lorsque l'attribut **alternative_unit_ids** est fourni, toutes les unités que l'attribut identifie doivent se résoudre en une entité **dic_unit** qui a un attribut **string_representation**.

5.10.3.13 boolean_type

L'entité **boolean_type** permet la prise en charge de valeurs des DET qui sont du type BOOLEAN (booléen).

EXPRESS specification:

```
* )
ENTITY boolean_type
SUBTYPE OF (simple_type);
END_ENTITY; -- boolean_type
(*
```

5.10.3.14 String_type

L'entité **string_type** permet la prise en charge de valeurs des DET qui sont du type STRING (chaîne).

EXPRESS specification:

```
* )
ENTITY string_type
SUPERTYPE OF ( ONEOF (
    translatable_string_type,
    non_translatable_string_type,
    URI_type,
    non_quantitative_code_type,
    date_time_data_type,
```



```

        date_data_type,
        time_data_type))
SUBTYPE OF(simple_type);
END_ENTITY; -- string_type
(*)

```

5.10.3.15 Translatable_string_type

L'entité **translatable_string_type** permet la prise en charge de valeurs pour les DET qui sont du type STRING, mais qui sont supposées être représentés comme des chaînes différentes dans des langues différentes.

NOTE 1 Les valeurs de telles propriétés ne peuvent pas être utilisées pour l'identification de produit.

NOTE 2 Les valeurs de telles propriétés peuvent être soit une simple **string_value** lorsque **global_language_assignment** définit une langue courante, soit une **translated_string_value** lorsque chaque valeur de chaîne est associée à une langue.

NOTE 3 Deux valeurs de la même propriété dont l'entité **data_type** est **translatable_string_type** peuvent seulement être comparées pour vérifier leur égalité si la propriété correspondante comme **source_language** est définie comme partie de ses **administrative_data** et si ces valeurs sont disponibles dans cette **source_language**. Il n'est pas supposé que dans des langues différentes de cette **source_language**, la même signification soit représentée par la même chaîne.

EXPRESS specification:

```

*)
ENTITY translatable_string_type
SUBTYPE OF(string_type);
END_ENTITY; -- translatable_string_type
(*)

```

5.10.3.16 Non_translatable_string_type

L'entité **non_translatable_string_type** permet la prise en charge des valeurs pour les DET qui sont du type STRING, mais qui sont représentées de la même façon dans n'importe quelle langue.

NOTE Les valeurs de telles propriétés peuvent être utilisées pour l'identification de produit.

EXPRESS specification:

```

*)
ENTITY non_translatable_string_type
SUBTYPE OF(string_type);
END_ENTITY; -- non_translatable_string_type
(*)

```

5.10.3.17 URI_type

L'entité **URI_type** permet la prise en charge des valeurs des DET qui sont du type STRING, mais contiennent un URI⁴ (Identificateur uniforme de ressource).

NOTE Une **URI_type** permet en particulier de fournir une URL⁵ (Localisateur uniforme de ressource).

⁴ URI = *Uniform Resource Identifier*.

⁵ URL = *Uniform Ressource Locator*.

EXPRESS specification:

```

*)
ENTITY URI_type
SUBTYPE OF(string_type);
END_ENTITY; -- URI_type
( *

```

5.10.3.18 Date_time_data_type

L'entité `date_time_data_type` permet la prise en charge des valeurs des DET qui sont du type `STRING`, mais contient un instant spécifique spécifié conformément à une représentation particulière conforme à l'ISO 8601.

NOTE 1 Seul un sous-ensemble des représentations lexicales autorisées par l'ISO 8601 est autorisé pour les valeurs de **date_time_data_type**. Celui-ci est spécifié par IP1.

NOTE 2 La restriction ci-dessus des représentations de l'ISO 8601 est la même que celle définie par le Schéma XML.

EXPRESS specification:

```

*)
ENTITY date_time_data_type
SUBTYPE OF(string_type);
END_ENTITY; -- date_time_data_type
( *

```

Propositions informelles:

IP1: la valeur d'une propriété dont le type de données est **date_time_data_type** doit être conforme à la représentation lexicale suivante, qui est un sous-ensemble des représentations lexicales autorisées par l'ISO 8601. Cette représentation lexicale est le format étendu CCYY-MM-DDThh:mm:ss de l'ISO 8601 où "CC" représente l'ordre du siècle (l'ordre du premier siècle étant "00"), "YY" l'année, "MM" le mois et "DD" le jour, précédé d'un signe "-" facultatif en début pour indiquer un nombre négatif. Si le signe est omis, "+" est admis implicitement. La lettre "T" est le séparateur date/heure et "hh", "mm", "ss" représentent respectivement les heures, les minutes et les secondes. Des chiffres supplémentaires peuvent être utilisés pour augmenter la précision des secondes fractionnaires si cela est souhaité, c'est-à-dire que le format ss.ss... avec n'importe quel nombre de chiffres après le point séparateur est pris en charge. La partie des secondes fractionnaires est facultative; les autres parties de la forme lexicale ne sont pas facultatives. Pour prendre en charge des valeurs d'années supérieures à 9999, des chiffres supplémentaires peuvent être ajoutés à la gauche de la représentation. Des zéros en tête sont requis si la valeur de l'année possède moins de quatre chiffres; autrement, ils sont interdits. L'an 0000 est interdit. Le champ CCYY doit avoir au moins quatre chiffres, les champs MM, DD, SS, hh, mm et ss exactement deux chiffres chacun (en ne comptant pas les secondes fractionnaires); des zéros en tête doivent être utilisés si le champ risque d'avoir trop peu de chiffres. Cette représentation peut être immédiatement suivie d'un "Z" pour indiquer le Temps Universel Coordonné (TUC) ou, pour indiquer le fuseau horaire, c'est-à-dire la différence entre l'heure locale et le Temps Universel Coordonné, immédiatement suivie d'un signe, + ou -, suivi de la différence par rapport au TUC représentée sous la forme hh:mm (note: la partie des minutes est requise). Voir l'ISO 8601 pour les détails concernant les valeurs légales dans les divers champs. Si le fuseau horaire est inclus, les heures et aussi les minutes doivent être présentes.

EXEMPLE Pour indiquer 1:20 de l'après-midi du 31 mai 1999 pour l'Heure Normale de l'Est qui est en retard de 5 h sur le Temps Universel Coordonné (TUC), on écrira: 1999-05-31T13:20:00-05:00.

5.10.3.19 Date_data_type

L'entité **date_data_type** permet la prise en charge des valeurs des DET qui sont du type STRING, mais contient une date calendaire spécifique spécifiée conformément à une représentation particulière conforme à l'ISO 8601.

NOTE 1 Seul un sous-ensemble des représentations lexicales autorisées par l'ISO 8601 est autorisé pour les valeurs de **date_data_type**. Celui-ci est spécifié par IP1.

NOTE 2 La restriction ci-dessus des représentations de l'ISO 8601 est la même que celle définie par le Schéma XML.

EXPRESS specification:

```
*)
ENTITY date_data_type
  SUBTYPE OF (string_type);
END_ENTITY; -- date_data_type
(*
```

Propositions informelles:

IP1: la valeur d'une propriété dont le type de données est **date_data_type** doit être conforme à la représentation lexicale suivante, qui est un sous-ensemble des représentations lexicales autorisées par l'ISO 8601. La représentation lexicale pour **date_data_type** est la représentation lexicale réduite (tronquée à droite) pour **date_time_data_type**: CCYY-MM-DF. Aucune troncature à gauche n'est permise. A la suite un qualificateur facultatif de fuseau horaire est permis comme dans le cas de **date_time_data_type**. Pour prendre en charge les valeurs d'années situées à l'extérieur de la plage 0001 à 9999, des chiffres supplémentaires peuvent être ajoutés à la gauche de cette représentation et un signe "-" placé devant est autorisé.

EXEMPLE 1999-05-31 est la représentation **date_data_type** du: 31 mai 1999.

5.10.3.20 Time_data_type

L'entité **time_data_type** permet la prise en charge des valeurs des DET qui sont du type STRING, mais contient une heure spécifique spécifiée conformément à une représentation particulière conforme à l'ISO 8601. Une valeur de **time_data_type** représente un instant temporel qui se reproduit chaque jour.

NOTE 1 Seul un sous-ensemble des représentations lexicales autorisées par l'ISO 8601 est autorisé pour les valeurs de **time_data_type**. Celui-ci est spécifié par IP1.

NOTE 2 La restriction ci-dessus des représentations de l'ISO 8601 est la même que celle définie par le Schéma XML.

NOTE 3 Sachant que la représentation lexicale autorise un indicateur facultatif de fuseau horaire, les valeurs de **time_data_type** sont partiellement ordonnées, car il peut être impossible de déterminer l'ordre de deux valeurs dont l'une a un fuseau horaire et l'autre non.

EXPRESS specification:

```
*)
ENTITY time_data_type
  SUBTYPE OF (string_type);
END_ENTITY; -- time_data_type
(*
```

Propositions informelles:

IP1: la valeur d'une propriété dont le type de données est **time_data_type** doit être conforme à la représentation lexicale suivante, qui est un sous-ensemble des représentations lexicales autorisées par l'ISO 8601. La représentation lexicale pour **time_data_type** est la représentation lexicale tronquée à gauche pour **date_time_data_type** hh:mm:ss.sss avec un indicateur facultatif de fuseau horaire suivant.

EXEMPLE 13:20:00-05:00 est la représentation **time_data_type** de: 1.20 de l'après-midi pour l'Heure Normale de l'Est qui est en retard de 5 h sur le Temps Universel Coordonné (TUC).

5.10.3.21 Non_quantitative_code_type

L'entité **non_quantitative_code_type** est un type énumération dans lequel des éléments de l'énumération sont représentés avec une valeur STRING (voir aussi ENTITY **non_quantitative_int_type** et la Figure 12).

EXPRESS specification:

```
*)
ENTITY non_quantitative_code_type
  SUBTYPE OF(string_type);
  domain: value_domain;
WHERE
  WR1: QUERY(v <* domain.its_values |
    NOT('ISO13584_IEC61360_DICTIONARY_SCHEMA.VALUE_CODE_TYPE'
IN
  TYPEOF(v.value_code))) = [];
END_ENTITY; -- non_quantitative_code_type
(*
```

Définitions des attributs:

domain: l'ensemble des valeurs énumérées décrites dans **value_domain**.

Propositions formelles:

WR1: les valeurs associées à la liste de **domain.its_values** ne doivent contenir que des éléments de type **value_code_type**.

5.10.3.22 Complex_type

L'entité **complex_type** permet la prise en charge de la définition des types dont les valeurs sont représentées comme étant des instances EXPRESS.

EXPRESS specification:

```
*)
ENTITY complex_type
  ABSTRACT SUPERTYPE OF(ONEOF(
    level_type,
    class_reference_type,
    entity_instance_type))
  SUBTYPE OF(data_type);
END_ENTITY; -- complex_type
(*
```

5.10.3.23 Level_type

L'entité **level_type** est un type complexe indiquant que la valeur d'une propriété est constituée d'une ou au maximum de quatre valeurs de mesures réelles ou entières, chacune étant qualifiée par un indicateur particulier spécifiant la signification de la valeur.

NOTE 1 Les valeurs d'instances d'un **level_type** contiennent des valeurs pour et seulement pour les indicateurs spécifiés dans l'attribut **levels**. Lorsque certaines de ces valeurs ne sont pas disponibles, elles sont représentées par des **null_value**.

EXEMPLE Si **level_type** spécifie que seules les valeurs *minimum* (minimale) et *typical* (type) doivent être fournies comme "integer" (entières), toute instance contient des valeurs entières (ou **null_value**) seulement pour les niveaux *minimum* et *typical* de la valeur d'instance de **level_type**.

EXPRESS specification:

```

*)
ENTITY level_type
SUBTYPE OF(complex_type);
    levels: LIST [1:4] OF UNIQUE level;
    value_type: simple_type;
WHERE
    WR1: ('ISO13584_IEC61360_DICTIONARY_SCHEMA.INT_MEASURE_TYPE'
        IN TYPEOF(value_type))
        OR
    ('ISO13584_IEC61360_DICTIONARY_SCHEMA.REAL_MEASURE_TYPE'
        IN TYPEOF(value_type));
    WR2: NOT EXISTS(SELF.levels[2]) OR
        (SELF.levels[1] < SELF.levels[2]);
    WR3: NOT EXISTS(SELF.levels[2]) OR NOT EXISTS(SELF.levels[3])
OR
        (SELF.levels[2] < SELF.levels[3]);
    WR4: NOT EXISTS(SELF.levels[3]) OR NOT EXISTS(SELF.levels[4])
OR
        (SELF.levels[3] < SELF.levels[4]);
END_ENTITY; -- level_type
(*

```

Définitions des attributs:

levels: la liste des indicateurs uniques qui spécifie quelles valeurs qualifiées doivent être associées à la propriété.

value_type: le type de données des valeurs qualifiées de **level_type**.

Propositions formelles:

WR1: le **SELF.value_type** doit être du type **int_measure_type** ou du type **real_measure_type**.

WR2: l'ordre du premier et du deuxième **level** (niveau), si les deux existent, doit suivre l'ordre d'énumération du type **level**.

WR3: l'ordre du deuxième et du troisième **level** (niveau), si les deux existent, doit suivre l'ordre d'énumération du type **level**.

WR4: l'ordre du troisième et du quatrième **level** (niveau), si les deux existent, doit suivre l'ordre d'énumération du type **level**.

5.10.3.24 Level

L'entité **level** spécifie les divers indicateurs qui peuvent être utilisés pour qualifier une valeur d'un **level_type**.

Ces indicateurs sont comme suit:

- minimum (minimum): valeur la plus basse spécifiée d'une grandeur, établie pour un ensemble spécifié de conditions de fonctionnement pour lesquelles un composant, un dispositif ou un matériel est exploitable et s'exécute conformément aux exigences spécifiées;
- nominal (nominale): valeur d'une grandeur utilisée pour désigner et identifier un composant, un dispositif, un matériel ou un système;
- typical (typique): valeur d'une grandeur généralement rencontrée, utilisée à des fins de spécification, établie pour un ensemble spécifié de conditions de fonctionnement d'un composant, dispositif, matériel ou système;
- maximum (maximale): valeur la plus élevée spécifiée d'une grandeur, établie pour un ensemble spécifié de conditions de fonctionnement pour lesquelles un composant, un dispositif ou un matériel est exploitable et s'exécute conformément aux exigences spécifiées.

NOTE 1 La valeur nominale est généralement une valeur arrondie.

EXEMPLE Une batterie de voiture de 12 V (nominale) contient 6 éléments ayant une tension typique d'environ 2,2 V chacun, ce qui donne une tension typique pour la batterie d'environ 13,5 V. En charge, la tension peut atteindre une valeur maximale d'environ 14,5 V, mais elle est considérée comme étant complètement déchargée lorsque la tension chute en dessous d'une valeur minimale de 12,5 V.

NOTE 2 Les valeurs qui sont données pour une propriété de valeur de type niveau sont spécifiées dans le dictionnaire.

NOTE 3 Il est conseillé de restreindre l'utilisation du type niveau aux DET qui sont applicables dans des domaines pour lesquels l'indication de valeurs multiples pour une seule et même caractéristique est reconnue comme une pratique courante et demandée, comme cela se vérifie dans l'industrie des composants électroniques.

EXPRESS specification:

```

*)
TYPE level = ENUMERATION OF (
    min,      (* la valeur minimale de la grandeur physique *)
    nom,      (* la valeur nominale de la grandeur physique *)
    typ,      (* la valeur typique de la grandeur physique *)
    max);     (* la valeur maximale de la grandeur physique *)
END_TYPE; -- level
(*

```

5.10.3.25 Class_reference_type

L'entité **class_reference_type** permet la prise en charge des valeurs des DET qui sont représentées comme étant des instances d'une **class**. Elle est utilisée, en particulier, pour la description d'assemblages ou pour décrire le matériau dont est constitué un composant (ou une partie de composant).

EXPRESS specification:

```

*)
ENTITY class_reference_type
    SUBTYPE OF (complex_type);
    domain: class_BSU;
END_ENTITY; -- class_reference_type

```

(*

Définitions des attributs:

domain: l'entité **class_BSU** renvoyant à la **class** représentant le type décrit.

5.10.3.26 Entity_instance_type

L'entité **entity_instance_type** permet la prise en charge des valeurs des DET qui sont représentées comme étant des instances de certains types de données d'entité EXPRESS. Un attribut **type_name** permet de spécifier quels types de données sont autorisés. Ces types de données sont des chaînes contenues dans un ensemble. Cet attribut, conjointement à la fonction EXPRESS TYPEOF appliquée à la valeur, permet une vérification poussée des types et le polymorphisme. Cette entité sera sous-typée pour certains types de données dont l'emploi est autorisé dans le schéma du dictionnaire.

NOTE Lorsqu'une entité EXPRESS est la valeur d'un DET donné dont le type de données est une entité **entity_instance_type**, il est possible de vérifier le typage correct en appliquant la fonction EXPRESS TYPEOF sur cette valeur de DET et de comparer les résultats de cette application de TYPEOF aux chaînes contenues dans l'attribut d'ensemble **type_name**.

EXPRESS specification:

```
*)
ENTITY entity_instance_type
  SUBTYPE OF (complex_type);
    type_name: SET OF STRING;
END_ENTITY; -- entity_instance_type
( *
```

Définitions des attributs:

type_name: l'ensemble des chaînes qui décrivent, dans le format de la fonction EXPRESS TYPEOF, les noms des types de données d'une entité EXPRESS qui doivent appartenir au résultat de la fonction EXPRESS TYPEOF lorsqu'elle est appliquée à une valeur qui référence l'entité présente comme son type de données.

5.10.3.27 Placement_type

L'entité **placement_type** permet la prise en charge des valeurs des DET qui sont des instances du type de données des entités **placement**.

NOTE 1 Les entités **placement** sont importées de l'ISO 10303-42. Conformément à l'ISO 10303-42, une instance de **placement** peut seulement exister si elle est reliée à une instance de **geometric_representation_context** dans une certaine instance de **representation**. Par conséquent, si certaines propriétés de classe ont des instances de **placement** comme valeurs, cette classe contient un **geometric_representation_context** (qui définit le contexte de ces placements) et une **representation** (qui rassemble ces **placements** avec leur contexte). Les entités **geometric_representation_context** et **representation** n'étant pas importées dans la présente partie de la CEI 61360, les entités **placement** ne peuvent pas être utilisées lorsque seuls des schémas de l'ISO 13584-42 sont utilisés. Ces entités sont introduites comme ressources pour les autres parties de l'ISO 13584.

NOTE 2 Les entités **placement** sont utilisées en particulier dans l'ISO 13584-32 (OntoML) et dans l'ISO 13584-25.

EXPRESS specification:

```
*)
ENTITY placement_type
  SUPERTYPE OF (ONEOF(
    axis1_placement_type,
    axis2_placement_2d_type,
```

```

        axis2_placement_3d_type))
SUBTYPE OF(entity_instance_type);
WHERE
    WR1: 'GEOMETRY_SCHEMA.PLACEMENT'
        IN SELF\entity_instance_type.type_name;
END_ENTITY; -- placement_type
(*)

```

Propositions formelles:

WR1: la chaîne "GEOMETRY_SCHEMA.PLACEMENT" doit être contenue dans l'ensemble défini par l'attribut **SELF**entity_instance_type.type_name.

5.10.3.28 Axis1_placement_type

L'entité **axis1_placement_type** permet la prise en charge des valeurs des DET qui sont des instances du type de données des entités **axis1_placement** (voir l'ISO 10303-42 pour les détails).

EXPRESS specification:

```

*)
ENTITY axis1_placement_type
SUBTYPE OF(placement_type);
WHERE
    WR1: 'GEOMETRY_SCHEMA.AXIS1_PLACEMENT' IN
        SELF\entity_instance_type.type_name;
END_ENTITY; -- axis1_placement_type
(*)

```

Propositions formelles:

WR1: la chaîne "GEOMETRY_SCHEMA.AXIS1_PLACEMENT" doit être contenue dans l'ensemble défini par l'attribut **SELF**entity_instance_type.type_name.

5.10.3.29 Axis2_placement_2d_type

L'entité **axis2_placement_2d_type** permet la prise en charge des valeurs des DET qui sont des instances du type de données des entités **axis2_placement_2d** (voir l'ISO 10303-42 pour les détails).

EXPRESS specification:

```

*)
ENTITY axis2_placement_2d_type
SUBTYPE OF(placement_type);
WHERE
    WR1: 'GEOMETRY_SCHEMA.AXIS2_PLACEMENT_2D'
        IN SELF\entity_instance_type.type_name;
END_ENTITY; -- axis2_placement_2d_type
(*)

```

Propositions formelles:

WR1: la chaîne "GEOMETRY_SCHEMA.AXIS2_PLACEMENT_2D" doit être contenue dans l'ensemble défini par l'attribut **SELF**entity_instance_type.type_name.

5.10.3.30 Axis2_placement_3d_type

L'entité **axis2_placement_3d_type** permet la prise en charge des valeurs des DET qui sont des instances du type de données des entités **axis2_placement_3d** (voir l'ISO 10303-42 pour les détails).

EXPRESS specification:

```

*)
ENTITY axis2_placement_3d_type
SUBTYPE OF (placement_type);
WHERE
    WR1: 'GEOMETRY_SCHEMA.AXIS2_PLACEMENT_3D'
        IN SELF\entity_instance_type.type_name;
END_ENTITY; -- axis2_placement_3d_type
(*)

```

Propositions formelles:

WR1: la chaîne "GEOMETRY_SCHEMA.AXIS2_PLACEMENT_3D" doit être contenue dans l'ensemble défini par l'attribut **SELF\entity_instance_type.type_name**.

5.10.3.31 Named_type

L'entité **named_type** permet la prise en charge de la référence à d'autres types par le biais du mécanisme des BSU.

EXPRESS specification:

```

*)
ENTITY named_type
SUBTYPE OF (data_type );
    referred_type: data_type_BSU;
END_ENTITY; -- named_type
(*)

```

Définitions des attributs:

referred_type: la BSU identifiant **data_type** à laquelle la présente entité se réfère.

5.10.4 Valeurs

Le présent article contient des définitions pour des types d'élément de données non quantitatifs (voir les entités **non_quantitative_int_type** et **non_quantitative_code_type**).

La Figure 12 présente, sous la forme d'un modèle de planification, les grandes lignes des principales données associées à des types d'élément de données non quantitatif.

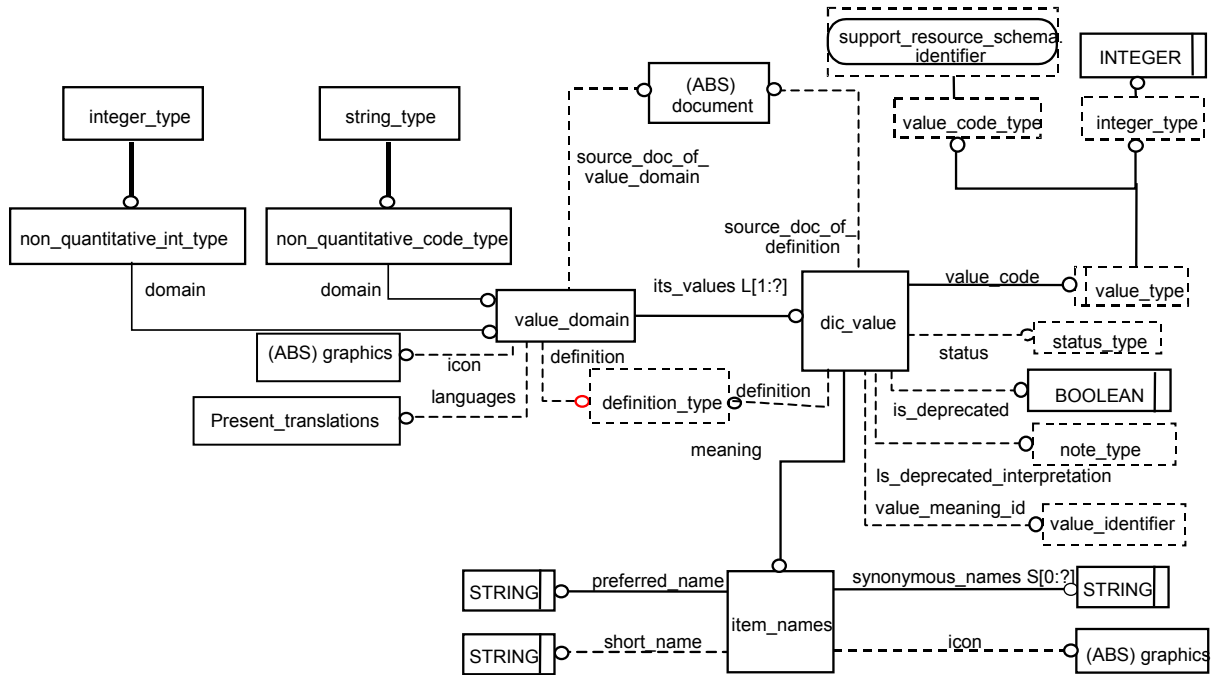


Figure 12 – Vue d'ensemble des types d'élément de données non quantitatif

5.10.5 Détail de structure

5.10.5.1 Value_domain

L'entité **value_domain** décrit l'ensemble des valeurs autorisées pour un type d'élément de données non quantitatif.

EXPRESS specification:

```
*)
ENTITY value_domain;
  its_values: LIST [1:?] OF dic_value;
  source_doc_of_value_domain: OPTIONAL document;
  languages: OPTIONAL present_translations;
  terms: LIST [0:?] OF item_names;
  definition: OPTIONAL definition_type;
  icon: OPTIONAL graphics;
WHERE
  WR1: NOT EXISTS(languages) OR (QUERY(v <* its_values |
    languages :<>: v.meaning.languages) = []);
  WR2: codes_are_unique(its_values);
  WR3: EXISTS(languages) OR (QUERY(v <* its_values |
    EXISTS(v.meaning.languages)) = []);
  WR4: EXISTS(languages) OR (QUERY(v <* its_values |
    EXISTS(v.definition.languages)) = []);
END_ENTITY; -- value_domain
(*
```

Définitions des attributs:

its_values: la liste d'énumération des valeurs contenues dans le domaine décrit.

source_doc_of_value_domain: le document décrivant le domaine associé à l'entité **value_domain** décrite.

languages: les langues facultatives dans lesquelles les traductions sont assurées.

terms: la liste des **item_names** pour permettre la prise en charge du lien CEI 61360 avec le dictionnaire des termes.

definition: le texte facultatif décrivant **value_domain**.

icon: une **icon** (icône) facultative qui représente graphiquement la description associée à **value_domain**.

Propositions formelles:

WR1: Si les significations de valeurs sont données dans plus d'une langue, l'ensemble des langues utilisées doit être le même que celui de l'ensemble des valeurs.

WR2: les codes de valeurs doivent être uniques au sein de ce type de données.

WR3: Si aucune **langue** n'est fournie, les significations des valeurs ne doivent avoir aucune langue assignée.

WR4: Si aucune **langue** n'est fournie, la définition des valeurs ne doit avoir aucune langue assignée.

5.10.5.2 Value_type

Chaque valeur d'un élément de données non quantitatif est associée à un code qui caractérise la valeur. Un **value_type** peut être soit un **INTEGER**, soit un **value_code_type**.

EXPRESS specification:

```
*)
TYPE integer_type = INTEGER;
END_TYPE; -- integer_type

TYPE value_type = SELECT(value_code_type, integer_type);
END_TYPE; -- value_type
(*
```

5.10.5.3 Dic_value

L'entité **dic_value** est l'une des valeurs de **value_domain**.

EXPRESS specification:

```
*)
ENTITY dic_value;
    value_code: value_type;
    meaning: item_names;
    source_doc_of_definition: OPTIONAL document;
    definition: OPTIONAL definition_type;
    status: OPTIONAL status_type;
    is_deprecated: OPTIONAL BOOLEAN;
END_ENTITY;
```

```

is_deprecated_interpretation: OPTIONAL note_type;
value_meaning_id: OPTIONAL dic_value_identifieur;
WHERE
    WR1: NOT EXISTS (SELF. is_deprecated)
        OR EXISTS (SELF. is_deprecated_interpretation);
END_ENTITY; -- dic_value
(*

```

Définitions des attributs:

value_code: le code associé à la valeur décrite. Il peut être soit un **value_code_type**, soit un **integer_type**.

meaning: la signification associée à cette valeur. Elle est fournie par des noms.

source_doc_of_definition: le document source facultatif dans lequel la valeur est définie.

definition: le texte facultatif décrivant **dic_value**.

status: un **status_type** qui définit l'état du cycle de vie de **dic_value**.

NOTE 1 Les valeurs admissibles d'un **status_type** ne sont pas normalisées. Elles sont définies pour chaque dictionnaire particulier par son fournisseur d'informations.

EXEMPLE 1 Un ensemble de valeurs admissibles pour le statut des articles proposés à la normalisation à une agence de maintenance de normes ISO est défini dans les directives ISO.

EXEMPLE 2 Un ensemble de valeurs admissibles pour le statut d'articles dans la norme de la base de données CEI est défini dans les directives CEI.

NOTE 2 Pour les entités **dic_value** qui ne sont pas encore distribuées pour insertion, la représentation de projets d'entités **dic_value** pourrait s'avérer utile.

EXEMPLE 3 Pour des besoins d'expérimentation avant validation.

NOTE 3 Si l'attribut **status** n'est pas fourni pour une **dic_value** et si l'utilisation de cette **dic_value** n'est pas déconseillée comme indiqué par un éventuel attribut **is_deprecated**, **dic_value** a le même statut de normalisation que le dictionnaire au complet. En particulier, si le dictionnaire est normalisé, cette **dic_value** est partie intégrante de l'édition courante de la norme.

is_deprecated: un Boolean qui spécifie, lorsqu'il est "true", que **dic_value** ne doit plus être utilisée.

NOTE 4 Lorsque l'attribut **is_deprecated** n'a aucune valeur, **dic_value** n'est pas déconseillée.

NOTE 5 Les entités **dic_value** déconseillées sont laissées dans les entités **value_domain** pour des raisons de compatibilité ascendante.

is_deprecated_interpretation: spécifie la justification de la dépréciation et comment il convient d'interpréter les valeurs de l'instance de l'élément déconseillé.

value_meaning_id: un **dic_value_identifieur** qui est un identificateur global de **dic_value**, quelle que soit **value_domain** dans laquelle elle est incluse.

NOTE 6 Cet identificateur permet de réutiliser la même définition de **dic_value** dans divers domaines.

Proposition formelle:

WR1: lorsque **is_deprecated** existe, **is_deprecated_interpretation** doit exister.

Proposition informelle:

IP1: les valeurs de l'instance de l'élément **is_deprecated_interpretation** doivent être définies au moment où la décision de dépréciation a été prise.

5.10.5.4 Administrative_data

Une entité **administrative_data** enregistre des informations relatives au cycle de vie d'un élément du dictionnaire.

EXPRESS specification:

```

*)
ENTITY administrative_data;
    status: OPTIONAL status_type;
    translation: LIST[0:?] of translation_data;
    source_language: language_code;
INVERSE
    administrated_element: dictionary_element FOR administration;
WHERE
    WR1: one_language_per_translation(SELF);
    WR2: SIZEOF(QUERY (trans <* SELF.translation |
        trans.language = source_language)) = 0;
END_ENTITY; -- administrative_data
( *
```

Définitions des attributs:

status: un **status_type** qui définit l'état du cycle de vie de l'élément du dictionnaire.

NOTE 1 Les valeurs admissibles d'un **status_type** ne sont pas normalisées. Elles sont définies pour chaque dictionnaire particulier par son fournisseur d'informations.

EXEMPLE 1 Un ensemble de valeurs admissibles pour le statut des articles proposés à la normalisation à une agence de maintenance de normes ISO est défini dans les directives ISO.

EXEMPLE 2 Un ensemble de valeurs admissibles pour le statut d'articles dans la norme de la base de données CEI est défini dans les directives CEI.

NOTE 2 Pour les entités **dictionary_element** qui ne sont pas encore distribuées pour insertion, la représentation de projets d'entités **dictionary_element** pourrait s'avérer utile.

EXEMPLE 3 Pour des besoins d'expérimentation avant validation.

NOTE 3 Si l'attribut **status** n'est pas fourni et si l'utilisation de cette entité **dictionary_element** n'est pas déconseillée comme indiqué par un éventuel attribut **is_deprecated**, **dictionary_element** a le même statut de normalisation que le dictionnaire au complet. En particulier, si le dictionnaire est normalisé, cette entité **dictionary_element** est partie intégrante de l'édition courante de la norme.

translation: description des traducteurs responsables dans les diverses langues.

source_language: la langue dans laquelle **dictionary_element** était initialement définie et qui fournit la signification de référence en cas de discordance de traduction.

NOTE 4 Un dictionnaire peut contenir des entités **dictionary_element** dont les attributs **source_language** sont différents, par exemple parce qu'elles avaient été importées de dictionnaires différents. Il est de la responsabilité du fournisseur de données du dictionnaire de s'assurer que les informations relatives à ces divers éléments sont fournies dans la même langue ou dans les mêmes langues.

administrated_element: l'entité **dictionary_element** dont les données du cycle de vie sont enregistrées dans **administrative_data**.

Propositions formelles:

WR1: les langues de traduction associées à une **administrative_data** sont uniques.

WR2: l'attribut **source_language** est absent dans les langues de traduction associées à une **administrative_data**.

5.10.5.5 Translation_data

Une entité **translation_data** enregistre des informations relatives aux possibles traductions d'un élément du dictionnaire.

EXPRESS specification:

```
*)
ENTITY translation_data;
    language: language_code;
    responsible_translator: supplier_BSU;
    translation_revision: revision_type;
    date_of_current_translation_revision: OPTIONAL date_type;
INVERSE
    belongs_to: administrative_data FOR translation;
END_ENTITY; -- translation_data
(*
```

Définitions des attributs:

language: la langue dans laquelle l'élément du dictionnaire est traduit.

NOTE 1 En cas de discordance entre la définition initiale d'un élément du dictionnaire et certaines de ses traductions, la signification réelle de l'élément du dictionnaire est celle de la langue de définition source.

responsible_translator: l'organisation responsable de la traduction dans l'élément langue.

translation_revision: le numéro de révision de la traduction correspondante.

NOTE 2 Le changement de **version** ou le changement de **revision** d'un élément du dictionnaire ne nécessite pas toujours un changement de leurs traductions. S'il n'y a pas de changement de traduction dû à un changement de **version** ou à un changement de **revision** d'un élément du dictionnaire, il convient que l'attribut **translation_revision** correspondant ne change pas. Cependant, tout changement d'une traduction impliquera le changement de l'attribut **translation_revision** correspondant.

date_of_current_translation: la date de la dernière révision de la traduction correspondante

belongs_to: l'entité **administrative_data** qui référence l'enregistrement de **translation_data**.

5.10.6 Extension pour les définitions d'unités de l'ISO 10303-41

5.10.6.1 Généralités

Le présent article définit les ressources pour la description d'unités dans un dictionnaire. Il étend les ressources définies dans l'ISO 10303-41.

5.10.6.2 Non_si_unit

L'entité **non_si_unit** étend le modèle d'unités de l'ISO 10303-41 pour permettre la prise en charge de la représentation des unités non SI qui ne dépendent pas du contexte ou qui ne sont pas basées sur une conversion (voir l'ISO 10303-41 pour les détails).

EXPRESS specification:

```

*)
ENTITY non_si_unit
  SUBTYPE OF(named_unit);
    name: label;
END_ENTITY; -- non_si_unit
(*

```

Définitions des attributs:

name: l'étiquette (**label**) utilisée pour désigner l'unité décrite.

5.10.6.3 Règle assert_ONEOF

La règle **assert_ONEOF** affirme qu'ONEOF est valide entre les sous-types suivants de **named_unit**: **si_unit**, **context_dependent_unit**, **conversion_based_unit**, **non_si_unit**.

EXPRESS specification:

```

*)
RULE assert_ONEOF FOR(named_unit);
WHERE
  QUERY(u <* named_unit |
    ('ISO13584_IEC61360_DICTIONARY_SCHEMA.NON_SI_UNIT'
    IN TYPEOF(u)) AND
    ('MEASURE_SCHEMA.SI_UNIT' IN TYPEOF(u))
    OR ('ISO13584_IEC61360_DICTIONARY_SCHEMA.NON_SI_UNIT'
    IN TYPEOF(u)) AND
    ('MEASURE_SCHEMA.CONTEXT_DEPENDENT_UNIT' IN TYPEOF(u))
    OR ('ISO13584_IEC61360_DICTIONARY_SCHEMA.NON_SI_UNIT'
    IN TYPEOF(u)) AND
    ('MEASURE_SCHEMA.CONVERSION_BASED_UNIT' IN TYPEOF(u))
  ) = [];
END_RULE; -- assert_ONEOF
(*

```

5.10.6.4 Dic_unit

La représentation de base des unités est donnée dans une forme structurée conformément à l'ISO 10303-41. Mais comme l'un des objectifs de la consignation d'unités dans le dictionnaire est la présentation à l'utilisateur, la représentation structurée seule ne suffit pas. Elle doit être complétée par une représentation sous forme de chaînes.

Les présentes définitions permettent diverses possibilités:

- la fonction **string_for_unit** (voir 5.12) peut être utilisée. Pour une représentation donnée structurée d'une unité, elle retourne une chaîne comme représentation correspondant à celle utilisée dans l'Annexe B de la CEI 61360-1:2009;
- une chaîne comme représentation peut être donnée sous la forme d'un texte clair (entité **mathematical_string**, attribut **text_representation**);
- une représentation MathML peut être donnée pour permettre la prise en charge d'une présentation évoluée de l'unité comportant des indices et des exposants, etc. (entité **mathematical_string**, attribut **MathML_representation**).

L'entité **dic_unit** décrit une unité devant être consignée dans un dictionnaire.

EXPRESS specification:

```

*)
ENTITY dic_unit;
    structured_representation: unit;
    string_representation: OPTIONAL mathematical_string;
END_ENTITY; -- dic_unit
( *

```

Définitions des attributs:

structured_representation: représentation structurée, issue de l'ISO 10303-41, comprenant l'extension définie en 5.10.6.

string_representation: la fonction **string_for_unit** peut être utilisée pour calculer une chaîne comme représentation à partir de l'attribut **structured_representation**, pour le cas où aucun attribut **string_representation** ne serait présent.

NOTE L'attribut **structured_representation** de **dic_unit** est utilisé pour coder l'attribut d'une propriété "Unit".

5.11 Types de base et définitions des entités**5.11.1 Définitions des types de base**

Le présent paragraphe contient les types de base et les définitions des entités qui ont été utilisées dans la partie principale du modèle. La section suivante contient les types de bases et les définitions des entités, classés par l'ordre alphabétique.

5.11.2 Détail de structure**5.11.2.1 Class_code_type**

Le type **class_code_type** identifie les valeurs autorisées pour un code de classe.

EXPRESS specification:

```

*)
TYPE class_code_type = code_type;
WHERE
    WR1: LENGTH(SELF) <= class_code_len;
END_TYPE; -- class_code_type
( *

```

Propositions formelles:

WR1: la longueur des valeurs correspondant au type **class_code_type** doit être inférieure ou égale à la longueur de **class_code_len** (à savoir 35).

5.11.2.2 Code_type

Le type **code_type** identifie les valeurs autorisées pour un type de code utilisé pour une identification.

NOTE Si le code est également destiné à être échangé en utilisant l'ISO/TS 29002-5, il est recommandé de satisfaire aux exigences définies par cette norme. Pour un code, seuls les "caractères sûrs" sont autorisés. Les caractères sûrs comprennent: lettres majuscules, chiffres, deux-points, points ou trait de soulignement. En outre, le caractère moins, à savoir '-', est autorisé pour des besoins particuliers.

EXPRESS specification:

```

*)
TYPE code_type = identifier;
WHERE
    WR1: NOT(SELF LIKE '*#*');
    WR2: NOT(SELF LIKE '* *');
    WR3: NOT(SELF = '');
END_TYPE; -- code_type
(*

```

Propositions formelles:

WR1: le caractère '#' ne doit pas être contenu dans une valeur de **code_type**. Le caractère '#' est utilisé pour concaténer des identificateurs, (voir: CONSTANT **sep_id**), ou un code et une version (voir: CONSTANT **sep_cv**).

WR2: les espaces sont interdits, afin de prévenir les problèmes de blancs en tête et en queue lors de la concaténation de codes.

WR3: un type **code_type** ne doit pas être une chaîne vide.

5.11.2.3 Currency_code**5.11.2.3.1 Généralités**

Le type **currency_code** identifie les valeurs autorisées pour un code de monnaie. Ces valeurs sont définies conformément à l'ISO 4217.

EXEMPLE Les valeurs sont: "CHF" pour les francs suisses, "CNY" pour le yuan renminbi (chinois), "JPY" pour le yen (japonais), "SUR" pour le rouble soviétique, "USD" pour les dollars américains, "EUR" pour l'euro.

EXPRESS specification:

```

*)
TYPE currency_code = identifier;
WHERE
    WR1: LENGTH(SELF) = 3;
END_TYPE; -- currency_code
(*

```

Propositions formelles:

WR1: la longueur d'une valeur de **currency_code** doit être égale à 3.

5.11.2.3.2 Data_type_code_type

Le type **data_type_code_type** identifie les valeurs autorisées pour un code de type de données.

EXPRESS specification:

```

*)
TYPE data_type_code_type = code_type;
WHERE
    WR1: LENGTH(SELF) = data_type_code_len;
END_TYPE; -- data_type_code_type

```

(*

Propositions formelles:

WR1: la longueur d'une valeur de **data_type_code_type** doit être égale à la valeur **data_type_code_len** (à savoir 35).

5.11.2.3.3 Date_type

Le type **date_type** identifie les valeurs autorisées pour une date. Ces valeurs sont définies conformément à l'ISO 8601.

EXEMPLE "1994-03-21".

EXPRESS specification:

```
*)
TYPE date_type = STRING(10) FIXED;
WHERE
    WR1: SELF LIKE '####-##-##';
END_TYPE; -- date_type
( *
```

5.11.2.4 Definition_type

Le type **définition_type** identifie les valeurs autorisées pour une définition.

EXPRESS specification:

```
*)
TYPE definition_type = translatable_text;
END_TYPE; -- definition_type
( *
```

5.11.2.5 DET_classification_type

Le type **DET_classification_type** identifie les valeurs autorisées pour une classification de DET. Ces valeurs sont utilisées pour la classification DET conformément à l'ISO 80000/CEI 80000 (anciennement ISO 31).

EXPRESS specification:

```
*)
TYPE DET_classification_type = identifier;
WHERE
    WR1: LENGTH(SELF) = DET_classification_len;
END_TYPE; -- DET_classification_type
( *
```

Propositions formelles:

WR1: la longueur d'une valeur de **DET_classification_type** doit être égale à la valeur d'un **DET_classification_len** (à savoir 3).

5.11.2.6 Note_type

Le type **note_type** identifie les valeurs autorisées pour une note.

EXPRESS specification:

```
*)
TYPE note_type = translatable_text;
END_TYPE; -- note_type
(*
```

5.11.2.7 Pref_name_type

Le type **pref_name_type** identifie les valeurs autorisées pour un nom préférentiel.

EXPRESS specification:

```
*)
TYPE pref_name_type = translatable_label;
WHERE
    WR1: check_label_length(SELF, pref_name_len);
END_TYPE; -- pref_name_type
(*
```

Propositions formelles:

WR1: la longueur d'une valeur de **pref_name_type** ne doit pas dépasser la longueur de **pref_name_len** (à savoir 255).

5.11.2.8 Property_code_type

Le type **property_code_type** identifie les valeurs autorisées pour un code de propriété.

EXPRESS specification:

```
*)
TYPE property_code_type = code_type;
WHERE
    WR1: LENGTH(SELF) <= property_code_len;
END_TYPE; -- property_code_type
(*
```

Propositions formelles:

WR1: la longueur d'une valeur de **property_code_type** ne doit pas dépasser la longueur de **property_code_len** (à savoir 35).

5.11.2.9 Remark_type

Le type **remark_type** identifie les valeurs autorisées pour une remarque.

EXPRESS specification:

```

*)
TYPE remark_type = translatable_text;
END_TYPE; -- remark_type
( *

```

5.11.2.10 Hierarchical_position_type

Le type **prefhierarchical_position_type** identifie les valeurs autorisées pour une position hiérarchique.

EXPRESS specification:

```

*)
TYPE hierarchical_position_type = identifier;
END_TYPE; -- hierarchical_position_type
( *

```

NOTE La représentation d'une position hiérarchique dans un type **hierarchical_position_type** est basée sur certaines conventions de codage. Cette convention de codage n'est pas définie par la présente partie de la CEI 61360.

5.11.2.11 Revision_type

Le type **revision_type** identifie les valeurs autorisées pour une révision.

NOTE Lorsqu'une nouvelle version est publiée, sa valeur de révision est mise à '0'.

EXPRESS specification:

```

*)
TYPE revision_type = code_type;
WHERE
    WR1: LENGTH(SELF) <= revision_len;
    WR2: EXISTS(VALUE(SELF)) AND ('INTEGER' IN
    TYPEOF(VALUE(SELF)))
    AND (VALUE(SELF) >= 0);
END_TYPE; -- revision_type
( *

```

Propositions formelles:

WR1: la longueur d'une valeur de **revision_type** ne doit pas dépasser la longueur de **revision_len** (à savoir 3).

WR2: le **revision_type** doit contenir des chiffres uniquement et le nombre entier qu'il représente doit être un nombre entier naturel.

5.11.2.12 Short_name_type

Le type **short_name_type** identifie les valeurs autorisées pour un nom court.

EXPRESS specification:

```

*)
TYPE short_name_type = translatable_label;
WHERE
    WR1: check_label_length(SELF, short_name_len);
END_TYPE; -- short_name_type
(*)

```

Propositions formelles:

WR1: la longueur d'une valeur de **short_name_type** ne doit pas dépasser la longueur de **short_name_len** (à savoir 30).

5.11.2.13 Supplier_code_type

Le type **supplier_code_type** identifie les valeurs autorisées pour un code fournisseur.

NOTE Si le code fournisseur est également destiné à être échangé en utilisant l'ISO/TS 29002-5, les diverses parties du code fournisseur telles que définies par l'ISO 6523 (ICD, OI, OPI, OPIS et AI) sont séparées par '-'.

EXPRESS specification:

```

*)
TYPE supplier_code_type = code_type;
WHERE
    WR1: LENGTH(SELF) <= supplier_code_len;
END_TYPE; -- supplier_code_type
(*)

```

Propositions formelles:

WR1: la longueur d'une valeur de **supplier_code_type** ne doit pas dépasser la longueur de **supplier_code_len** (à savoir 149).

5.11.2.14 Syn_name_type

Le type **syn_name_type** identifie les valeurs autorisées pour un nom synonyme.

EXPRESS specification:

```

*)
TYPE syn_name_type = SELECT(label_with_language, label);
WHERE
    WR1: check_syn_length(SELF, syn_name_len);
END_TYPE; -- syn_name_type
(*)

```

Propositions formelles:

WR1: la longueur d'une valeur de **syn_name_type** ne doit pas dépasser la longueur de **syn_name_len** (à savoir 255).

5.11.2.15 Keyword_type

Le type **keyword_type** identifie les valeurs autorisées pour un mot-clé.

EXPRESS specification:

```

*)
TYPE keyword_type = SELECT(label_with_language, label);
END_TYPE; -- keyword_type
( *

```

5.11.2.16 ISO_29002_IRDI_type

Le type **ISO_29002_IRDI_type** est un identificateur global qui identifie un article administré dans un registre. La structure de cet identificateur obéit à la syntaxe des identificateurs définie dans l'ISO/TS 29002-5.

NOTE 1 Un type **ISO_29002_IRDI_type** peut être utilisé pour n'importe quelle sorte d'informations envisagées dans l'ISO/TS 29002-5 et qui peuvent être associées à un identificateur IRDI. Trois cas spéciaux sont identifiés ci-dessous, car ils sont spécifiquement utilisés dans le schéma **ISO13584_IEC61360_dictionary_schema**: **constraint_identifier**, **dic_unit_identifier** et **dic_value_identifier**.

EXPRESS specification:

```

*)
TYPE ISO_29002_IRDI_type = identifier;
WHERE
    WR1: LENGTH (SELF) <= 290;
END_TYPE; -- syn_name_type
( *

```

Propositions formelles:

WR1: comme spécifié dans l'ISO/TS 29002-5, la longueur de l'identificateur ne doit pas être supérieure à 290.

Propositions informelles:

IP1: l'identificateur doit satisfaire aux exigences spécifiées dans l'ISO/TS 29002-5 pour un "international registration data identifier" (IRD "identificateur international des données d'enregistrement").

NOTE 2 Conformément à l'ISO/TS 29002-5, un IRDI consiste soit en une chaîne, qui ne contient pas le caractère "#", pour identifier une organisation, soit en trois sous-chaînes, qui ne contiennent pas le caractère "#" et qui sont séparées par le caractère "#", pour identifier n'importe quel autre article administré.

NOTE 3 Au cas où l'IRD ne serait pas utilisé pour identifier une organisation:

- la première sous-chaîne, appelée "Registration Authority Identifier" (RAI «identificateur d'autorité d'enregistrement»), identifie l'organisation qui est responsable de l'administration de l'article administré;
- la deuxième sous-chaîne, appelée le "Data Identifier" (DI «identificateur de données»), contient à la fois une catégorisation de l'article administré, représentée par deux caractères suivis du signe moins ('-') comme défini dans l'ISO/TS 29002-5 (par exemple: classe, propriété, unité) et l'identificateur assigné à l'article administré par le RAI;
- la troisième sous-chaîne correspond à "Version Identifier" (VI «identificateur de version) de l'IRD.

5.11.2.17 Constraint_identifier

Le **constraint_identifier** est un identificateur de type **ISO_29002_IRDI_type** qui fournit un identificateur global à une contrainte représentée comme étant une entité **constraint**. La structure de cet identificateur obéit à la syntaxe des identificateurs définie dans l'ISO/TS 29002-5.

NOTE Un **constraint_identifieur** peut être associé à un service de résolution conforme à l'ISO/TS 29002-20. Ce service serait capable de retourner une définition formelle de la contrainte identifiée par le **constraint_identifieur** conforme au modèle EXPRESS de **constraint** dans la syntaxe définie par l'ISO 13584-32 (OntoML) et éventuellement dans la syntaxe de l'ISO 10303-21.

EXPRESS specification:

```
*)
TYPE constraint_identifieur = ISO_29002_IRDI_type;
END_TYPE; -- constraint_identifieur
(*
```

Propositions informelles:

IP1: la partie de l'identificateur après le deuxième caractère '#', c'est-à-dire le Data Identifier, doit commencer par '04-' pour identifier une contrainte comme spécifié dans l'ISO/TS 29002-5.

5.11.2.18 Dic_unit_identifieur

Le **dic_unit_identifieur** est un identificateur de type **ISO_29002_IRDI_type** qui identifie une unité dont la représentation de **dic_unit** doit être téléchargeable à partir du serveur ISO/TS 29002-20. La structure de cet identificateur obéit à la syntaxe des identificateurs définie dans l'ISO/TS 29002-5.

EXPRESS specification:

```
*)
TYPE dic_unit_identifieur = ISO_29002_IRDI_type;
END_TYPE; -- dic_unit_identifieur
(*
```

Propositions informelles:

IP1: un **dic_unit_identifieur** doit être associé à un service de résolution conforme à l'ISO/TS 29002, et ce service doit être capable de retourner une définition formelle de l'unité identifiée par le **dic_unit_identifieur** conforme au modèle EXPRESS **dic_unit** dans la syntaxe définie dans l'ISO 13584-32 (OntoML), et éventuellement dans la syntaxe de l'ISO 10303-21.

IP2: la partie de l'identificateur après le deuxième caractère '#', c'est-à-dire le Data Identifier, doit commencer par '05-' pour identifier une unité comme spécifié dans l'ISO/TS 29002-5.

NOTE Un **dic_unit_identifieur** constitue un International Registration Data Identifier (IRDI) tel que défini par l'ISO/CEI 11179-5.

5.11.2.19 Dic_value_identifieur

Le **dic_value_identifieur** est un identificateur de type **ISO_29002_IRDI_type** qui fournit un identificateur global à une valeur de propriété représentée comme étant une entité **dic_value**. La structure de cet identificateur obéit à la syntaxe des identificateurs définie dans l'ISO/TS 29002-5.

NOTE 1 Le fait d'assigner un **dic_value_identifieur** permet de réutiliser la même définition de **dic_value** dans plusieurs entités **value_domain**.

NOTE 2 Un **dic_value_identifieur** peut être associé à un service de résolution conforme à l'ISO/TS 29002. Ce service serait capable de retourner une définition formelle de la valeur identifiée par le **dic_value_identifieur** conforme au modèle EXPRESS de **dic_value** dans la syntaxe définie par l'ISO 13584-32 (OntoML) et éventuellement dans la syntaxe de l'ISO 10303-21.

EXPRESS specification:

```
*)
TYPE dic_value_identifier = ISO_29002_IRDI_type;
END_TYPE; -- dic_value_identifier
(*
```

Proposition informelle:

IP1: la partie de l'identificateur après le deuxième caractère '#', c'est-à-dire le Data Identifier, doit commencer par '07-' pour identifier une valeur de propriété comme spécifié dans l'ISO/TS 29002-5.

NOTE 3 Un **dic_value_identifier** constitue un International Registration Data Identifier (IRDI) tel que défini par l'ISO/CEI 11179-5.

5.11.2.20 Value_code_type

Le type **value_code_type** identifie les valeurs autorisées pour un code de valeur.

EXPRESS specification:

```
*)
TYPE value_code_type = identifier;
WHERE
    WR1: LENGTH(SELF) <= value_code_len;
END_TYPE; -- value_code_type
(*
```

Propositions formelles:

WR1: la longueur d'une valeur de **value_code_type** ne doit pas dépasser la longueur de **value_code_len** (à savoir 35).

5.11.2.21 Value_format_type

Le type **value_format_type** identifie les valeurs autorisées pour un format de valeur. Ces valeurs sont définies conformément à l'Annexe D.

EXPRESS specification:

```
*)
TYPE value_format_type = identifier;
WHERE
    WR1: LENGTH(SELF) <= value_format_len;
END_TYPE; -- value_format_type
(*
```

Propositions formelles:

WR1: la longueur d'une valeur de **value_format_type** ne doit pas dépasser la longueur de **value_format_len** (à savoir 80).

5.11.2.22 Version_type

Le type **version_type** identifie les valeurs autorisées pour une version.

EXPRESS specification:

```

*)
TYPE version_type = code_type;
WHERE
    WR1: LENGTH(SELF) <= version_len;
    WR2: EXISTS(VALUE(SELF)) AND ('INTEGER' IN
    TYPEOF(VALUE(SELF)))
    AND (VALUE(SELF) >= 0);
END_TYPE; -- version_type
(*

```

Propositions formelles:

WR1: la longueur d'une valeur de **version_type** ne doit pas dépasser la longueur de **version_len** (à savoir 10).

WR2: le type **version_type** doit contenir des chiffres uniquement.

5.11.2.23 Status_type

Le type **status_type** identifie les valeurs autorisées pour un statut. Les valeurs admissibles d'un **status_type** ne sont pas normalisées. Elles doivent être définies pour chaque dictionnaire particulier par le fournisseur des données du dictionnaire.

EXEMPLE 1 Un ensemble de valeurs admissibles pour le statut des articles proposés à la normalisation à une agence de maintenance de normes ISO est défini dans les directives ISO.

EXEMPLE 2 Un ensemble de valeurs admissibles pour le statut d'articles dans la norme base de données CEI est défini dans les directives CEI.

NOTE Un statut peut être associé à **dictionary_element** et/ou à **dic_value**.

EXPRESS specification:

```

*)
TYPE status_type = identifier;
END_TYPE; -- status_type
(*

```

5.11.2.24 Dictionary_code_type

Le type **dictionary_code_type** est **code_type** qui identifie un dictionnaire.

EXPRESS specification:

```

*)
TYPE dictionary_code_type = identifier;
WHERE
    WR1: LENGTH(SELF) <= dictionary_code_len;
END_TYPE; -- dictionary_code_type
(*

```

Propositions formelles:

WR1: la longueur d'une valeur de **dictionary_code_type** ne doit pas dépasser la longueur de **dictionary_code_len** (à savoir 131).

5.11.3 Définitions de base des entités**5.11.3.1 Généralités**

Le présent paragraphe contient les définitions de base des entités, classées par ordre alphabétique.

5.11.3.2 Dates

L'entité **dates** décrit les trois dates respectivement associées à la première description stable, à la version courante et à la révision courante d'une description donnée.

NOTE Pour toutes les règles de gestion d'un dictionnaire particulier, il est de la responsabilité du fournisseur d'informations du dictionnaire de choisir quel instant correspond à la première description stable d'un article.

EXPRESS specification:

```

*)
ENTITY dates;
    date_of_original_definition: date_type;
    date_of_current_version: date_type;
    date_of_current_revision: OPTIONAL date_type;
END_ENTITY; -- dates
( *
```

Définitions des attributs:

date_of_original_definition: la date associée à la première version stable d'un article.

date_of_current_version: la date associée à la présente version.

date_of_current_revision: la date associée à la dernière révision.

5.11.3.3 Document

L'entité **document** est une ressource abstraite qui représente un document. Le schéma du dictionnaire permet seulement la prise en charge de l'identification des documents pour les échanges (voir ci-dessous). L'entité **document** peut également être sous-typée avec des entités mettant en œuvre un moyen d'échanger des données documentaires.

EXEMPLE Par référence à un fichier externe et à l'exacte spécification du format du fichier.

EXPRESS specification:

```

*)
ENTITY document
ABSTRACT SUPERTYPE;
END_ENTITY; -- document
( *
```

5.11.3.4 Graphics

L'entité **graphics** (graphiques) est une ressource abstraite devant être sous-typée avec des entités mettant en œuvre un moyen d'échanger des données graphiques.

EXEMPLE Par référence à un fichier externe et à l'exacte spécification du format du fichier.

EXPRESS specification:

```
*)
ENTITY graphics
ABSTRACT SUPERTYPE;
END_ENTITY; -- graphics
(*
```

5.11.3.5 External_graphics

L'entité **external_graphics** (graphiques externes) permet la prise en charge de l'échange de données graphiques au moyen de fichiers externes référencés par une entité **graphic_files** (fichiers graphiques).

EXPRESS specification:

```
*)
ENTITY external_graphics
SUBTYPE OF (graphics);
    representation: graphic_files;
END_ENTITY; -- external_graphics
(*
```

Définitions des attributs:

representation: représentation d'un graphique au moyen de fichiers externes.

5.11.3.6 Graphic_files

Une **graphic_files** est un **external_item** dont le contenu est une image.

NOTE 1 Une entité **external_item**, définie dans l'ISO 13584-24:2003, est un article dont le contenu peut être fourni sous forme d'un ou plusieurs fichiers externes de la bibliothèque. Elle se réfère à **external_file_protocol** qui spécifie comment il convient que le(s) fichier(s) externe(s) de la bibliothèque soi(en)t traité(s) et à **external_content** qui spécifie le(s) fichier(s) externe(s) de la bibliothèque qui représente(nt) son contenu.

NOTE 2 Les deux entités **external_file_protocol** et **external_content** sont définies dans l'ISO 13584-24:2003.

NOTE 3 Seules les entités **external_content** qui sont constituées de fichiers **http_file** et seules les entités **external_file_protocol** qui sont des **http_protocol** sont référencées par le schéma **ISO13584_IEC61360_dictionary_schema** et peuvent être utilisées dans le contexte de la présente partie de la CEI 61360.

NOTE 4 Les fichiers d'une entité **graphic_files** peuvent dépendre de la langue; cela est spécifié par les sous-types suivants de l'entité **external_content**: **not_translatable_external_content**, **not_translated_external_content** et **translated_external_content**.

NOTE 5 **http_file**, **http_protocol**, **not_translatable_external_content**, **not_translated_external_content** et **translated_external_content** sont définis dans l'ISO 13584-24:2003 et référencés par le schéma **ISO13584_IEC61360_dictionary_schema**.

EXPRESS specification:

```
*)
ENTITY graphic_files
SUBTYPE OF (external_item);
END_ENTITY; -- graphic_files
(*
```

5.11.3.7 Identified_document

L'entité **identified_document** décrit un document identifié par son étiquette.

EXPRESS specification:

```
*)
ENTITY identified_document
SUBTYPE OF (document);
    document_identifier: translatable_label;
WHERE
    WR1:
    check_label_length(SELF.document_identifier, source_doc_len);
END_ENTITY; -- identified_document
(*
```

Définitions des attributs:

document_identifier: l'étiquette du document décrit.

Propositions formelles:

WR1: la longueur d'une valeur d'attribut de type **document_identifier** ne doit pas dépasser la longueur de **source_doc_len** (à savoir 255).

5.11.3.8 Item_names

L'entité **item_names** identifie les noms qui peuvent être associés à une description donnée. Elle énonce le nom préférentiel, l'ensemble de noms synonymes, le nom court et les langues de l'attribut **languages** dans lesquelles les différents noms sont donnés. Elle peut être associée à une icône.

EXPRESS specification:

```
*)
ENTITY item_names;
    preferred_name: pref_name_type;
    synonymous_names: SET OF syn_name_type;
    short_name: OPTIONAL short_name_type;
    languages: OPTIONAL present_translations;
    icon: OPTIONAL graphics;
WHERE
    WR1: NOT(EXISTS(languages )) OR (
        ('ISO13584_IEC61360_LANGUAGE_RESOURCE_SCHEMA'
        + '.TRANSLATED_LABEL' IN TYPEOF(preferred_name))
        AND (languages ==:
```

```

        preferred_name\translated_label.languages)
    AND (NOT(EXISTS(short_name)) OR
        ('ISO13584_IEC61360_LANGUAGE_RESOURCE_SCHEMA'
        + '.TRANSLATED_LABEL' IN TYPEOF(short_name))
    AND                                     (languages
short_name\translated_label.languages))
    AND (QUERY(s <* synonymous_names |
    NOT('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
        '.LABEL_WITH_LANGUAGE' IN TYPEOF(s))) = []));
WR2: NOT EXISTS(languages) OR (QUERY(s <* synonymous_names |
    EXISTS(s.language) AND NOT(s.language IN
    QUERY(l <* languages.language_codes | TRUE
    ))) = []);
WR3: EXISTS(languages) OR
    (('SUPPORT_RESOURCE_SCHEMA.LABEL' IN
    TYPEOF(preferred_name))
    AND (NOT(EXISTS(short_name)) OR
        ('SUPPORT_RESOURCE_SCHEMA.LABEL' IN
    TYPEOF(short_name)))
    AND (QUERY(s <* synonymous_names |
        'ISO13584_IEC61360_DICTIONARY_SCHEMA.LABEL_WITH_LANGUAGE'
IN
        TYPEOF(s)) = []));
END_ENTITY; -- item_names
(*

```

Définitions des attributs:

preferred_name: le nom qui est préférentiel à l'emploi.

synonymous_names: l'ensemble des noms synonymes.

short_name: l'abréviation du nom préférentiel.

languages: la liste facultative des langues dans lesquelles les différents noms sont donnés.

icon: une icône facultative qui représente graphiquement la description associée à **item_names**.

Propositions formelles:

WR1: si des noms préférentiels et des noms courts sont donnés dans plus d'une langue, tous les attributs **languages** des **translated_label** doivent contenir l'instance **present_translations** comme dans l'attribut "languages" de cette instance **item_names**.

WR2: si des noms synonymes sont donnés dans plus d'une langue, seules les langues indiquées dans l'instance de **present_translations** dans l'attribut **languages** de cette instance de **item_names** peuvent être utilisées.

WR3: si aucun attribut **languages** n'est fourni, **preferred_name**, **short_name** et **synonymous_names** ne doivent pas être traduits.

NOTE 1 Les attributs **preferred_name**, **synonymous_names** et **short_name** sont utilisés pour coder respectivement les attributs "Preferred name", "Synonymous name" et "Short name" pour des propriétés et des classes.

NOTE 2 L'attribut **languages** est utilisé pour définir la séquence de traductions (si cela est demandé pour des attributs).

5.11.3.9 Label_with_language

L'entité **label_with_language** fournit des ressources pour associer une étiquette à une langue.

EXPRESS specification:

```
*)
ENTITY label_with_language;
    l: label;
    language: language_code;
END_ENTITY; -- label_with_language
(*
```

Définitions des attributs:

l: l'étiquette associée à une langue.

language: le code de la langue étiquetée.

5.11.3.10 Mathematical_string

L'entité **mathematical_string** fournit des ressources définissant une représentation pour des chaînes mathématiques. Elle permet également une représentation au format MathML.

EXPRESS specification:

```
*)
ENTITY mathematical_string;
    text_representation: text;
    MathML_representation: OPTIONAL text;
END_ENTITY; -- mathematical_string
(*
```

Définitions des attributs:

text_representation: forme "linéaire" d'une chaîne mathématique, utilisant l'ISO 843, s'il y a lieu.

MathML_representation: texte MathML, balisé conformément à la DTD (définition de type de document)⁶ XML pour MathML. Le texte MathML doit être contrôlé afin qu'il soit traité comme une seule chaîne au cours de l'échange (voir l'ISO 10303-21).

5.12 Définitions des fonctions

5.12.1 Généralités

Le présent paragraphe contient des fonctions qui sont référencées dans les Articles "WHERE" pour affirmer la cohérence des données ou qui fournissent des ressources pour le développement d'applications.

⁶ DTD = *Document Type Definition*.

5.12.2 Fonction **acyclic_superclass_relationship**

La fonction **acyclic_superclass_relationship** vérifie qu'il n'y pas de cycle dans la relation de superclasse. Par la cardinalité de l'attribut **its_superclass** dans la classe ENTITY, il est assuré qu'il existe un arbre d'héritage, mais pas de graphe acyclique. Ainsi, cette fonction doit simplement vérifier qu'aucune instance de classe ne se réfère, dans l'attribut **its_superclass**, à une autre qui soit essentiellement une sous-classe.

EXPRESS specification:

```

*)
FUNCTION acyclic_superclass_relationship(
    current: class_BSU; visited: SET OF class): LOGICAL;

IF SIZEOF(current.definition) = 1 THEN
    IF current.definition[1] IN visited THEN
        RETURN(FALSE);
        (* incorrect: la courante déclare une sous-classe comme étant
sa superclasse *)
    ELSE
        IF EXISTS(current.definition[1]\class.its_superclass)
        THEN
            RETURN(acyclic_superclass_relationship(
                current.definition[1]\class.its_superclass,
                visited + current.definition[1]));
        ELSE
            RETURN(TRUE);
        END_IF;
    END_IF;
ELSE
    RETURN(UNKNOWN);
END_IF;
END_FUNCTION; -- acyclic_superclass_relationship
(*

```

5.12.3 Fonction **check_syn_length**

La fonction **check_syn_length** vérifie que la longueur de **s** ne dépasse pas la longueur indiquée par **s_length**.

EXPRESS specification:

```

*)
FUNCTION      check_syn_length(s:      syn_name_type;      s_length:
INTEGER):BOOLEAN;

IF 'ISO13584_IEC61360_DICTIONARY_SCHEMA.LABEL_WITH_LANGUAGE'
    IN TYPEOF(s)
THEN
    RETURN(LENGTH(s.1) <= s_length);
ELSE
    RETURN(LENGTH(s) <= s_length);
END_IF;
END_FUNCTION; -- check_syn_length
(*

```

5.12.4 Fonction codes_are_unique

La fonction **codes_are_unique** retourne TRUE si les attributs **value_code** sont uniques au sein de la liste des valeurs.

EXPRESS specification:

```

*)
FUNCTION codes_are_unique(values: LIST OF dic_value): BOOLEAN;
LOCAL
    ls: SET OF STRING := [];
    li: SET OF INTEGER := [];
END_LOCAL;

IF('ISO13584_IEC61360_DICTIONARY_SCHEMA.VALUE_CODE_TYPE' IN
   TYPEOF(values[1].value_code))
THEN
    REPEAT i := 1 TO SIZEOF(values);
        ls := ls + values[i].value_code;
    END_REPEAT;

    RETURN(SIZEOF(values) = SIZEOF(ls));
ELSE
    IF('ISO13584_IEC61360_DICTIONARY_SCHEMA.INTEGER_TYPE' IN
       TYPEOF(values[1].value_code))
    THEN
        REPEAT i := 1 TO SIZEOF(values);
            li := li + values[i].value_code;
        END_REPEAT;

        RETURN(SIZEOF(values) = SIZEOF(li));
    ELSE
        RETURN(?);
    END_IF;
END_IF;

END_FUNCTION; -- codes_are_unique
(*

```

5.12.5 Fonction definition_available_implies

La fonction **definition_available_implies** vérifie si la définition correspondant au paramètre **BSU** existe ou non. Ensuite, si la définition existe, le paramètre **expression** est retourné.

EXPRESS specification:

```

*)
FUNCTION definition_available_implies(
    BSU: basic_semantic_unit;
    expression: LOGICAL): LOGICAL;

RETURN(NOT(SIZEOF(BSU.definition) = 1) OR expression);

END_FUNCTION; -- definition_available_implies
(*

```


5.12.6 Fonction **is_subclass**

La fonction **is_subclass** retourne TRUE si **sub** est soit **super**, soit une sous-classe de **super**. Si certains attributs **dictionary_definition** de classe ne sont pas disponibles, la fonction retourne UNKNOWN.

EXPRESS specification:

```

*)
FUNCTION is_subclass(sub, super: class): LOGICAL;
    IF (NOT EXISTS(sub)) OR (NOT EXISTS(super)) THEN
        RETURN(UNKNOWN);
    END_IF;

    IF sub = super
    THEN
        RETURN(TRUE);
    END_IF;

    IF NOT EXISTS(sub.its_superclass)
    THEN
        (* fin de chaîne atteinte, n'a pas satisfait à super
        jusqu'ici *)
        RETURN(FALSE);
    END_IF;

    IF SIZEOF(sub.its_superclass.definition) = 1
    THEN
        (* définition disponible *)
        IF (sub.its_superclass.definition[1] = super)
        THEN
            RETURN(TRUE);
        ELSE
            RETURN(is_subclass(sub.its_superclass.definition[1],
                                super));
        END_IF;
    ELSE
        RETURN(UNKNOWN);
    END_IF;

END_FUNCTION; -- is_subclass
(*

```

5.12.7 Fonction **string_for_derived_unit**

La fonction **string_for_derived_unit** retourne une représentation STRING de la **derived_unit** (conformément à l'ISO 10303-41) passée comme paramètre. Tout d'abord, les éléments de l'unité dérivée sont séparés selon le signe de l'exposant. S'il existe des éléments des deux sortes, la notation '/' est utilisée pour séparer ceux ayant le signe positif de ceux ayant le signe négatif. S'il n'y a que des exposants négatifs, la notation u-e est utilisée. Un point '.' (code décimal 46 selon l'ISO/CEI 8859-1, voir l'ISO 10303-21) est utilisé pour séparer les éléments individuels.

EXPRESS specification:

```

*)
FUNCTION string_for_derived_unit(u: derived_unit): STRING;

    FUNCTION string_for_derived_unit_element(
        u: derived_unit_element; neg_exp: BOOLEAN
        (* imprimer les exposants négatifs avec la puissance -1
        *)): STRING;
        (* retourne une représentation STRING de
        derived_unit_element (conformément à l'ISO 10303-41)
        passée comme paramètre *)

    LOCAL
        result : STRING;
    END_LOCAL;

    result := string_for_named_unit(u.unit);
    IF (u.exponent <> 0)
    THEN
        IF (u.exponent > 0) OR NOT neg_exp
        THEN
            result := result + '**' + FORMAT(
                ABS(u.exponent), '2I')[2];
        ELSE
            result := result + '**' + FORMAT(u.exponent,
            '2I')[2];
        END_IF;
    END_IF;
    RETURN(result);
END_FUNCTION; -- string_for_derived_unit_element

LOCAL
    pos, neg: SET OF derived_unit_element;
    us: STRING;
END_LOCAL;

(* séparer les éléments d'unités selon le signe des exposants: *)
pos := QUERY(ue <* u.elements | ue.exponent > 0);
neg := QUERY(ue <* u.elements | ue.exponent < 0);
us := '';
IF SIZEOF(pos) > 0 THEN
    (* il y a des éléments d'unités avec un signe positif *)
    REPEAT i := LOINDEX(pos) TO HIINDEX(pos);
        us := us + string_for_derived_unit_element(pos[i],
        FALSE);
        IF i <> HIINDEX(pos)
        THEN
            us := us + '.';
        END_IF;
    END_REPEAT;

    IF SIZEOF(neg) > 0
    THEN

```

```

        (* il y a des éléments d'unités avec signe négatif,
        utiliser '/'
           la notation: *)
        us := us + '/';

        IF SIZEOF(neg) > 1
        THEN
            us := us + '(';
        END_IF;

        REPEAT i := LOINDEX(neg) TO HIINDEX(neg);
            us := us + string_for_derived_unit_element(
                neg[i], FALSE);
            IF i <> HIINDEX(neg)
            THEN
                us := us + '.';
            END_IF;
        END_REPEAT;

        IF SIZEOF(neg) > 1
        THEN
            us := us + ')';
        END_IF;
    END_IF;
ELSE
    (* seulement des signes négatifs, utiliser la notation u-e *)
    IF SIZEOF(neg) > 0 THEN
        REPEAT i := LOINDEX(neg) TO HIINDEX(neg);
            us := us + string_for_derived_unit_element(
                neg[i], TRUE);
            IF i <> HIINDEX(neg)
            THEN
                us := us + '.';
            END_IF;
        END_REPEAT;
    END_IF;
END_IF;

RETURN(us);

END_FUNCTION; -- string_for_derived_unit
(*

```

5.12.8 Fonction string_for_named_unit

La fonction **string_for_named_unit** retourne une représentation STRING de la **named_unit** (conformément à l'ISO 10303-41 et à l'extension dans 5.10.6.2) passée comme paramètre.

EXPRESS specification:

```

*)
FUNCTION string_for_named_unit(u: named_unit): STRING;

IF 'MEASURE_SCHEMA.SI_UNIT' IN TYPEOF(u) THEN

```

```

RETURN(string_for_SI_unit(u));
ELSE
  IF 'MEASURE_SCHEMA.CONTEXT_DEPENDENT_UNIT' IN TYPEOF(u)
  THEN
    RETURN(u\context_dependent_unit.name);
  ELSE
    IF 'MEASURE_SCHEMA.CONVERSION_BASED_UNIT' IN TYPEOF(u)
    THEN
      RETURN(u\conversion_based_unit.name);
    ELSE
      IF 'ISO13584_IEC61360_DICTIONARY_SCHEMA'
        +'.NON_SI_UNIT' IN TYPEOF(u)
      THEN
        RETURN(u\non_si_unit.name);
      ELSE
        RETURN('name_unknown');
      END_IF;
    END_IF;
  END_IF;
END_IF;

END_FUNCTION; -- string_for_named_unit
(*

```

5.12.9 Fonction string_for_SI_unit

La fonction **string_for_SI_unit** retourne une représentation STRING de **si_unit** (conformément à l'ISO 10303-41) passée comme paramètre.

EXPRESS specification:

```

*)
FUNCTION string_for_SI_unit(unit: si_unit): STRING;

LOCAL
  prefix_string, unit_string: STRING;
END_LOCAL;

IF EXISTS(unit.prefix) THEN
  CASE unit.prefix OF
    exa      : prefix_string := 'E';
    peta     : prefix_string := 'P';
    tera     : prefix_string := 'T';
    giga     : prefix_string := 'G';
    mega     : prefix_string := 'M';
    kilo     : prefix_string := 'k';
    hecto    : prefix_string := 'h';
    deca     : prefix_string := 'da';
    deci     : prefix_string := 'd';
    centi    : prefix_string := 'c';
    milli    : prefix_string := 'm';
    micro    : prefix_string := 'u';
    nano     : prefix_string := 'n';
    pico     : prefix_string := 'p';
    femto    : prefix_string := 'f';

```

```

        atto      : prefix_string := 'a';
    END_CASE;
ELSE
    prefix_string := '';
END_IF;

CASE unit.name OF
    metre      : unit_string:= 'm';
    gram       : unit_string := 'g';
    second     : unit_string := 's';
    ampere     : unit_string := 'A';
    kelvin     : unit_string := 'K';
    mole       : unit_string := 'mol';
    candela    : unit_string := 'cd';
    radian     : unit_string := 'rad';
    steradian  : unit_string := 'sr';
    hertz      : unit_string := 'Hz';
    newton     : unit_string := 'N';
    pascal     : unit_string := 'Pa';
    joule      : unit_string := 'J';
    watt       : unit_string := 'W';
    coulomb    : unit_string := 'C';
    volt       : unit_string := 'V';
    farad      : unit_string := 'F';
    ohm        : unit_string := 'Ohm';
    siemens    : unit_string := 'S';
    weber      : unit_string := 'Wb';
    tesla      : unit_string := 'T';
    henry      : unit_string := 'H';
    degree_Celsius : unit_string := 'Cel';
    lumen      : unit_string := 'lm';
    lux        : unit_string := 'lx';
    becquerel  : unit_string := 'Bq';
    gray       : unit_string := 'Gy';
    sievert    : unit_string := 'Sv';
END_CASE;

RETURN(prefix_string + unit_string);

END_FUNCTION; -- string_for_SI_unit
(*)

```

5.12.10 Fonction string_for_unit

La fonction **string_for_unit** retourne une représentation STRING de l'**unit** (conformément à l'ISO 10303-41) passée comme paramètre.

NOTE La fonction **string_for_unit** n'est pas appelée à partir du code EXPRESS. Il s'agit d'une fonction utilitaire permettant de calculer une chaîne comme représentation à partir de l'attribut **structured_representation** de **dic_unit**, lorsqu'aucun attribut **string_representation** n'est présent. Cette chaîne comme représentation correspond à celle utilisée dans l'Annexe B de la CEI 61360-1:2009.

EXPRESS specification:

```

*)
FUNCTION string_for_unit(u: unit): STRING;
    IF 'MEASURE_SCHEMA.DERIVED_UNIT' IN TYPEOF(u)
    THEN
        RETURN(string_for_derived_unit(u));
    ELSE (* 'MEASURE_SCHEMA.NAMED_UNIT' IN TYPEOF(u) holds true *)
        RETURN(string_for_named_unit(u));
    END_IF;
END_FUNCTION; -- string_for_unit
(*)

```

5.12.11 Fonction all_class_descriptions_reachable

La fonction **all_class_descriptions_reachable** vérifie si, oui ou non, les entités **dictionary_element** qui décrivent une classe, référencée par **class_BSU**, et toutes ses superclasses, peuvent être calculées dans l'arbre d'héritage défini par la hiérarchie des classes.

EXPRESS specification:

```

*)
FUNCTION all_class_descriptions_reachable(cl: class_BSU): BOOLEAN;

    IF NOT EXISTS(cl)
    THEN
        RETURN(?);
    END_IF;

    IF SIZEOF(cl.definition) = 0
    THEN
        RETURN(FALSE);
    END_IF;

    IF NOT(EXISTS(cl.definition[1]\class.its_superclass))
    THEN
        RETURN(TRUE);
    ELSE
        RETURN(all_class_descriptions_reachable(
            cl.definition[1]\class.its_superclass));
    END_IF;

END_FUNCTION; -- all_class_descriptions_reachable
(*)

```

5.12.12 Fonction compute_known_visible_properties

La fonction **compute_known_visible_properties** calcule l'ensemble des propriétés qui sont visibles pour une classe donnée. Lorsqu'une définition n'est pas disponible, elle retourne seulement les propriétés visibles qui peuvent être calculées.

NOTE Lorsque l'attribut **dictionary_definition** d'une certaine classe est absent dans le même contexte d'échange (un contexte d'échange PLIB n'est jamais supposé être complet), la superclasse d'une classe peut ne pas être connue. Par conséquent, les propriétés définies comme étant visibles par cette superclasse ne peuvent pas être calculées par la fonction **compute_known_visible_properties**. C'est seulement pour le système de

réception que tous les attributs **dictionary_definition** des **BSU** sont disponibles. Par conséquent, pour le système de réception, la fonction **compute_known_visible_properties** calcule toutes les propriétés qui sont visibles à une classe pour la référencer (ou l'une quelconque de ses superclasses) par leur attribut **name_scope**.

EXPRESS specification:

```

*)
FUNCTION compute_known_visible_properties(cl: class_BSU):
    SET OF property_BSU;
LOCAL
    s: SET OF property_BSU := [];
END_LOCAL;

s := s + USEDIN(cl, 'ISO13584_IEC61360_DICTIONARY_SCHEMA' +
    '.PROPERTY_BSU.NAME_SCOPE');
IF SIZEOF(cl.definition) = 0
THEN
    RETURN(s);
ELSE
    IF EXISTS(cl.definition[1]\class.its_superclass) THEN
        s := s + compute_known_visible_properties(
            cl.definition[1]\class.its_superclass);
    END_IF;

    RETURN(s);
END_IF;

END_FUNCTION; -- compute_known_visible_properties
(*)

```

5.12.13 Fonction compute_known_visible_data_types

La fonction **compute_known_visible_data_types** calcule l'ensemble des **data_type** qui sont visibles pour une classe donnée. Lorsqu'une définition n'est pas disponible, elle retourne seulement les entités **data_type** visibles qui peuvent être calculées.

NOTE Lorsque l'attribut **dictionary_definition** d'une certaine classe est absent dans le même contexte d'échange (un contexte d'échange PLIB n'est jamais supposé être complet), la superclasse d'une classe peut ne pas être connue. Par conséquent, les entités **data_type** définies comme étant visibles par cette superclasse ne peuvent pas être calculées par la fonction **compute_known_visible_data_types**. C'est seulement pour le système de réception que tous les attributs **dictionary_definition** des **BSU** sont disponibles. Par conséquent, sur le système de réception, la fonction **compute_known_visible_data_types** calcule toutes les **data_type** qui sont visibles à une classe pour la référencer (ou l'une quelconque de ses superclasses) par leur attribut **name_scope**.

EXPRESS specification:

```

*)
FUNCTION compute_known_visible_data_types(cl: class_BSU):
    SET OF data_type_BSU;
LOCAL
    s: SET OF data_type_BSU :=[ ];
END_LOCAL;

s := s + USEDIN(cl, 'ISO13584_IEC61360_DICTIONARY_SCHEMA' +
    '.DATA_TYPE_BSU.NAME_SCOPE');

IF SIZEOF(cl.definition) = 0
THEN

```

```

RETURN(s);
ELSE
  IF EXISTS(cl.definition[1]\class.its_superclass)
  THEN
    s := s + compute_known_visible_data_types(
      cl.definition[1]\class.its_superclass);
  END_IF;

  RETURN(s);
END_IF;

END_FUNCTION; -- compute_known_visible_data_types
(*)

```

5.12.14 Fonction compute_known_applicable_properties

La fonction **compute_known_applicable_properties** calcule l'ensemble des propriétés qui sont applicables pour une classe donnée. Lorsqu'une définition n'est pas disponible, elle retourne seulement les propriétés applicables qui peuvent être calculées.

NOTE Lorsque l'attribut **dictionary_definition** d'une certaine classe est absent dans le même contexte d'échange (un contexte d'échange PLIB n'est jamais supposé être complet), la superclasse d'une classe peut ne pas être connue. Par conséquent, les propriétés définies comme étant applicables par cette superclasse ne peuvent pas être calculées par la fonction **compute_known_applicable_properties**. C'est seulement pour le système de réception que tous les attributs **dictionary_definition** des **BSU** sont disponibles. Par conséquent, sur le système de réception, la fonction **compute_known_applicable_properties** calcule toutes les propriétés qui sont applicables à une classe pour être référencées par un attribut **described_by** ou être importées par le biais d'**a_priori_semantic_relationship**.

EXPRESS specification:

```

*)
FUNCTION compute_known_applicable_properties(cl: class_BSU):
  SET OF property_BSU;

LOCAL
  s: SET OF property_BSU := [];
END_LOCAL;

IF SIZEOF(cl.definition)=0
THEN
  RETURN(s);
ELSE
  REPEAT i := 1 TO SIZEOF(cl.definition[1]\class.described_by);
    s := s + cl.definition[1]\class.described_by[i];
  END_REPEAT;

  IF (('ISO13584_IEC61360_ITEM_CLASS_CASE_OF_SCHEMA.'
    + 'A_PRIORI_SEMANTIC_RELATIONSHIP')
    IN TYPEOF (cl.definition[1]))
  THEN
    s := s +
      cl.definition[1]\a_priori_semantic_relationship.referenced_properties;
  END_IF;

  IF EXISTS(cl.definition[1]\class.its_superclass)

```



```

    THEN
        s := s + compute_known_applicable_properties(
            cl.definition[1]\class.its_superclass);
    END_IF;

    RETURN(s);
END_IF;
END_FUNCTION; -- compute_known_applicable_properties
(*)

```

5.12.15 Fonction compute_known_applicable_data_types

La fonction **compute_known_applicable_data_types** calcule l'ensemble des **data_type** qui sont applicables pour une classe donnée. Lorsqu'une définition n'est pas disponible, elle retourne seulement les entités **data_type** applicables qui peuvent être calculées.

NOTE Lorsque l'attribut **dictionary_definition** d'une certaine classe est absent dans le même contexte d'échange (un contexte d'échange PLIB n'est jamais supposé être complet), la superclasse d'une classe peut ne pas être connue. Par conséquent, les entités **data_type** définies comme étant applicables par cette superclasse ne peuvent pas être calculées par la fonction **compute_known_applicable_data_types**. C'est seulement pour le système de réception que tous les attributs **dictionary_definition** des **BSU** sont disponibles. Par conséquent, sur le système de réception, la fonction **compute_known_applicable_data_types** calcule toutes les entités **data_type** qui sont applicables à une classe pour être référencées par un attribut **defined_types** ou être importées par le biais d'une **a_priori_semantic_relationship**.

EXPRESS specification:

```

*)
FUNCTION compute_known_applicable_data_types(cl: class_BSU):
    SET OF data_type_BSU;
LOCAL
    s: SET OF data_type_BSU := [];
END_LOCAL;

IF SIZEOF(cl.definition) = 0
THEN
    RETURN(s);
ELSE
    REPEAT i := 1 TO SIZEOF(cl.definition[1]\class.defined_types);
        s := s + cl.definition[1]\class.defined_types[i];
    END_REPEAT;

    IF (('ISO13584_IEC61360_ITEM_CLASS_CASE_OF_SCHEMA.'
        + 'A_PRIORI_SEMANTIC_RELATIONSHIP')
        IN TYPEOF (cl.definition[1]))
    THEN
        s := s + cl.definition[1]\a_priori_semantic_relationship.referenced_data_types;
    END_IF;

    IF EXISTS(cl.definition[1]\class.its_superclass)
    THEN
        s := s + compute_known_applicable_data_types(
            cl.definition[1]\class.its_superclass);
    END_IF;

```

```

        RETURN(s);
    END_IF;

    END_FUNCTION; -- compute_known_applicable_data_types
    (*

```

5.12.16 Fonction list_to_set

La fonction **list_to_set** crée un SET à partir d'une LIST nommée **l**, le type d'élément pour le SET sera le même que celui présent dans la LIST d'origine.

EXPRESS specification:

```

    *)
    FUNCTION list_to_set(l: LIST [0:?] OF GENERIC:type_elem):
        SET OF GENERIC: type_elem;

    LOCAL
        s: SET OF GENERIC: type_elem := [];
    END_LOCAL;

    REPEAT i := 1 TO SIZEOF(l);
        s := s + l[i];
    END_REPEAT;

    RETURN(s);
    END_FUNCTION; -- list_to_set
    (*

```

5.12.17 Fonction check_properties_applicability

La fonction **check_properties_applicability** s'assure que seules les propriétés qui ne sont pas applicables pour une classe par héritage deviennent applicables à cette classe pour être référencées par l'attribut **described_by**.

EXPRESS specification:

```

    *)
    FUNCTION check_properties_applicability(cl: class): LOGICAL;
    LOCAL
        inter: SET OF property_bsu := [];
    END_LOCAL;

    IF EXISTS(cl.its_superclass)
    THEN
        IF (SIZEOF(cl.its_superclass.definition)=1)
        THEN
            inter := (list_to_set(cl.described_by) *
                cl.its_superclass.definition[1]\class.
                known_applicable_properties);
            RETURN(inter = []);
        ELSE
            RETURN(UNKNOWN);
        END_IF;
    END_IF;

```

```

ELSE
    RETURN(TRUE);
END_IF;

END_FUNCTION; -- check_properties_applicability
(*)

```

5.12.18 Fonction **check_datatypes_applicability**

La fonction **check_datatypes_applicability** s'assure que seuls les datatypes qui ne sont pas applicables pour une classe par héritage deviennent applicables à cette classe pour être référencés par l'attribut **defined_types**.

EXPRESS specification:

```

*)
FUNCTION check_datatypes_applicability(cl: class): LOGICAL;
LOCAL
    inter: SET OF data_type_bsu := [];
END_LOCAL;

IF EXISTS(cl.its_superclass)
THEN
    IF (SIZEOF(cl.its_superclass.definition) = 1)
    THEN
        inter := cl.defined_types *
            cl.its_superclass.definition[1]\class.
            known_applicable_data_types;
        RETURN(inter = []);
    ELSE
        RETURN(UNKNOWN);
    END_IF;
ELSE
    RETURN(TRUE);
END_IF;

END_FUNCTION; -- check_datatypes_applicability
(*)

```

5.12.19 Fonction **one_language_per_translation**

La fonction **one_language_per_translation** s'assure que les langues de traduction dans **administrative_data** sont uniques.

EXPRESS specification:

```

*)
FUNCTION one_language_per_translation (adm: administrative_data)
    : LOGICAL;
LOCAL
    count: INTEGER;
    lang: language_code;
END_LOCAL;

```

```

    REPEAT i := 1 TO SIZEOF (adm.translation);
        lang := adm.translation[i].language;
        count := 0;
        REPEAT j :=1 TO SIZEOF (adm.translation);
            IF lang = adm.translation[j].language
            THEN
                count := count+1;
            END_IF;
        END_REPEAT;
        IF count >1
        THEN RETURN (FALSE);
        END_IF;
    END_REPEAT;
    RETURN(TRUE);

END_FUNCTION; -- one_language_per_translation
(*)

```

5.12.20 Fonction **allowed_values_integer_types**

La fonction **allowed_values_integer_types** calcule l'ensemble des valeurs de type **integer_type** autorisées par une entité **non_quantitative_int_type**. Si le paramètre est indéterminé, elle retourne une valeur indéterminée.

EXPRESS specification:

```

*)
FUNCTION                allowed_values_integer_types                (nqit:
non_quantitative_int_type)                                         (nqit:
                                                                    : SET OF integer_type;

LOCAL
    s : SET OF integer_type :=[];
END_LOCAL;

REPEAT i:=1 TO SIZEOF (nqit.domain.its_values);
    s := s + nqit.domain.its_values[i].value_code;
END_REPEAT;
RETURN(s);

END_FUNCTION; -- allowed_values_integer_types
(*)

```

5.12.21 Fonction **is_class_valued_property**

La fonction **is_class_valued_property** retourne TRUE si la propriété **prop** est définie comme étant une propriété de valeur d'une classe pour la classe **cl** au moyen d'un attribut **sub_class_properties** dans la classe **cl** ou dans l'une quelconque de ses superclasses. Si certains attributs **dictionary_definition** de classe ne sont pas disponibles pour calculer toutes les superclasses de la classe **cl**, la fonction retourne UNKNOWN.

EXPRESS specification:

```

*)
FUNCTION is_class_valued_property(
    prop: property_BSU; cl: class_BSU): LOGICAL;

```

```

    IF (SIZEOF(cl.definition) = 0)
    THEN
        RETURN (UNKNOWN);
    ELSE
        IF NOT (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
            +'.ITEM_CLASS') IN TYPEOF(cl.definition[1]))
        THEN
            RETURN (FALSE);
        END_IF;
        IF prop IN cl.definition[1].sub_class_properties
        THEN RETURN (TRUE);
        END_IF;
        IF NOT EXISTS(cl.definition[1].its_superclass)
        THEN
            (* fin de chaîne atteinte, ne satisfait à super jusqu'ici
            *)
            RETURN(FALSE);
        END_IF;
        RETURN(is_class_valued_property(prop,
            cl.definition[1].its_superclass));
    END_IF;

END_FUNCTION; -- is_class_valued_property
(*

```

5.12.22 Fonction class_value_assigned

La fonction **class_value_assigned** retourne l'ensemble des valeurs de la propriété **prop** qui ont été assignées à une classe **cl** au moyen d'un attribut **class_constant_values** dans la classe **cl** ou dans n'importe quelle superclasse de la classe **cl**. Si plusieurs valeurs sont assignées dans plusieurs superclasses, la fonction retourne l'ensemble de toutes les valeurs assignées. Si certains attributs **dictionary_definition** de la classe ne sont pas disponibles pour calculer toutes les superclasses de la classe **cl**, seules les valeurs calculées sont retournées.

EXPRESS specification:

```

*)
FUNCTION class_value_assigned (prop: property_BSU;
    cl: class_BSU) : SET OF primitive_value;
    LOCAL
        val: SET OF primitive_value := [];
        cva : SET OF class_value_assignment := [];
    END_LOCAL;
    IF (SIZEOF(cl.definition) = 0)
    THEN
        RETURN (val);
    END_IF;
    IF NOT (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
        +'.ITEM_CLASS') IN TYPEOF(cl.definition[1]))
    THEN
        RETURN (val);
    END_IF;
    IF EXISTS(cl.definition[1])
    THEN

```

```

cva:= QUERY
  (a <* cl.definition[1].class_constant_values
   | a.super_class_defined_property = prop);
REPEAT i :=1 TO SIZEOF (cva);
  val := val + cva[i].assigned_value;
END_REPEAT;
IF NOT EXISTS (cl.definition[1].its_superclass)
THEN
  RETURN (val);
ELSE RETURN (val + class_value_assigned
  (prop,cl.definition[1].its_superclass));
END_IF;
END_IF;
END_FUNCTION; -- class_value_assigned

END_SCHEMA; -- ISO13584_IEC61360_dictionary_schema
(*)

```

6 ISO13584_IEC61360_language_resource_schema

6.1 Vue d'ensemble

Le schéma suivant fournit des ressources pour les chaînes autorisées en diverses langues. Il a été extrait du schéma du dictionnaire, car il pourrait être utilisé dans d'autres schémas conceptuels. Il repose largement sur le **support_resource_schema** issu de l'ISO 10303-41, et il peut être vu comme une extension à cela. Il permet l'utilisation d'une langue spécifique dans tout un contexte d'échange (fichier physique) sans la surcharge introduite lorsque plusieurs langues sont utilisées. Voir Figure 13 pour une description graphique.

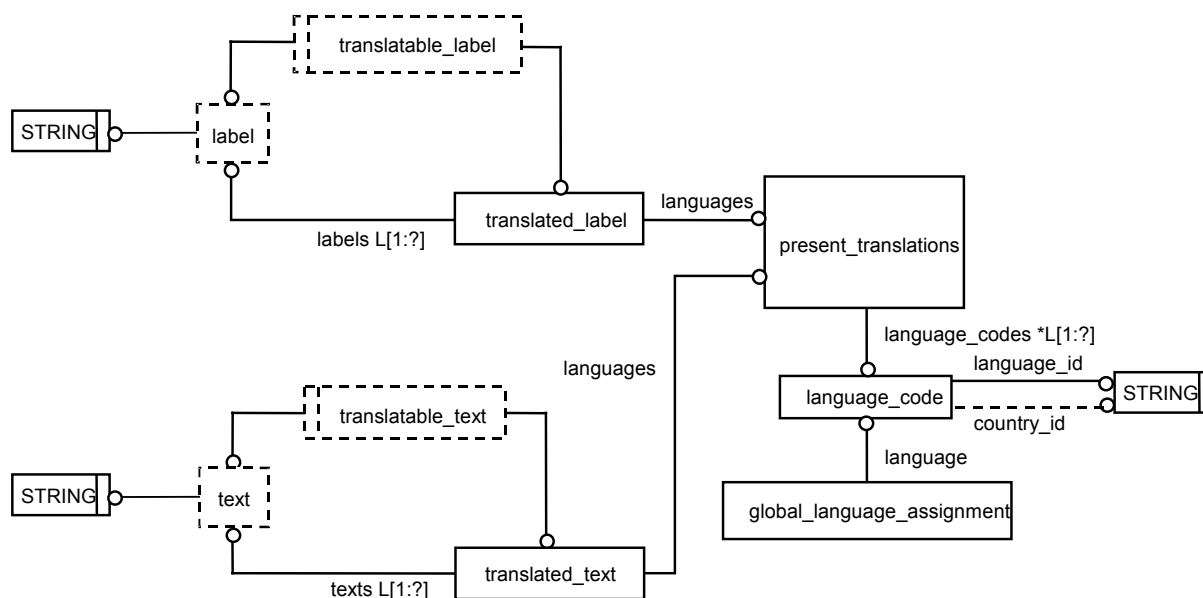


Figure 13 – ISO13584_IEC61360_language_resource_schema et support_resource_schema

EXPRESS specification:

```

*)
SCHEMA ISO13584_IEC61360_language_resource_schema;

REFERENCE FROM support_resource_schema(identifiant, label, text);

( *

```

NOTE Le schéma **support_resource_schema** référencé ci-dessus peut être consulté dans l'ISO 10303-41.

6.2 Type ISO13584_IEC61360_language_resource_schema et définitions d'entités

6.2.1 Généralités

Le présent paragraphe contient le type EXPRESS et les définitions des entités dans le schéma **ISO13584_IEC61360_language_resource_schema**.

6.2.2 Language_code

L'entité **language_code** permet d'identifier un langage conformément à l'ISO 639-1. Elle contient deux codes:

- la langue telle que définie dans l'ISO 639-1 ou dans l'ISO 639-2, et, facultativement
- le code de pays, tel que défini dans l'ISO 3166-1, spécifiant le pays dans lequel la langue est parlée.

EXPRESS specification:

```

*)
ENTITY language_code;
    language_id: identifiant;
    country_id: OPTIONAL identifiant;
WHERE
    WR1: (LENGTH (language_id) = 2) OR (LENGTH (language_id) = 3);
    WR2: LENGTH (country_id) = 2;
END_ENTITY; -- language_code
( *

```

Définitions des attributs:

language_id: le code qui spécifie la langue telle que définie par l'ISO 639-1 ou par l'ISO 639-2.

country_id: le code qui spécifie le pays dans lequel la langue est parlée, tel que défini par l'ISO 3166-1.

Propositions formelles:

WR1: la longueur d'une valeur **language_id** doit être égale à 2 ou à 3.

WR2: la longueur d'une valeur **currency_id** doit être égale à 2.

6.2.3 Global_language_assignment

L'entité **global_language_assignment** spécifie la langue pour les entités **translatable_label** et **translatable_text**, si **label** et **text** sont respectivement sélectionnés, (c'est-à-dire sans indication explicite de langue comme cela est réalisé dans **translated_label** et **translated_text**).

EXPRESS specification:

```
*)
ENTITY global_language_assignment;
    language: language_code;
END_ENTITY; -- global_language_assignment
(*
```

Définitions des attributs:

language: le code de la langue assignée.

6.2.4 Present_translations

L'entité **present_translations** sert à indiquer les langues utilisées pour **translated_label** et **translated_text**.

EXPRESS specification:

```
*)
ENTITY present_translations;
    language_codes: LIST [1:?] OF UNIQUE language_code;
    UNIQUE
        UR1: language_codes;
END_ENTITY; -- present_translations
(*
```

Définitions des attributs:

language_codes: la liste des codes de langue uniques correspondant à la langue dans laquelle une traduction est réalisée.

Proposition formelle:

UR1: pour chaque liste de **language_code**, il doit y voir une instance unique de **present_translations**.

6.2.5 Translatable_label

Un **translatable_label** définit un type de valeurs qui peuvent être des "label" (étiquettes) ou des "translated_labels" (étiquettes traduites).

EXPRESS specification:

```
*)
TYPE translatable_label = SELECT(label, translated_label);
END_TYPE; -- translatable_label
(*
```


6.2.6 Translated_label

L'entité **translated_label** définit les étiquettes qui sont traduites et les langues de traduction correspondantes.

EXPRESS specification:

```

*)
ENTITY translated_label;
    labels: LIST [1:?] OF label;
    languages: present_translations;
WHERE
    WR1: SIZEOF(labels) = SIZEOF(languages.language_codes);
END_ENTITY; -- translated_label
( *

```

Définitions des attributs:

labels: la liste des **labels** (étiquettes) qui sont traduites.

languages: la liste des **languages** (langues) dans lesquelles chaque étiquette est traduite.

Propositions formelles:

WR1: le nombre de **labels** dans la liste **labels** doit être égal au nombre de langues fournies dans l'attribut **languages.language_codes**.

Propositions informelles:

IP1: le contenu de **labels[i]** est la langue identifiée par **languages.language_codes[i]**.

6.2.7 Translatable_text

Un **translatable_text** définit un type de valeurs qui peut être soit **text** (texte) soit **translated_text** (texte traduit).

EXPRESS specification:

```

*)
TYPE translatable_text = SELECT(text, translated_text);
END_TYPE; -- translatable_text
( *

```

6.2.8 Translated_text

L'entité **translated_text** définit les **texts** qui sont traduits et les **languages** de traduction correspondantes.

EXPRESS specification:

```

*)
ENTITY translated_text;
    texts: LIST [1:?] OF text;
    languages: present_translations;

```

```
WHERE
    WR1: SIZEOF(texts) = SIZEOF(languages.language_codes);
END_ENTITY; -- translated_text
(*
```

Définitions des attributs:

texts: la liste des **text** (textes) qui sont traduits.

languages: la liste des langues dans lesquelles chaque text est traduit.

Propositions formelles:

WR1: le nombre de **text** dans la liste **texts** doit être égal au nombre de langues fournies dans l'attribut **languages.language_codes**.

Propositions informelles:

IP1: le contenu de **texts[i]** est la langue identifiée par **languages.language_codes[i]**.

6.3 Définitions des fonctions de l'ISO13584_IEC61360_language_resource_schema

6.3.1 Généralités

Le présent paragraphe contient une fonction qui est référencée dans les articles WHERE pour affirmer la cohérence des données.

6.3.2 Fonction check_label_length

La fonction **check_label_length** s'assure qu'aucune étiquette dans **l** ne dépasse la longueur indiquée par **l_length**.

EXPRESS specification:

```
*)
FUNCTION check_label_length(l: translatable_label;
    l_length: INTEGER): BOOLEAN;

IF 'ISO13584_IEC61360_LANGUAGE_RESOURCE_SCHEMA.TRANSLATED_LABEL'
    IN TYPEOF(l)
THEN
    REPEAT i :=1 TO SIZEOF(l.labels);
        IF LENGTH(l.labels[i]) > l_length
        THEN
            RETURN(FALSE);
        END_IF;
    END_REPEAT;

    RETURN(TRUE);

ELSE (* l'argument l est une chaîne simple *)
    RETURN(LENGTH(l) <= l_length);
END_IF;
END_FUNCTION; -- check_label_length
(*
```

6.4 Définition des règles de l'ISO13584_IEC61360_language_resource_schema

La règle **single_language_assignment** affirme qu'une seule langue peut être assignée pour être utilisée dans **translatable_label** et **translatable_text**.

EXPRESS specification:

```

*)
RULE single_language_assignment FOR(global_language_assignment);
WHERE
    SIZEOF(global_language_assignment) <= 1;
END_RULE; -- single_language_assignment

END_SCHEMA; -- ISO13584_IEC61360_language_resource_schema
(*)

```

7 ISO13584_IEC61360_class_constraint_schema

7.1 Généralités

Le présent article définit les exigences pour **class_constraint_schema**. La déclaration EXPRESS suivante introduit le bloc **ISO13584_IEC61360_class_constraint_schema** et identifie les références externes nécessaires.

EXPRESS specification:

```

*)
SCHEMA ISO13584_IEC61360_class_constraint_schema;

REFERENCE FROM ISO13584_IEC61360_dictionary_schema (
    class_BSU,
    property_BSU,
    definition_available_implies,
    is_subclass,
    data_type,
    simple_type,
    complex_type,
    named_type,
    allowed_values_integer_types);

REFERENCE FROM ISO13584_extended_dictionary_schema
    (data_type_typeof,
    data_type_class_of,
    data_type_type_name);

REFERENCE FROM ISO13584_instance_resource_schema
    (Boolean_value,
    compatible_class_and_class,
    complex_value,
    dic_class_instance,
    entity_instance_value,
    int_level_spec_value,

```

```

integer_value,
level_spec_value,
number_value,
primitive_value,
rational_value,
real_level_spec_value,
real_value,
right_values_for_level_spec,
same_translations,
simple_value,
string_value,
translatable_string_value,
translated_string_value,
property_or_data_type_BSU);

```

```

REFERENCE FROM ISO13584_aggregate_value_schema
(aggregate_entity_instance_value,
list_value,
set_value,
bag_value,
array_value,
set_with_subset_constraint_value,
compatible_complete_types_and_value);
(*)

```

NOTE Les schémas conceptuels référencés ci-dessus peuvent être consultés dans les documents suivants:

ISO13584_IEC61360_dictionary_schema	CEI 61360-2
(qui est dupliqué pour la commodité en 4.5 et après).	
ISO13584_extended_dictionary_schema	ISO 13584-24:2003
ISO13584_instance_resource_schema	ISO 13584-24:2003
ISO13584_aggregate_value_schema	ISO 13584-25

7.2 Introduction à l'ISO13584_IEC61360_class_constraint_schema type

Le schéma **ISO13584_IEC61360_class_constraint_schema** fournit des constructions EXPRESS permettant de redéfinir, par restriction, le domaine des valeurs d'une propriété donnée lorsqu'il est appliqué à une sous-classe de la classe de caractérisation dans laquelle la propriété était définie comme visible. Cette contrainte doit seulement expliciter une restriction du domaine de valeurs qui résulte déjà de la structure de classes.

EXEMPLE Dans l'ISO 13584-511, la classe *boulon/vis à filetage métrique* est une classe définie comme suit: "élément de fixation à filetage externe et à tête avec tige cylindrique, qui peut être partiellement ou complètement fileté et la tête peut être garnie d'une empreinte". Cette classe a, entre autres, deux propriétés appelées *type de tête* et *propriétés de tête*. Le domaine des valeurs de la propriété *type de tête* est un type de données non quantitatif qui inclut en particulier les valeurs suivantes: *hexagon_head* (tête hexagonale), *octagonal_head* (tête octogonale) et *round_head* (tête ronde). La propriété *propriétés de tête* est une caractéristique intrinsèque. Cela signifie qu'elle a un type de données *item_class* («classe d'articles»), dont le domaine est une classe *tête* qui définit toute sorte de tête. La classe *tête* a plusieurs sous-classes, y compris: *tête hexagonale*, associée à toutes les propriétés permettant de décrire une tête hexagonale (par exemple: *sur plats*), et *tête ronde*, associées à toutes les propriétés permettant de décrire une tête ronde (par exemple: *diamètre tête*).

La classe *boulon/vis à filetage métrique* a une sous-classe appelée *vis à tête hexagonale* définie comme suit: "élément de fixation à filetage externe métrique avec tête hexagonale fileté jusqu'à la tête". Cette classe hérite des propriétés *type de tête* et *tête*. À partir de la définition de la sous-classe *vis à tête hexagonale*, il est clair que la propriété *type de tête* pourrait seulement prendre la valeur *hexagon_head* (tête hexagonale) et que la propriété *propriétés de tête* pourrait seulement être une instance de la classe de caractéristique intrinsèque *tête hexagonale*. Mais ces contraintes sont implicites: elles sont juste énoncées de façon informelle dans la définition.

Ainsi, ces contraintes ne sont pas interprétables par un ordinateur. Les contraintes définies dans l'**ISO13584_IEC61360_class_constraint_schema** permettraient d'expliciter ces deux contraintes par association avec la classe *vis à tête hexagonale*: (1) une **enumeration_constraint** pour la propriété *type de tête* (autorisant

seulement le code *hexagon_head*) et (2) une **subclass_constraint** pour la propriété propriétés de tête (autorisant seulement la classe de caractéristique intrinsèque tête hexagonale).

Les contraintes sont héritées. Lorsqu'une propriété dont le domaine de valeurs a déjà été limité dans une classe C par une contrainte a besoin d'être davantage limitée dans une sous-classe C par une autre contrainte, les deux contraintes s'appliquent ensemble. Ainsi, le domaine réel de valeurs dans la sous-classe C est l'intersection des deux domaines définis par les deux contraintes. Le mécanisme proposé est semblable au fonctionnement de redéfinition du mécanisme des types disponible dans le langage EXPRESS.

Ce schéma permet d'exprimer les contraintes qui peuvent s'appliquer à des types de données issus du système type de l'**ISO13584_IEC61360_dictionary_schema**. Des règles sont utilisées dans les entités qui référencent une contrainte pour assurer que chaque contrainte peut s'appliquer au type de données auquel elle est connexe.

7.3 Définitions d'entités de l'**ISO13584_IEC61360_class_constraint_schema**

7.3.1 Généralités

Le présent article définit les entités dans l'**ISO13584_IEC61360_class_constraint_schema**.

7.3.2 Constraint

L'entité **constraint** permet de définir une contrainte.

EXPRESS specification:

```
*)
ENTITY constraint
ABSTRACT SUPERTYPE OF ( ONEOF (
    property_constraint,
    class_constraint));
    constraint_id: OPTIONAL constraint_identifier;
END_ENTITY; -- constraint
(*
```

Définitions des attributs:

constraint_id: le **constraint_identifier** qui identifie la contrainte.

7.3.3 Property_constraint

L'entité **property_constraint** est une contrainte qui limite l'ensemble admissible d'instances par la seule restriction du domaine des valeurs de l'une de ses propriétés.

EXPRESS specification:

```
*)
ENTITY property_constraint
ABSTRACT SUPERTYPE OF ( ONEOF (
    integrity_constraint,
    context_restriction_constraint))
SUBTYPE OF (constraint);
    constrained_property: property_BSU;
END_ENTITY; -- property_constraint
(*
```

Définitions des attributs:

constrained_property: l'entité **property_BSU** pour laquelle la contrainte s'applique.

7.3.4 Class_constraint

L'entité **class_constraint** est une contrainte qui limite l'ensemble admissible d'instances d'une classe en contraignant plusieurs propriétés ou par plusieurs contraintes globales.

EXPRESS specification:

```
*)
ENTITY class_constraint
ABSTRACT SUPERTYPE OF (configuration_control_constraint)
SUBTYPE OF (constraint);
END_ENTITY; -- class_constraint
(*
```

7.3.5 Configuration_control_constraint

L'entité **configuration_control_constraint** permet de limiter l'ensemble des instances, appelées "instances référencées", qu'une instance particulière, appelée "instance de référence", peut référencer, directement ou indirectement, au moyen d'une chaîne de propriétés. L'instance de référence est toute instance d'une classe qui référence **configuration_control_constraint** au moyen de son attribut **constraints** ou hérite d'une classe qui le fait. L'entité **configuration_control_constraint** définit un attribut **precondition** (précondition) facultatif qui spécifie la condition imposée sur l'instance de référence pour que la restriction s'applique. Elle définit un attribut **postcondition** qui spécifie les ensembles admissibles de valeurs pour certaines propriétés de la classe d'instances référencées.

EXEMPLE Un *assemblage boulonné* est constitué de l'ensemble suivant d'éléments de fixation: un *élément de fixation à filetage externe*, un nombre quelconque de *rondelles* et un ou plusieurs *écrous*. Il existe différentes sortes de filetages, notamment *filetage de vis autotaraudeuse*, *filetage de vis à bois*, *filetage métrique externe*, *filetage métrique interne*, *filetage interne impérial* et *filetage externe impérial*. Supposons que l'on souhaite décrire un *assemblage boulonné métrique*. Il est nécessaire de s'assurer que, quelle que soit la structure précise de l'assemblage, l'*élément de fixation à filetage externe* et aussi tous les *écrous* impliqués dans l'assemblage ont un filetage métrique. Cela peut être réalisé en spécifiant dans la classe *assemblage boulonné métrique* l'entité **configuration_control_constraint** qui assure que tout élément de fixation décrit dans l'ISO 13584-511 référencé par une instance quelconque de cette classe, ou de n'importe laquelle de ses sous-classes, appartienne à des classes soit qui n'ont pas de valeurs pour la propriété *type of filetage* (par exemple *rondelle*), soit dont les valeurs appartiennent à l'ensemble: {*filetage externe métrique*, *filetage interne métrique*}.

NOTE 1 Les deux attributs **precondition** et **postcondition** peuvent seulement restreindre les propriétés dont le type de données est **non_quantitative_code_type**. Ces propriétés peuvent avoir une valeur qui leur est assignée soit au niveau de l'instance, soit au niveau de la classe si elles sont déclarées comme étant des propriétés valuées d'une classe, à savoir **sub_class_properties** dans une **class**.

NOTE 2 Il convient que les propriétés référencées dans l'attribut **precondition** soient applicables à la classe qui référence **configuration_control_constraint**.

NOTE 3 Dans **configuration_control_constraint**, les entités **filter** (filtre) servent aussi bien à représenter la précondition imposée sur l'instance de référence qu'à exprimer des contraintes imposées sur des instances référencées.

EXPRESS specification:

```
*)
ENTITY configuration_control_constraint
SUBTYPE OF (class_constraint);
    precondition: SET [0:?] OF filter;
    postcondition: SET [1:?] OF filter;
END_ENTITY; -- configuration_control_constraint
```

(*

Définitions des attributs:

precondition: les entités **filter** qui doivent être valides sur l'instance de référence pour que la restriction s'applique.

NOTE 4 Si l'ensemble des filtres est vide, la restriction s'applique à toute instance de référence.

postcondition: les entités **filter** qui doivent être valides sur une instance référencée pour être admissible comme référence.

7.3.6 Filter

L'entité **filter** est une entité **enumeration_constraint** qui limite le domaine admissible d'une propriété dont le type de données est soit **non_quantitative_code_type**, soit **non_quantitative_int_type**.

EXPRESS specification:

```

*)
ENTITY filter;
    referenced_property: property_BSU;
    domain: enumeration_constraint;
WHERE
    WR1: definition_available_implies (
        referenced_property,
        (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
        +'.NON_QUANTITATIVE_CODE_TYPE') IN TYPEOF(
        referenced_property.
        definition[1]\property_DET.domain))
    OR
        (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
        +'.NON_QUANTITATIVE_INT_TYPE') IN TYPEOF(
        referenced_property.
        definition[1]\property_DET.domain)));
    WR2: definition_available_implies (
        referenced_property,
        correct_constraint_type(domain,
        referenced_property.definition[1].domain));
END_ENTITY; -- filter
( *
```

Définitions des attributs:

referenced_property: la propriété dont le domaine des valeurs est limité par **filter**.

domain: l'entité **enumeration_constraint** qui limite le domaine des valeurs de la propriété référencée.

Propositions formelles:

WR1: le type de données de **referenced_property** doit être soit **non_quantitative_code_type**, soit **non_quantitative_int_type**.

WR2: l'attribut **domain** doit définir un domaine de valeurs qui peuvent limiter le domaine initial des valeurs de la propriété.

7.3.7 Integrity_constraint

L'entité **integrity_constraint** est une contrainte de propriété particulière qui permet d'expliciter le fait que pour une certaine classe particulière, en raison de la définition de la classe, et de toutes ses sous-classes, seule une restriction du domaine des valeurs spécifié par un type de données est autorisée pour une propriété.

EXEMPLE Dans le dictionnaire de référence défini pour les éléments de fixation dans l'ISO 13584-511, un/une *boulon/vis à filetage métrique* a une propriété *propriétés de tête* que peut prendre, comme valeur, un membre de n'importe quelle sous-classe de la classe de caractéristiques intrinsèques *tête*. Si le/la *boulon/vis à filetage métrique* est également un membre de la sous-classe *vis à tête hexagonale*, la propriété *propriétés de tête* peut seulement être membre de la classe de caractéristiques intrinsèques *tête hexagonale*, autrement le/la *boulon/vis à filetage métrique* ne peut pas être membre de la sous-classe *vis à tête hexagonale*.

NOTE Dans l'exemple ci-dessus, la contrainte d'intégrité ne change nullement la signification de la propriété *propriétés de tête* héritée de *boulon/vis à filetage métrique* dans *vis à tête hexagonale*. Elle explicite seulement le fait que dans le contexte de la sous-classe *vis à tête hexagonale*, seul un sous-ensemble des valeurs admissibles pour cette propriété dans le contexte de la classe *boulon/vis à filetage métrique* reste autorisé.

EXPRESS specification:

```
*)
ENTITY integrity_constraint
SUBTYPE OF (property_constraint);
    redefined_domain: domain_constraint;
WHERE
    WR1: definition_available_implies (constrained_property,
        correct_constraint_type(redefined_domain,
            constrained_property.definition[1].domain));
END_ENTITY; -- integrity_constraint
(*
```

Définitions des attributs:

redefined_domain: la contrainte qui s'applique sur le domaine des valeurs de la propriété contrainte.

Propositions formelles:

WR1: **redefined_domain** doit définir un domaine des valeurs qui limite le domaine initial des valeurs de la propriété.

7.3.8 Context_restriction_constraint

L'entité **context_restriction_constraint** est une **property_constraint** qui limite le domaine admissible des valeurs pour les paramètres de contexte dont dépend une propriété dépendant d'un contexte.

EXPRESS specification:

```
*)
ENTITY context_restriction_constraint
SUBTYPE OF (property_constraint);
    context_parameter_constraints: SET [1:?] OF
property_constraint;
WHERE
```



```

WR1: definition_available_implies(constrained_property,
    QUERY (cp < *SELF.context_parameter_constraints
        | NOT (cp.constrained_property IN
            constrained_property.definition[1].depends_on))=[]);
WR2: QUERY (cp < *SELF.context_parameter_constraints
    | NOT (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
        +'.INTEGRITY_CONSTRAINT') IN TYPEOF (cp)))=[];
WR3: definition_available_implies(constrained_property,
    'ISO13584_IEC61360_DICTIONARY_SCHEMA.DEPENDENT_P_DET'
    IN TYPEOF(constrained_property.definition[1]));
END_ENTITY; -- context_restriction_constraint
(*)

```

Définitions des attributs:

context_parameter_constraints: la contrainte qui s'applique sur le domaine de valeurs des paramètres de contexte.

Propositions formelles:

WR1: l'ensemble des propriétés dont le domaine est contraint par la propriété **context_parameter_constraints** doit être constitué des paramètres de contexte dont dépend la propriété contrainte.

WR2: toutes les **context_parameter_constraints** doivent être des entités **integrity_constraint**.

WR3: la propriété **constrained_property** doit être une propriété dépendant du contexte **dependent_P_DET**.

7.3.9 Domain_constraint

Une entité **domain_constraint** définit une contrainte qui limite le domaine des valeurs d'un type de données.

EXPRESS specification:

```

*)
ENTITY domain_constraint
ABSTRACT SUPERTYPE OF(ONEOF(
    subclass_constraint,
    entity_subtype_constraint,
    enumeration_constraint,
    range_constraint,
    string_size_constraint,
    string_pattern_constraint,
    cardinality_constraint
));
END_ENTITY; -- domain_constraint
(*)

```

7.3.10 Subclass_constraint

Une entité **subclass_constraint** limite le domaine de valeurs de **class_reference_type** à une ou plusieurs sous-classes de la classe qui définit son domaine initial.

EXPRESS specification:

```

*)
ENTITY subclass_constraint
SUBTYPE OF (domain_constraint);
    subclasses: SET [1:?] OF class_BSU;
END_ENTITY; -- subclass_constraint
( *

```

Définitions des attributs:

subclasses: les entités **class_BSU** qui redéfinissent la classe à laquelle la valeur de la propriété **constrained_property** doit appartenir.

7.3.11 Entity_subtype_constraint

Une entité **entity_subtype_constraint** limite le domaine de **entity_instance_type** à un sous-type de l'ENTITY qui définit son domaine initial.

EXPRESS specification:

```

*)
ENTITY entity_subtype_constraint
SUBTYPE OF (domain_constraint);
    subtype_names: SET [1:?] OF STRING;
END_ENTITY; -- entity_subtype_constraint
( *

```

Définitions des attributs:

subtype_names: l'ensemble des chaînes qui décrivent, dans le format de la fonction EXPRESS TYPEOF, les noms des types de données de l'entité EXPRESS qui doivent appartenir au résultat de la fonction EXPRESS TYPEOF lorsqu'elle est appliquée à une valeur qui référence la propriété redéfinie **constrained_property**.

7.3.12 Enumeration_constraint

Une entité **enumeration_constraint** limite le domaine des valeurs d'un type de données à une liste de valeurs définies dans l'extension. L'ordre défini par la liste est l'ordre recommandé pour des besoins de présentation. Une description particulière peut facultativement être associée à chaque valeur d'un sous-ensemble au moyen d'un **non_quantitative_int_type**, dont la $i^{\text{ème}}$ valeur décrit la signification de la $i^{\text{ème}}$ valeur du sous-ensemble.

Pour les sous-types de **number_type** qui sont associés à **dic_unit** et à des unités de substitution, et éventuellement à un **dic_unit_identifieur** et des identificateurs d'unités de substitution, la contrainte s'applique à la valeur correspondant à **dic_unit**, ou au seul **dic_unit_identifieur**. S'ils existent tous les deux, ils correspondent à la même unité.

Pour les sous-types de **number_type** qui sont associés à une monnaie, la contrainte s'applique à la monnaie spécifiée dans leur définition des types de données. Si aucune monnaie n'est spécifiée dans la définition des types de données, la contrainte ne doit pas être utilisée.

Pour les valeurs qui appartiennent à **translatable_string_type**, la contrainte s'applique à la chaîne qui est dans la langue source dans laquelle le domaine de la propriété était défini.

Cette langue source peut être définie dans l'attribut **source_language** de **administrative_data** de la propriété. Si cet attribut n'existe pas, cette langue source est supposée être connue de l'utilisateur du dictionnaire.

Si une autre **enumeration_constraint** est appliquée sur une propriété déjà associée à une **enumeration_constraint** dans une certaine superclasse, les deux contraintes s'appliquent. Ainsi, l'ensemble admissible des valeurs est l'intersection des deux sous-ensembles. Quant à l'ordre de présentation, et l'éventuelle signification associée à chaque valeur, seules les significations définies dans l'**enumeration_constraint** inférieure s'appliquent.

EXEMPLE 1 Si, dans la classe C1, cette propriété est associée à une **enumeration_constraint** dont l'attribut **subset** (sous-ensemble) est {1, 3, 5, 7}, alors dans la classe C1, et n'importe laquelle de ses sous-classes, la propriété P1 peut seulement prendre l'une des quatre valeurs suivantes: 1, 3, 5 ou 7.

EXEMPLE 2 Si le type de données de la propriété P1 est LIST [1:4] OF INTEGER, et si dans la classe C1, cette propriété est associée à une **enumeration_constraint** dont l'attribut **subset** est { {1}, {3, 5}, {7}, {1, 3, 7} }, alors dans la classe C1 et n'importe laquelle de ses sous-classes, la propriété P1 peut seulement prendre l'une des quatre valeurs suivantes: {1} ou {3, 5} ou {7} ou {1, 3, 7}.

EXPRESS specification:

```
*)
ENTITY enumeration_constraint
SUBTYPE OF (domain_constraint);
    subset: LIST [1:?] OF UNIQUE primitive_value;
    value_meaning: OPTIONAL non_quantitative_int_type;
WHERE
    WR1: (NOT(EXISTS(SELF.value_meaning)))
        OR
        (integer_values_in_range(1, SIZEOF(SELF.subset))
         = allowed_values_integer_types(SELF.value_meaning));
END_ENTITY; -- enumeration_constraint
(*
```

Définitions des attributs:

subset: la liste décrivant le sous-ensemble des valeurs qui sont admissibles comme valeurs possibles de la propriété identifiée par **constrained_property**.

value_meaning: l'ensemble des entités **dic_value** qui définissent la signification de chaque valeur appartenant au **subset**.

Propositions formelles:

WR1: si **value_meaning non_quantitative_int_type** existe, l'ensemble des **value_code** de ses **dic_value** doit se situer dans la plage 1.. SIZE_OF(subset).

7.3.13 Range_constraint

Une entité **range_constraint** limite le domaine des valeurs d'un type ordonné à un sous-ensemble de valeurs définies par une plage.

NOTE 1 Les chaînes ne sont pas considérées comme étant des types ordonnés et ne peuvent pas être contraintes par **range_constraint**.

Pour les sous-types de **number_type** qui sont associés à **dic_unit** et à des unités de remplacement, et éventuellement à un **dic_unit_identifieur** et des identificateurs d'unités de remplacement, la contrainte s'applique à la valeur correspondant à **dic_unit**, ou au seul **dic_unit_identifieur**. S'ils existent tous les deux, ils correspondent à la même unité.

Pour les sous-types de **number_type** qui sont associés à une monnaie, la contrainte s'applique à la monnaie spécifiée dans leur définition des types de données. Si aucune monnaie n'est spécifiée dans la définition des types de données, la contrainte ne doit pas être utilisée.

NOTE 2 Pour **non_quantitative_int_type**, la contrainte s'applique à l'attribut **value_code**.

EXPRESS specification:

```

*)
ENTITY range_constraint
SUBTYPE OF (domain_constraint);
    min_value, max_value: OPTIONAL NUMBER;
    min_inclusive, max_inclusive: OPTIONAL BOOLEAN;
WHERE
    WR1: min_value <= max_value;
    WR2: TYPEOF(min_value) = TYPEOF(max_value);
    WR3: NOT EXISTS (min_value) OR EXISTS (min_inclusive);
    WR4: NOT EXISTS (max_value) OR EXISTS (max_inclusive);
END_ENTITY; -- range_constraint
(*

```

Définitions des attributs:

min_value: le nombre définissant la borne inférieure de la plage des valeurs; une valeur inexistante signifie absence de borne inférieure.

max_value: le nombre définissant la borne supérieure de la plage de valeurs; une valeur inexistante signifie absence de borne supérieure.

min_inclusive: spécifie si **min_value** appartient à la plage ou non; une valeur inexistante signifie qu'il n'y a pas de borne inférieure.

max_inclusive: spécifie si **max_value** appartient à la plage ou non; une valeur inexistante signifie qu'il n'y a pas de borne supérieure.

Propositions formelles:

WR1: **min_value** doit être inférieure ou égale à **max_value**.

WR2: **min_value** et **max_value** doivent avoir les mêmes types de données.

WR3: si **min_value** a une valeur, **min_inclusive** doit aussi avoir une valeur.

WR4: si **max_value** a une valeur, **max_inclusive** doit aussi avoir une valeur.

7.3.14 String_size_constraint

Une entité **string_size_constraint** limite la longueur des valeurs STRING admissibles pour un type STRING, ou n'importe lequel de ses sous-types.

NOTE 1 Un domaine de valeurs de propriétés **string_type** est soit un **string_type**, soit un **non_translatable_string_type**, soit un **translatable_string_type**, soit un **URI_type**, soit un **non_quantitative_code_type**, soit un **date_data_type**, soit un **time_data_type**, soit un **date_time_data_type**.

NOTE 2 Pour **non_quantitative_code_type**, la contrainte s'applique au code.